



UNIVERSITAS INDONESIA

**RANCANG BANGUN SISTEM OPTIMISASI PEMAKAIAN BATUBARA
PADA PROSES PRODUKSI FERONIKEL**

SKRIPSI

**ALDRIAN RAFFI WICAKSONO
2106653256**

**FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK KOMPUTER
DEPOK
2025**



UNIVERSITAS INDONESIA

**RANCANG BANGUN SISTEM OPTIMISASI PEMAKAIAN BATUBARA
PADA PROSES PRODUKSI FERONIKEL**

SKRIPSI

**Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Teknik**

ALDRIAN RAFFI WICAKSONO

2106653256

**FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK KOMPUTER
DEPOK
JULI 2025**

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Aldrian Raffi Wicaksono

NPM : 2106653256

Tanda Tangan :



Tanggal : 04 Juni 2025

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :

Nama : Aldrian Raffi Wicaksono

NPM : 2106653256

Program Studi : Teknik Komputer

Judul Skripsi : Rancang Bangun Sistem Optimisasi Pemakaian Batubara
Pada Proses Produksi Feronikel

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Teknik pada Program Studi Teknik Komputer, Fakultas Teknik, Universitas Indonesia.

DEWAN PENGUJI

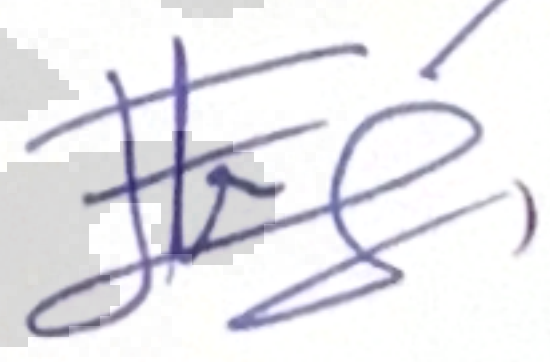
Pembimbing : Dr. Eng. Mia Rizkinia, S.T., M.T.

()

Penguji 1 : Dr. Prima Dewi Purnamasari, S.T, M.T., M.Sc.

()

Penguji 2 : Muhammad Firdaus Syawaludin Lubis, S.T., MT.,
Ph.D.

()

Ditetapkan di : Depok

Tanggal : 11 Juni 2025

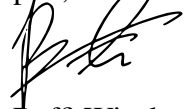
KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan skripsi dengan judul "Rancang Bangun Model Machine Learning untuk Optimisasi Parameter pada Proses Produksi Feronikel" ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu syarat untuk meraih kelulusan di Jurusan Teknik Komputer, Departemen Teknik Elektro, Universitas Indonesia. Penyusunan laporan ini tidak lepas dari bantuan dan dukungan banyak pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pembimbing yang telah memberikan bimbingan dan arahan yang sangat berharga dalam penulisan skripsi ini.
2. Orang tua penulis yang telah memberikan dukungan, doa, dan kasih sayang tanpa henti sepanjang perjalanan hidup penulis.
3. Teman-teman seangkatan di Teknik Komputer 2021 yang telah memberikan dukungan dan kebersamaan yang menyenangkan selama masa kuliah.
4. Teman-teman seperjuangan dalam bimbingan, yang bersama-sama menghadapi tantangan dalam proses penelitian dan penulisan skripsi.
5. Semua pihak yang telah memberikan kontribusi dalam bentuk dukungan, semangat, maupun motivasi selama ini, meskipun tidak dapat disebutkan satu per satu.

Penulis menyadari bahwa laporan skripsi ini masih jauh dari kesempurnaan, oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan untuk penyempurnaan di masa yang akan datang.

Depok, 3 Juni 2025



Aldrian Raffi Wicaksono

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Aldrian Raffi Wicaksono
NPM : 2106653256
Program Studi : Teknik Komputer
Fakultas : Teknik
Jenis Karya : Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Rancang Bangun Sistem Optimisasi Pemakaian Batubara Pada Proses Produksi Feronikel

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok

Pada tanggal : 04 Juni 2025

Yang menyatakan


(Aldrian Raffi Wicaksono)

ABSTRAK

Nama : Aldrian Raffi Wicaksono
Program Studi : Teknik Komputer
Judul : Rancang Bangun Sistem Optimisasi Pemakaian Batubara
Pada Proses Produksi Feronikel
Pembimbing : Dr. Eng. Mia Rizkinia, S.T., M.T.

Proses produksi feronikel di industri menggunakan teknologi *Rotary Kiln Electric Furnace* (RKEF) yang sangat intensif energi. Penelitian ini bertujuan mengoptimalkan parameter operasional RKEF untuk meningkatkan efisiensi energi sambil menjaga kualitas produk, menggunakan model prediktif *Ridge Regression* dan algoritma optimisasi. Dua metode utama, yaitu *Sequential Least Squares Programming* (SLSQP) dan *Genetic Algorithm* (GA), diterapkan dan dievaluasi berdasarkan efisiensi energi, kepatuhan *constraint*, kewajaran solusi, akurasi *output*, dan waktu komputasi.

Hasil menunjukkan SLSQP (Skenario e) unggul dalam kepatuhan *constraint* (0 pelanggaran) dan efisiensi komputasi (23.8 detik), serta menghasilkan solusi *input* yang lebih wajar (rata-rata *outlier* 19). Sementara itu, GA (Skenario d) mencapai efisiensi energi yang lebih tinggi (total efisiensi 1.85 Ton/jam) dan akurasi *output* lebih baik (rata-rata deviasi 4.69), namun dengan kewajaran *input* yang lebih rendah (rata-rata *outlier* 40.27) dan waktu komputasi jauh lebih lama (365.78 detik). Berdasarkan pertimbangan stabilitas operasional dan kecepatan, SLSQP dengan skenario e lebih direkomendasikan untuk implementasi industri.

Kata kunci:

Feronikel, *Rotary Kiln Electric Furnace*, *Machine Learning*, Optimisasi, Efisiensi Energi, *Ridge Regression*, *SLSQP*, *Genetic Algorithm*

ABSTRACT

Name : Aldrian Raffi Wicaksono
Study Program : Teknik Komputer
Title : Development of Optimization System for Coal Usage in
Ferronickel Production Process
Counsellor : Dr. Eng. Mia Rizkinia, S.T., M.T.

The ferronickel production process in the industry utilizes the highly energy-intensive rotary kiln electric furnace (RKEF) technology. This study aims to optimize the operational parameters of the RKEF to improve energy efficiency while maintaining product quality, using predictive models and optimization algorithms. Two main methods, namely Sequential Least Squares Programming (SLSQP) and Genetic Algorithm (GA), were applied and evaluated based on energy efficiency, constraint compliance, solution reasonableness, output accuracy, and computation time.

Results show SLSQP (Scenario e) excels in constraint compliance (0 violations) and computational efficiency (23.8 seconds), and produces more reasonable solutions (average *outlier* 19). Meanwhile, GA (Scenario d) achieved higher energy efficiency (total efficiency 1.85 Ton/hour) and better *output* accuracy (mean deviation 4.69), but with lower *input* reasonableness (mean *outlier* 40.27) and much longer computation time (365.78 seconds). Based on operational stability and speed considerations, SLSQP with scenario e is more recommended for industrial implementation.

Key words:

Ferronickel, *Rotary Kiln Electric Furnace*, *Machine Learning*, Optimization, Energy Efficiency, *Ridge Regression*, *SLSQP*, *Genetic Algorithm*

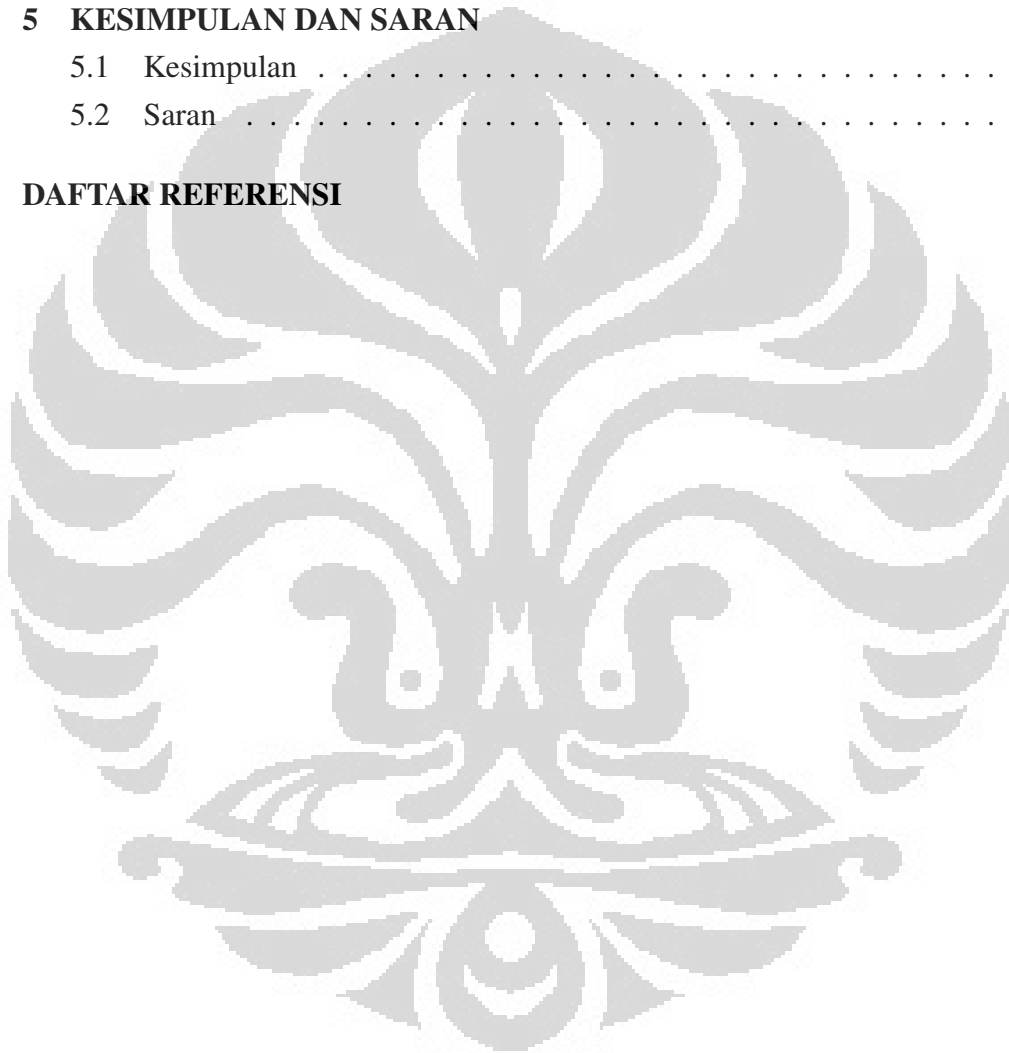
DAFTAR ISI

HALAMAN JUDUL	i
LEMBAR PERNYATAAN ORISINALITAS	ii
LEMBAR PENGESAHAN	iii
KATA PENGANTAR	iv
LEMBAR PERSETUJUAN PUBLIKASI ILMIAH	iv
ABSTRAK	vi
DAFTAR ISI	viii
DAFTAR GAMBAR	xii
DAFTAR TABEL	xiii
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	2
1.4 Batasan Masalah	3
1.5 Metodologi Penelitian	3
1.6 Sistematika Penulisan	4
2 LANDASAN TEORI	5
2.1 Mekanisme Proses Sistem RKEF pada Industri Feronikel	5
2.1.1 Proses Rotary Dryer	5
2.1.2 Proses Rotary Kiln	6
2.1.3 Proses Electrical Furnace	6
2.1.4 Peran Batubara dalam Produksi dan Kebutuhan Optimasi . .	6
2.2 Machine Learning	7
2.3 Tipe-tipe Machine Learning	7
2.3.1 Supervised learning (SL)	8
2.3.2 Unsupervised learning (UL)	8

2.3.3	Semi-supervised learning (SSL)	8
2.3.4	Reinforcement learning (RL)	8
2.4	Algoritma Machine Learning yang Digunakan	9
2.4.1	Regresi	9
2.4.2	Ridge Regression (RR)	10
2.5	Optimisasi, Constraint, dan Variabel	12
2.6	Klasifikasi Algoritma Optimisasi dalam Machine Learning	12
2.6.1	Exact Algorithms	13
2.6.2	Heuristics and Metaheuristics Algorithms	13
2.6.3	Surrogate-based optimization	14
2.6.4	Meta-optimization and hyperheuristics	15
2.7	Algoritma Optimasi yang Digunakan	16
2.7.1	Genetic Algorithm (GA)	16
2.7.2	Sequential Least Squares Quadratic Programming (SLSQP)	17
2.8	Metrik Evaluasi Model Machine Learning	20
2.8.1	MSE	20
2.8.2	RMSE	20
2.8.3	MAE	21
3	PERANCANGAN SISTEM MACHINE LEARNING UNTUK OPTIMASI PROSES PRODUKSI FERONIKEL	22
3.1	Dataset	22
3.1.1	Sumber Data	22
3.1.2	Variabel Pada Data	24
3.1.3	Frekuensi Data	28
3.1.4	Metode Pengambilan	28
3.1.5	Desain Arsitektur Sistem Optimisasi	29
3.2	Formulasi Permasalahan Optimisasi	31
3.2.1	Variabel Input (Parameter Proses yang Dioptimasi)	31
3.2.2	Fungsi Objektif: Meminimalkan Selisih Prediksi Terhadap Nilai Aktual	32
3.2.3	Batasan (Constraint): Rentang Nilai Parameter Proses	32
3.2.3.1	Batasan pada Variabel Input (<i>Input Constraints</i>)	33
3.2.3.2	Batasan pada Variabel Output (<i>Output Constraints</i>)	34
3.2.3.3	Perbandingan Pendekatan Konfigurasi Kendala (SLSQP vs GA)	35
3.3	Skenario Eksperimen	36
3.4	Struktur Eksperimen dan Tahapan Analisis	38

3.4.1	Tahap 1 – Evaluasi Antar-Skenario dengan Parameter <i>Default</i>	39
3.4.1.1	Mekanisme Evaluasi Tahap 1:	39
3.4.2	Tahap 2 – Perbandingan Lintas-Metode pada Skenario Terbaik	42
3.4.2.1	Mekanisme Evaluasi Tahap 2	43
3.5	Implementasi Constraint pada Kode	44
3.5.1	Genetic Algorithm (GA)	44
3.5.1.1	Constraint Selama Pembuatan Kromosom (Chromosome Creation)	44
3.5.1.2	Constraint Selama Mating (Crossover dan Mutasi)	45
3.5.1.3	Constraint Melalui Fungsi Fitness (Penalti)	46
3.5.2	Sequential Least Squares Programming (SLSQP)	47
3.5.2.1	Constraint Sebagai Batas (Bounds)	47
3.5.2.2	Constraint Sebagai Fungsi (Persamaan dan Ketidaksetaraan)	48
3.5.3	Perbedaan Penempatan <i>Constraint</i>	50
4	HASIL DAN PEMBAHASAN	53
4.1	Hasil Optimasi Menggunakan Genetic Algorithm (GA)	53
4.1.1	Perbandingan Efisiensi Energi	53
4.1.2	Evaluasi Kewajaran Solusi (Feasibility)	54
4.1.3	Evaluasi Pelanggaran Constraint	56
4.1.4	Evaluasi Waktu Komputasi	58
4.1.5	Evaluasi Deviasi Output	59
4.1.6	Skor dan Pemeringkatan Skenario GA	61
4.1.7	Analisis Skenario Terbaik GA	63
4.2	Hasil Optimasi Menggunakan Sequential Least Square Quadratic Programming (SLSQP)	65
4.2.1	Perbandingan Efisiensi Energi	65
4.2.2	Evaluasi Kewajaran Solusi (Feasibility)	67
4.2.3	Evaluasi Pelanggaran Constraint	69
4.2.4	Evaluasi Waktu Komputasi	70
4.2.5	Evaluasi Deviasi Output	72
4.2.6	Skor dan Pemeringkatan Skenario SLSQP	74
4.2.7	Analisis Skenario Terbaik SLSQP	76
4.3	Perbandingan Hasil Optimisasi: SLSQP vs GA	77
4.3.1	Perbandingan Efisiensi Energi	77
4.3.2	Perbandingan Kewajaran Solusi (Jumlah Outlier)	78
4.3.3	Perbandingan Jumlah Pelanggaran Constraint	80

	xi
4.3.4 Perbandingan Durasi Komputasi	80
4.3.5 Perbandingan Akurasi Output (Deviasi)	81
4.3.6 Perbandingan Skor Akhir Antar Metode	82
4.4 Dampak terhadap Lingkungan, Masyarakat, dan/atau Bidang Lain .	83
4.4.1 Dampak terhadap Lingkungan	84
4.4.2 Dampak Sosial dan Kemasyarakatan	84
4.4.3 Dampak terhadap Industri dan Teknologi	85
5 KESIMPULAN DAN SARAN	86
5.1 Kesimpulan	86
5.2 Saran	88
DAFTAR REFERENSI	90



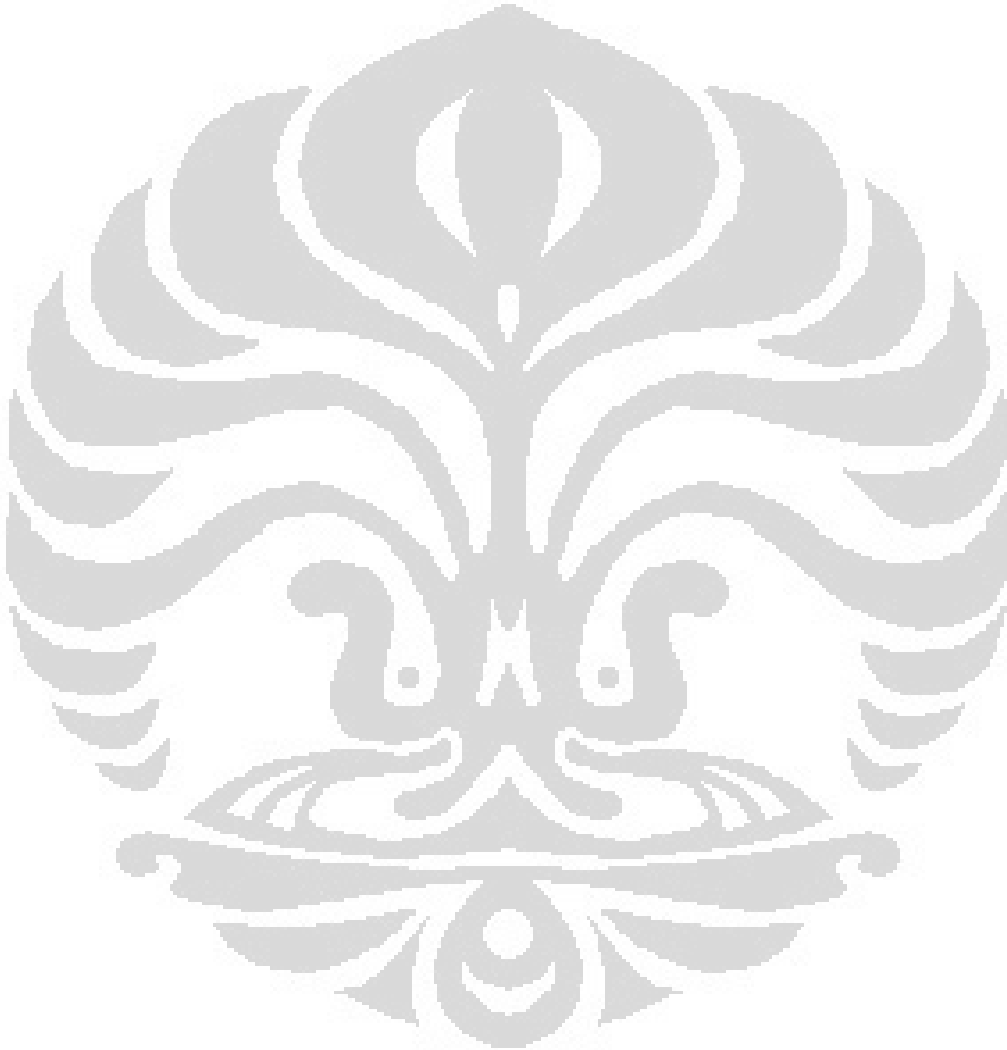
DAFTAR GAMBAR

Gambar 2.1	Ilustrasi sistem RKEF [1]	5
Gambar 2.2	Representasi Geometris Ridge Regression[2]	11
Gambar 4.1	Nilai Efisiensi Metode GA	54
Gambar 4.2	Nilai Kewajaran Input Metode GA	56
Gambar 4.3	Jumlah Pelanggaran Constraint Metode GA	57
Gambar 4.4	Durasi Komputasi Metode GA	59
Gambar 4.5	Nilai Deviasi Output Metode GA	61
Gambar 4.6	Total Skor Skenario Metode GA	62
Gambar 4.7	Nilai Efisiensi pada Metode SLSQP	65
Gambar 4.8	Nilai Kewajaran Input Skenario SLSQP	67
Gambar 4.9	Durasi Komputasi Skenario SLSQP	71
Gambar 4.10	Nilai Deviasi Output Metode SLSQP	72
Gambar 4.11	Total Skor Skenario Metode SLSQP	75
Gambar 4.12	Efisiensi Energi - SLSQP vs GA	78
Gambar 4.13	Jumlah Outlier - SLSQP vs GA	79
Gambar 4.14	Jumlah Pelanggaran Konstrain - SLSQP vs GA	80
Gambar 4.15	Durasi Komputasi - SLSQP vs GA	81
Gambar 4.16	Deviasi Output - SLSQP vs GA	82

DAFTAR TABEL

Tabel 3.1	Daftar Variabel Input Sistem	24
Tabel 3.1	Daftar Variabel Input Sistem (lanjutan)	25
Tabel 3.1	Daftar Variabel Input Sistem (lanjutan)	26
Tabel 3.2	Daftar Variabel Output Sistem	26
Tabel 3.2	Daftar Variabel Output Sistem (lanjutan)	27
Tabel 3.3	Informasi Shift	28
Tabel 3.4	Batasan Variabel Input (dalam ruang ternormalisasi)	34
Tabel 3.5	Batasan Variabel Output (dalam satuan asli, sebelum normalisasi)	35
Tabel 3.6	Daftar Skenario Eksperimen dan Keterangan	38
Tabel 3.7	Parameter Default Optimisasi untuk Tahap Evaluasi Antar-Skenario	42
Tabel 3.8	Perbedaan Penempatan <i>Constraint</i> antara GA dan SLSQP .	51
Tabel 4.1	Nilai Efisiensi Konsumsi Energi Tiap Skenario	53
Tabel 4.2	Jumlah Outlier dari Hasil Optimasi Tiap Skenario	55
Tabel 4.3	Jumlah Pelanggaran Constraint per Skenario	57
Tabel 4.4	Durasi Waktu Komputasi per Skenario (dalam detik)	58
Tabel 4.5	Nilai Deviasi Output Tiap Skenario	60
Tabel 4.6	Nilai Metrik Skenario GA Sebelum Pembobotan	61
Tabel 4.7	Nilai Metrik Skenario GA Setelah Pembobotan dan Total Skor	62
Tabel 4.8	Hasil Evaluasi Efisiensi Energi pada Metode GA	65
Tabel 4.9	Jumlah Outlier pada Hasil Optimisasi SLSQP per Variabel	68
Tabel 4.10	Jumlah Pelanggaran Constraint pada Setiap Skenario (SLSQP)	70
Tabel 4.11	Waktu Komputasi Optimisasi per Skenario (SLSQP)	71
Tabel 4.12	Nilai Deviasi Output Metode SLSQP	72
Tabel 4.13	Nilai Metrik Skenario SLSQP Sebelum Pembobotan	74
Tabel 4.14	Nilai Metrik Skenario SLSQP Setelah Pembobotan dan Total Skor	74
Tabel 4.15	Perbandingan Nilai Efisiensi Energi	77
Tabel 4.16	Perbandingan Jumlah Outlier Solusi Input	79

Tabel 4.17	Perbandingan Jumlah Pelanggaran Constraint	80
Tabel 4.18	Perbandingan Durasi Komputasi	80
Tabel 4.19	Perbandingan Deviasi Output	81
Tabel 4.20	Rekapitulasi Perbandingan Metode	83



DAFTAR KODE

1	Pembuatan Kromosom	45
2	Perkawinan Kromosom	46
3	Perhitungan <i>Fitness</i>	47
4	Inisialisasi Nilai <i>Fixed</i> dan <i>Optimized Inputs</i> untuk SLSQP	48
5	Konfigurasi dan Penggunaan Constraint pada Fungsi SLSQP	50



BAB 1

PENDAHULUAN

1.1 Latar Belakang

Industri feronikel memiliki peran strategis dalam perekonomian Indonesia, mengingat negara ini merupakan salah satu produsen nikel terbesar di dunia dengan kontribusi sekitar 30% dari total produksi global [3]. Produk feronikel digunakan sebagai bahan utama dalam produksi stainless steel, menjadikannya komoditas penting dalam ekspor nasional dan hilirisasi industri logam. Dalam rangka meningkatkan nilai tambah serta efisiensi produksi, pemerintah Indonesia mendorong percepatan hilirisasi mineral termasuk feronikel melalui kebijakan strategis.

Proses produksi feronikel umumnya menggunakan teknologi Rotary Kiln Electric Furnace (RKEF), yang melibatkan beberapa tahapan seperti pengeringan, reduksian awal, dan peleburan. Walaupun teknologi ini cukup efisien dari sisi output logam, prosesnya sangat intensif energi, khususnya konsumsi batubara dan listrik yang berdampak besar terhadap biaya produksi dan emisi karbon. Data historis menunjukkan konsumsi batubara dapat mencapai lebih dari 9.700 ton per bulan, menjadi sumber dominan emisi karbon dioksida (CO_2) dan tantangan utama dalam konteks keberlanjutan.

Untuk menjawab tantangan tersebut, pendekatan optimisasi parameter proses menjadi sangat penting. Optimisasi bertujuan untuk mencari konfigurasi operasional terbaik yang dapat meminimalkan konsumsi energi tanpa mengorbankan kualitas produk. Dalam konteks ini, metode optimisasi berbasis numerik seperti Sequential Least Squares Programming (SLSQP) dan metode evolutionary seperti Genetic Algorithm (GA) menawarkan solusi menjanjikan karena mampu menangani permasalahan nonlinier dengan banyak variabel dan batasan.

Kedua metode ini memerlukan fungsi objektif yang andal, yang dalam penelitian ini dibangun dari model prediktif berbasis machine learning. Model tersebut memetakan hubungan kompleks antar parameter proses dan keluaran produksi, seperti kadar nikel (`ni_met`) dan konsumsi bahan bakar (`total_fuel`). Dengan mengintegrasikan model prediktif ke dalam kerangka optimisasi, SLSQP dan GA dapat mengevaluasi ribuan kemungkinan konfigurasi parameter untuk mencari solusi yang paling efisien dan layak secara operasional.

Penerapan machine learning dalam konteks ini bukanlah hal baru. Studi se-

belumnya oleh Harmse dkk. [4] menunjukkan bahwa integrasi model prediksi ke dalam sistem pengendalian tambang dapat menurunkan konsumsi energi sistem ventilasi hingga 2.9%, menghasilkan penghematan signifikan secara ekonomi. Di sisi lain, Saldaña dkk. [5] berhasil mengoptimalkan proses penggilingan pada tambang menggunakan pendekatan serupa, menghasilkan peningkatan output 4.42% dan pengurangan energi 7.62%.

Dengan latar belakang tersebut, penelitian ini bertujuan untuk menerapkan pendekatan optimisasi berbasis SLSQP dan GA terhadap parameter-parameter proses produksi feronikel dalam skema RKEF, dengan tujuan utama mengurangi konsumsi bahan bakar serta menjaga kualitas metalurgi produk. Fokus penelitian mencakup perbandingan performa kedua metode dari sisi efisiensi energi, kestabilan proses, serta waktu komputasi, untuk menghasilkan pendekatan yang optimal dan aplikatif dalam konteks industri berat.

1.2 Rumusan Masalah

1. Belum tersedianya optimisasi parameter operasional berbasis *machine learning* dari proses produksi feronikel untuk meningkatkan efisiensi.
2. Proses optimalisasi secara manual menjadi sulit dilakukan karena banyaknya batasan (constraints) dan kompleksitas sistem, sehingga diperlukan pendekatan berbasis simulasi atau pemodelan dengan algoritma yang tepat.
3. Belum ada evaluasi komprehensif terhadap efektivitas berbagai metode optimisasi dalam menangani kompleksitas sistem produksi feronikel, sehingga belum diketahui metode yang paling sesuai untuk diterapkan.

1.3 Tujuan

1. Merancang dan mengembangkan model optimisasi berbasis machine learning untuk menentukan parameter operasional yang optimal pada proses produksi feronikel.
2. Menentukan kombinasi parameter operasional yang optimal guna meningkatkan efisiensi produksi dengan mempertimbangkan batasan teknis dan kualitas produk.
3. Mengevaluasi dan membandingkan performa beberapa algoritma optimisasi dalam menangani kompleksitas proses produksi feronikel, untuk memilih metode yang paling efektif dan efisien.

1.4 Batasan Masalah

1. Fokus penelitian pada rangkaian proses Rotary Kiln Electric Furnace (RKEF) dalam produksi feronikel, tanpa mencakup tahapan lain di luar RKEF.
2. Optimasi yang dilakukan terbatas pada konsumsi bahan bakar, khususnya batu bara, guna mengurangi pengeluaran biaya operasional.
3. Data yang digunakan berasal dari parameter operasional pabrik feronikel yang telah tersedia.
4. Sistem optimisasi berbasis model prediktif dikembangkan dengan asumsi tetap, tanpa mempertimbangkan variabel eksternal seperti fluktuasi harga bahan bakar atau regulasi pemerintah.

1.5 Metodologi Penelitian

1. Studi literatur
Melakukan kajian pustaka dengan membaca dan mempelajari penelitian yang relevan untuk menemukan implementasi yang tepat terkait penelitian yang akan dilakukan.
2. Konsultasi dengan pihak operasional pabrik
Melakukan diskusi dengan pihak operasional pabrik mengenai interpretasi data yang tersedia untuk memastikan pemahaman yang tepat terhadap parameter operasional yang akan digunakan.
3. Konsultasi dengan dosen pembimbing
Meminta saran dan arahan dari dosen pembimbing untuk memastikan penelitian dilakukan sesuai dengan tujuan yang direncanakan.
4. Perancangan sistem
Mendefinisikan alur proses pembuatan dan pengujian sistem, termasuk arsitektur sistem dan komponen model prediktif serta algoritma optimisasi yang akan diterapkan.
5. Implementasi sistem
Mengimplementasikan rancangan sistem ke dalam model prediktif. Proses ini melibatkan pelatihan model dengan data yang tersedia, evaluasi akurasi model, serta penerapan algoritma optimisasi pada parameter operasional.

6. Analisis hasil

Menganalisis data hasil pengujian, membandingkan akurasi dari berbagai model prediktif dan algoritma optimisasi yang telah diterapkan untuk menentukan solusi terbaik.

7. Kesimpulan

Menyusun kesimpulan berdasarkan hasil analisis data pengujian dan memberikan rekomendasi untuk penelitian selanjutnya.

1.6 Sistematika Penulisan

Sistematika penulisan laporan adalah sebagai berikut:

- **Bab 1 PENDAHULUAN**

Bab ini membahas latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, metodologi penelitian, serta sistematika penulisan laporan.

- **Bab 2 LANDASAN TEORI**

Bab ini menyajikan tinjauan pustaka yang relevan dengan penelitian, mencakup konsep dasar proses produksi feronikel, model prediktif berbasis machine learning, serta metode optimisasi yang digunakan dalam penelitian ini.

- **Bab 3 PERANCANGAN SISTEM MACHINE LEARNING UNTUK OPTIMASI PROSES PRODUKSI FERONIKEL**

Bab ini menjelaskan rancangan eksperimen optimisasi berbasis model prediktif. Pembahasan mencakup parameter yang dioptimasi, formulasi masalah optimisasi, penggunaan constraint, serta metode dan skenario optimisasi yang digunakan.

- **Bab 4 HASIL DAN PEMBAHASAN**

Bab ini menyajikan hasil eksperimen optimisasi dan analisisnya. Termasuk di dalamnya evaluasi performa kedua metode optimisasi SLSQP dan Genetic Algorithm.

- **Bab 5 KESIMPULAN DAN SARAN**

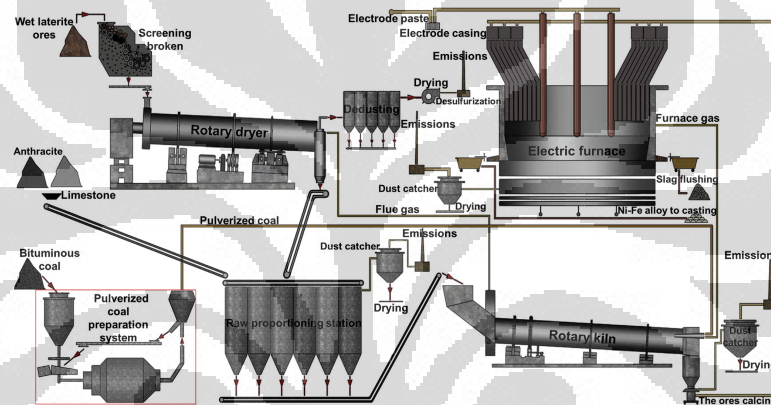
Bab ini berisi kesimpulan dari hasil penelitian serta saran untuk pengembangan dan penelitian lebih lanjut.

BAB 2

LANDASAN TEORI

2.1 Mekanisme Proses Sistem RKEF pada Industri Feronikel

Menurut Peng [6], proses rotary kiln-electric furnace (RKEF) merupakan salah satu teknologi yang paling banyak digunakan dalam produksi feronikel. Skema proses RKEF yang melibatkan rotary dryer, rotary kiln, dan electric furnace dapat dilihat pada Gambar 2.1.



Gambar 2.1: Ilustrasi sistem RKEF [1]

Proses RKEF terdiri dari tiga tahap utama: tahap dehidrasi menggunakan rotary dryer, tahap roasting dan pra-reduksi di rotary kiln, serta tahap reduksi dan peleburan di electric furnace. Proses ini melibatkan berbagai fenomena fisika dan kimia, seperti dehidrasi (penghilangan kelembapan bebas, air kristal, dan air yang terikat secara kimia), reaksi oksidoreduksi, reaksi disosiasi, reaksi penggantian, reaksi slagging, dan pemisahan slag dengan alloy [7].

2.1.1 Proses Rotary Dryer

Dalam proses rotary dryer, bijih laterit basah (biasanya mengandung 23% kelembapan bebas dan 12% air yang terikat secara kimia) terlebih dahulu dipecah menjadi partikel-partikel kecil melalui proses penyaringan dan penghancuran, kemudian dimasukkan ke dalam rotary dryer. Untuk menghilangkan kelembapan bebas dari bijih basah, digunakan panas limbah gas buang dari rotary kiln dan pembakaran batubara bituminus, yang memanaskan bijih hingga mencapai suhu dehidrasi (yaitu

220 °C). Setelah melalui proses rotary dryer, bijih laterit semi-kering (dengan 7% sisa kelembapan dan 12% air yang terikat secara kimia) dicampur dengan batu kapur dan antrasit di stasiun proporsi bahan baku. Selanjutnya, menggunakan teknik aglomerasi, partikel bijih laterit yang mengandung nikel berhasil dipulihkan.

2.1.2 Proses Rotary Kiln

Dalam proses rotary kiln, menggunakan gas furnace dari electric furnace dan batubara bituminus sebagai bahan bakar, kelembapan bebas yang tersisa sepenuhnya diuapkan. Selanjutnya, dalam proses pra-reduksi dan pemanggangan, air kristal dan air yang terikat secara kimia dari bijih laterit dihilangkan pada suhu tinggi (yaitu 290–820°C), dan dekomposisi batu kapur terjadi pada rentang suhu 820–1000°C. Bijih laterit yang telah dipanggang kemudian dipindahkan ke dalam electric furnace untuk proses reduksi peleburan.

Sementara itu, gas buang dari rotary kiln dialirkan ke rotary dryer untuk mengeringkan bijih basah. Selain itu, karbon dioksida akan direduksi oleh antrasit menjadi karbon monoksida yang selanjutnya mereduksi oksida logam pada suhu sekitar 220°C. Oleh karena itu, sebagian besar oksida logam direduksi oleh karbon monoksida di dalam rotary kiln, sementara hanya sebagian kecil oksida logam yang direduksi oleh antrasit.

2.1.3 Proses Electrical Furnace

Dalam proses electric furnace, karena pengaruh antrasit dan kalsium oksida (CaO), oksida logam akan mengalami reaksi reduksi dan reaksi pembentukan terak. Feronikel dan terak kemudian dipisahkan. Sementara itu, gas furnace (yang mengandung karbon monoksida) juga diproduksi. Setelah menjalani proses pemurnian, gas furnace ini didaur ulang ke dalam rotary kiln sebagai bahan bakar. Selain itu, untuk menghasilkan alloy ferronikel dengan kandungan nikel tinggi, reduksi oksida nikel dilakukan sepenuhnya dengan reduksi terbatas pada oksida besi. Sementara itu, kalsium oksida (CaO) dapat mengubah oksida nikel/besi dari mineral yang meningkatkan aktivitas oksida logam, meningkatkan rasio pemulihan nikel/besi, dan berkontribusi pada pemisahan terak/alloy melalui penurunan titik leleh terak.

2.1.4 Peran Batubara dalam Produksi dan Kebutuhan Optimasi

Batubara memegang peranan krusial dalam keseluruhan proses produksi feronikel melalui sistem RKEF, terutama sebagai sumber energi dan agen pereduksi. Pada

tahap rotary dryer, batubara bituminus digunakan sebagai bahan bakar tambahan untuk membantu proses pengeringan bijih laterit basah, melengkapi panas limbah dari rotary kiln. Selanjutnya, di dalam rotary kiln, batubara bituminus dan antrasit tidak hanya berfungsi sebagai bahan bakar utama untuk mencapai suhu tinggi yang diperlukan untuk pra-reduksi dan dekomposisi batu kapur, tetapi juga berperan sebagai agen pereduksi. Karbon dari batubara (khususnya antrasit) bereaksi untuk menghasilkan karbon monoksida (CO), yang kemudian mereduksi oksida-oksida logam dalam bijih.

Mengingat perannya yang sentral sebagai sumber energi dan pereduksi, konsumsi batubara memiliki dampak signifikan terhadap biaya operasional dalam produksi feronikel. Oleh karena itu, optimasi penggunaan batubara menjadi sangat penting. Optimasi ini dapat mencakup peningkatan efisiensi pembakaran dan pengelolaan rasio batubara dan mineral yang tepat.

2.2 Machine Learning

Machine learning (ML) adalah cabang dari kecerdasan buatan (AI) yang memungkinkan komputer untuk meningkatkan kinerjanya dengan pengalaman, khususnya dalam tugas-tugas tertentu, berdasarkan data pelatihan yang relevan. Secara umum, ML bertujuan untuk mengotomatiskan proses pembuatan model analitis untuk melakukan tugas kognitif. Hal ini dicapai melalui penerapan algoritma yang secara iteratif belajar dari data, memungkinkan komputer untuk menemukan wawasan tersembunyi dan pola tanpa perlu diprogram secara eksplisit. Terutama dalam tugas yang melibatkan data berdimensi tinggi. Dengan belajar dari perhitungan sebelumnya dan mengekstrak pola, ML dapat membantu menghasilkan keputusan yang dapat diandalkan dan dapat diulang [8].

2.3 Tipe-tipe Machine Learning

Dalam bidang machine learning, terdapat berbagai tipe yang digunakan tergantung pada jenis data yang digunakan untuk melatih sistem. Menurut F.A.H Alnuaimi dan H.K. Albaldawi (2024), empat kategori utama machine learning yang telah dikembangkan meliputi supervised learning, unsupervised learning, semi-supervised learning, dan reinforcement learning [9].

2.3.1 Supervised learning (SL)

Supervised learning (SL) adalah pendekatan yang menggunakan data berlabel untuk melatih sistem dalam memprediksi hasil berdasarkan data pelatihan. Proses ini mirip dengan cara manusia belajar di bawah bimbingan instruktur. Dalam SL, terdapat dua kategori utama, yaitu regresi, yang digunakan untuk memahami hubungan antara variabel dependen dan independen, seperti dalam ramalan pasar atau cuaca, dan klasifikasi, yang digunakan untuk mengelompokkan data ke dalam kategori yang telah ditentukan sebelumnya, seperti dalam pengklasifikasian email menjadi "spam" atau "bukan spam".

2.3.2 Unsupervised learning (UL)

Unsupervised learning, melibatkan pelatihan tanpa mengetahui output, menggunakan hanya sampel input atau label. Teknik ini berguna untuk menemukan pola tersembunyi dalam data dan membuat kluster data secara otomatis. Dua jenis utama dari unsupervised learning adalah klusterisasi dan analisis asosiasi, yang digunakan untuk mengidentifikasi pola yang tidak diketahui dalam data, seperti pada sistem rekomendasi di toko online.

2.3.3 Semi-supervised learning (SSL)

Semi-supervised learning (SSL) menggabungkan pendekatan supervised dan unsupervised untuk meningkatkan akurasi pembelajaran. SSL sangat berguna dalam situasi dengan data berlabel yang terbatas namun memiliki banyak data yang tidak berlabel. Teknik ini diterapkan dalam berbagai bidang, seperti analisis suara dan klasifikasi teks.

2.3.4 Reinforcement learning (RL)

Reinforcement learning (RL) adalah bentuk machine learning yang memungkinkan mesin untuk menentukan perilaku optimal guna mencapai tujuan yang diinginkan. RL mengandalkan eksperimen trial-and-error, di mana agen menerima umpan balik berupa penghargaan atau hukuman, dan dengan pengalaman, agen ini semakin baik dalam membuat keputusan untuk mencapai tujuannya. Salah satu contoh aplikasi RL adalah dalam kendaraan otonom

2.4 Algoritma Machine Learning yang Digunakan

2.4.1 Regresi

Regresi adalah metode statistika yang digunakan untuk memodelkan hubungan antara satu atau lebih variabel independen (prediktor) dengan satu variabel dependen (respons). Tujuan utama dari regresi adalah untuk memprediksi nilai variabel dependen berdasarkan nilai variabel independen, atau untuk memahami bagaimana perubahan pada variabel independen memengaruhi variabel dependen [10].

Beberapa **algoritma regresi** yang umum digunakan meliputi:

- Regresi Linier Sederhana

Merupakan model regresi yang paling dasar, di mana hubungan antara variabel independen (X) dan variabel dependen (Y) dimodelkan sebagai garis lurus. Persamaan matematisnya adalah:

$$Y = \beta_0 + \beta_1 X + \varepsilon \quad (2.1)$$

di mana β_0 adalah *intercept*, β_1 adalah *slope*, dan ε adalah *error term* [10].

- Regresi Linier Berganda

Merupakan perluasan dari regresi linier sederhana, yang melibatkan dua atau lebih variabel independen untuk memprediksi variabel dependen. Persamaan umumnya adalah:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon \quad (2.2)$$

di mana $X_1, X_2, \dots, X_p$ adalah variabel independen dan $\beta_0, \beta_1, \dots, \beta_p$ adalah koefisien regresi [10].

- Regresi Polinomial

Memodelkan hubungan antara variabel independen dan dependen sebagai polinomial berderajat n . Ini memungkinkan model untuk menangkap hubungan non-linier. Persamaannya dapat ditulis sebagai:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_n X^n + \varepsilon \quad (2.3)$$

- Regresi Logistik

Meskipun namanya "regresi", regresi logistik sebenarnya digunakan untuk masalah klasifikasi biner, di mana variabel dependen adalah kategorikal (mis-

alnya, 0 atau 1). Ini memodelkan probabilitas bahwa suatu peristiwa akan terjadi [11].

- Support Vector Regression (SVR)

Merupakan adaptasi dari Support Vector Machine (SVM) untuk masalah regresi. SVR mencari *hyperplane* yang paling cocok untuk data dengan *margin error* tertentu [12].

- Decision Tree Regression

Menggunakan struktur pohon keputusan untuk memprediksi nilai variabel dependen. Pohon ini dibagi berdasarkan fitur-fitur input hingga mencapai simpul daun yang berisi nilai prediksi [13].

- Random Forest Regression

Merupakan metode *ensemble learning* yang membangun banyak pohon keputusan dan menggabungkan prediksi mereka untuk menghasilkan hasil yang lebih akurat dan stabil dibandingkan dengan pohon keputusan tunggal [14].

2.4.2 Ridge Regression (RR)

A. E. Hoerl pertama kali mengusulkan pada tahun 1962 bahwa untuk mengatasi inflasi dan ketidakstabilan umum yang terkait dengan estimasi metode kuadrat terkecil (least squares), dapat digunakan pendekatan berikut:

$$\hat{\beta} = [X'X + kI]^{-1}X'Y, \quad k \geq 0 \quad (2.4)$$

$$= WX'Y \quad (2.5)$$

Pendekatan ini menghasilkan kumpulan estimasi untuk $k \geq 0$ yang memiliki kesamaan matematis dengan fungsi respons kuadratik. Oleh karena itu, pendekatan ini disebut sebagai "ridge regression" [15].

Hoerl dan Kennard (1970) menjelaskan bahwa potensi ketidakstabilan pada estimasi metode kuadrat terkecil (Least Square Estimator)

$$\hat{\beta} = (X'X)^{-1}X'Y \quad (2.6)$$

dapat diperbaiki dengan menambahkan konstanta kecil λ pada elemen diagonal matriks $X'X$ sebelum melakukan inversi. Hal ini menghasilkan estimator Ridge Regression sebagai berikut:

$$\hat{\beta}_{ridge} = (X'X + \lambda I_p)^{-1} X'Y \quad (2.7)$$

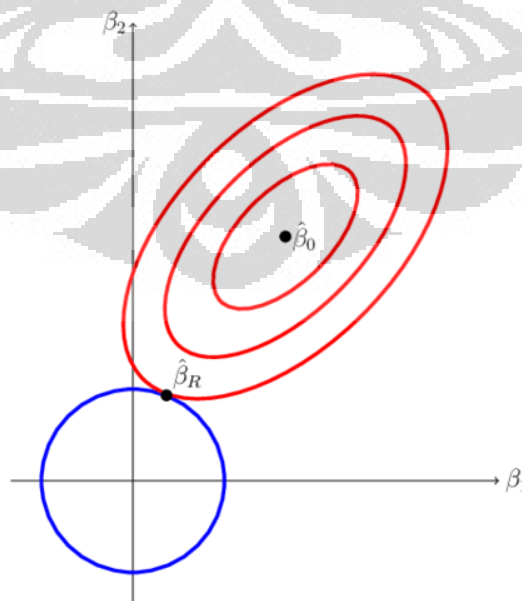
Ridge Regression menerapkan batasan khusus pada parameter β , di mana $\hat{\beta}_{ridge}$ dipilih untuk meminimalkan jumlah kuadrat penalti (*penalized sum of squares*):

$$\hat{\beta}_{ridge} = \sum_{i=1}^n \left(y_i - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (2.8)$$

Persamaan ini setara dengan meminimalkan $\sum_{i=1}^n \left(y_i - \sum_{j=1}^p x_{ij} \beta_j \right)^2$ dengan syarat bahwa, untuk suatu $c > 0$, $\sum_{j=1}^p \beta_j^2 < c$. Artinya, Ridge Regression membatasi jumlah kuadrat dari koefisien (β_j) dalam model linier.

Dalam kasus ini, Ridge Regression tidak hanya meminimalkan *residual sum of squares* (RSS) tetapi juga menambahkan penalti pada nilai koefisien β_j . Penalty ini berupa konstanta λ (*pre-chosen constant*) dikalikan dengan norma kuadrat dari vektor β . Dengan demikian, jika nilai β_j menjadi terlalu besar, fungsi optimasi akan dikenai penalti. Hal ini mendorong nilai β_j untuk lebih kecil atau mendekati nol guna mengurangi pengaruh penalti.

Ridge Regression, juga dikenal sebagai *L2-regularized regression*, sering digunakan dalam situasi di mana jumlah fitur (p) lebih besar daripada jumlah sampel (n). Metode ini meminimalkan RSS dengan menambahkan penalti L_2 yang mengecilkan nilai koefisien regresi ke arah nol. Penalty ini mengurangi varians pada koefisien dengan sedikit meningkatkan bias, sehingga menghasilkan estimasi yang lebih stabil ketika terjadi korelasi antar prediktor [16].



Gambar 2.2: Representasi Geometris Ridge Regression[2]

Pada Gambar 2.2, elips merepresentasikan kontur RSS, di mana elips bagian dalam menunjukkan nilai RSS yang lebih kecil. Nilai RSS diminimalkan pada estimasi metode kuadrat terkecil (OLS). Estimasi Ridge diperoleh pada titik di mana elips dan lingkaran saling bersentuhan.

Secara garis besar, Ridge Regression memberikan bobot lebih besar pada komponen utama (*principal components*) yang mendominasi variasi data, sehingga metode ini efektif dalam situasi di mana hanya sedikit komponen utama yang menjelaskan sebagian besar variasi prediktor [16].

Dalam data berdimensi tinggi, Ridge Regression menawarkan solusi konsisten untuk masalah prediksi. Penalti L_2 memungkinkan metode ini menangkap struktur data yang kompleks tanpa memerlukan seleksi variabel eksplisit seperti pada LASSO [16]. Ridge Regression juga sering menunjukkan performa yang lebih baik dibandingkan metode berbasis *sparsity*, terutama ketika data memiliki korelasi tinggi antar prediktor.

2.5 Optimisasi, Constraint, dan Variabel

Optimisasi, khususnya *Global Optimization* (GO), adalah proses menemukan solusi terbaik secara keseluruhan untuk suatu masalah [17]. Jörg Stork mengatakan dalam konteks minimisasi, tujuan GO adalah menemukan nilai-nilai variabel keputusan yang meminimalkan nilai fungsi objektif. Permasalahan ini muncul dari sistem fisik yang kompleks, proses bisnis, dan aplikasi kecerdasan buatan. Secara matematis, optimisasi global berusaha menemukan lokasi $x^* \in \mathbb{R}^n$ yang meminimalkan fungsi $f(x)$ dalam ruang pencarian global $\Phi = \{x | x_l \leq x \leq x_u\}$, di mana x_l dan $x_u \in \mathbb{R}^n$ adalah batas bawah dan atas eksplisit yang terbatas pada x .

Dalam masalah optimisasi, seringkali terdapat *constraint* (batasan) yang perlu dipertimbangkan. Teknik penanganan batasan ini dapat menjadi bagian langsung dari algoritma optimisasi, atau ditambahkan pada algoritma yang dirancang untuk meminimalkan fungsi objektif. Batasan ini dapat diintegrasikan dengan menyesuaikan nilai *fitness*, misalnya, dengan menambahkan *penalty terms*.

2.6 Klasifikasi Algoritma Optimisasi dalam Machine Learning

Dalam tulisannya Jörg Stork dan rekannya menyatakan algoritma optimisasi dikelompokkan ke dalam empat kelas besar [17]. Kelas pertama adalah Exact Algorithms, yaitu algoritma deterministik yang menjamin pencapaian solusi optimal, biasanya digunakan untuk masalah diskrit atau kombinatorial, maupun masalah kon-

tinu dengan toleransi tertentu. Kelas kedua, Heuristics and Metaheuristics, mencakup metode yang tidak menjamin solusi optimal, namun efektif dalam mencari solusi yang memadai dengan cepat melalui fungsi fitness yang mengukur kualitas solusi. Kelas ketiga adalah Surrogate-Based Optimization Algorithms, yang digunakan untuk masalah black-box yang memerlukan evaluasi mahal. Model pengganti atau surrogate digunakan untuk mengaproksimasi fungsi objektif, sehingga proses pencarian solusi menjadi lebih efisien dalam hal sumber daya. Terakhir, kelas keempat adalah Meta-Optimization and Hyperheuristics, yang berfokus pada proses optimisasi parameter dari algoritma itu sendiri untuk meningkatkan kinerjanya, menggunakan metode seperti parameter tuning atau parameter control yang dapat dilakukan sebelum atau selama proses optimisasi.

2.6.1 Exact Algorithms

Exact algorithms, juga dikenal sebagai non-heuristic atau complete algorithms [18], merupakan sebuah kelas khusus dari teknik optimasi deterministik, sistematis, atau exhaustive. Menurut Jorg Stork, algoritma ini dapat diterapkan baik pada domain diskret atau kombinatorial, di mana ruang pencarian memiliki jumlah solusi yang terbatas, maupun pada domain kontinu, dengan syarat bahwa pencarian dilakukan dalam toleransi tertentu [17]. Exact algorithms memiliki jaminan untuk menemukan global optimum dengan menggunakan sumber daya yang dapat diprediksi, seperti evaluasi fungsi atau waktu komputasi [18] - [19].

Namun, untuk mencapai jaminan ini, diperlukan informasi awal yang cukup tentang fungsi objektif, misalnya nilai fungsi objektif terbaik yang mungkin. Jika informasi awal tidak tersedia, kriteria penghentian (*stopping criterion*) harus ditentukan menggunakan pendekatan heuristik, yang dapat mengurangi jaminan terhadap optimalitas. Contoh algoritma *exact* yang terkenal adalah Branch-and-Bound, yang berdasarkan prinsip pemisahan, yaitu membagi suatu masalah menjadi sub-masalah yang lebih kecil dan masing-masing diselesaikan hingga optimal [20].

2.6.2 Heuristics and Metaheuristics Algorithms

Heuristik dan metaheuristik merupakan teknik penyelesaian masalah yang digunakan dalam optimasi modern. Kedua metode ini berlandaskan pada prinsip pencarian sistematis, di mana kandidat solusi dievaluasi dan diberi nilai berdasarkan *fitness*. Konsep *fitness* berasal dari komputasi evolusioner, di mana *fitness* menggambarkan kemampuan individu dalam bersaing dalam proses reproduksi. Dalam bentuk dasar, *fitness* adalah nilai dari fungsi objektif $y = f(x)$ yang berkaitan dengan

tujuan optimasi, seperti dalam masalah minimisasi, di mana nilai yang lebih kecil memiliki *fitness* yang lebih tinggi dibandingkan nilai yang lebih besar. Selain itu, *fitness* juga dapat menjadi bagian dari strategi pencarian, seperti di-skala atau disesuaikan dengan fungsi tambahan, terutama dalam optimasi multi-objektif atau *constrained*.

Heuristik dapat didefinisikan sebagai algoritma yang bergantung pada masalah tertentu, yang dikembangkan atau disesuaikan dengan karakteristik masalah optimasi spesifik [21]. Secara umum, heuristik melakukan evaluasi secara sistematis, meskipun melibatkan elemen stokastik. Heuristik memanfaatkan prinsip ini untuk menghasilkan solusi numerik yang cepat, namun tidak selalu optimal untuk masalah optimasi. Selain itu, heuristik sering kali bersifat serakah (*greedy*) untuk memperoleh solusi dengan cepat, namun sering terjebak dalam optima lokal dan gagal mencapai optimum global.

Di sisi lain, metaheuristik dapat didefinisikan sebagai algoritma optimasi yang independen dari masalah dan bersifat umum. Algoritma ini dapat diterapkan pada berbagai jenis masalah dan instansi masalah. Metaheuristik memiliki ciri-ciri sebagai berikut [22]:

- Algoritma ini terinspirasi oleh alam; mereka mengikuti prinsip-prinsip yang berasal dari fenomena atau perilaku alami, seperti evolusi biologis, fisika, atau perilaku sosial.
- Proses pencarian melibatkan elemen stokastik; menggunakan distribusi probabilitas dan proses acak.
- Karena dirancang untuk digunakan secara umum, mereka mencakup serangkaian parameter kontrol yang memungkinkan penyesuaian strategi pencarian.
- Mereka tidak bergantung pada informasi yang tersedia sebelum optimasi dimulai, yang dikenal sebagai informasi *a priori*. Namun, mereka masih dapat memanfaatkan informasi tersebut (misalnya, untuk mengatur parameter kontrol).

2.6.3 Surrogate-based optimization

Algoritma optimisasi berbasis surrogate dirancang untuk menangani masalah yang mahal dan kompleks, yang sering muncul dari aplikasi dunia nyata. Masalah-masalah ini biasanya bersifat black-box, yang berarti memiliki informasi domain

yang sangat terbatas. Oleh karena itu, informasi mengenai masalah perlu dieksploitasi melalui eksperimen atau evaluasi fungsi. Optimisasi berbasis surrogate bertujuan untuk memodelkan informasi yang tersedia tentang solusi kandidat yang sudah dievaluasi, untuk memanfaatkannya secara maksimal. Model surrogate merupakan pendekatan yang menggantikan fungsi objektif yang mahal, proses dunia nyata, simulasi fisik, atau proses komputasi selama optimisasi. Secara umum, surrogate dapat berupa model fisik atau numerik yang disederhanakan berdasarkan pengetahuan tentang sistem fisik, atau model fungsional empiris yang didasarkan pada pengetahuan yang diperoleh dari evaluasi dan pengambilan sampel parameter ruang yang terbatas [23].

Optimisasi berbasis surrogate dapat dibagi menjadi tiga lapisan utama. Lapisan pertama adalah aplikasi dunia nyata yang diberikan oleh proses fisik itu sendiri atau model fisik, yang seringkali sulit atau bahkan tidak mungkin dioptimalkan secara langsung karena memerlukan eksperimen yang mahal atau berisiko. Lapisan kedua adalah model komputasi, yang diberikan oleh simulasi proses fisik atau model komputasi yang kompleks, seperti model dinamika fluida komputasional atau model dinamika struktural. Lapisan ketiga adalah surrogate, yang merupakan model regresi berbasis data yang lebih murah untuk dievaluasi, meskipun memiliki fidelitas yang lebih rendah dibandingkan dengan dua lapisan lainnya. Keuntungan utama menggunakan surrogate adalah pengurangan jumlah evaluasi fungsi yang mahal selama proses optimisasi, yang pada gilirannya dapat menghemat sumber daya komputasi. Salah satu contoh algoritma dari kelas ini adalah Efficient Global Optimization (EGO) [24].

2.6.4 Meta-optimization and hyperheuristics

Meta-optimasi, atau penyetelan parameter [25], merujuk pada proses pencarian set parameter yang optimal untuk algoritma optimisasi [17]. Proses ini juga merupakan suatu bentuk optimisasi itu sendiri, yang bisa sangat mahal dari segi evaluasi fungsi objektif, karena diperlukan evaluasi terhadap set parameter suatu algoritma tertentu.

Hyperheuristic [26] adalah pendekatan tingkat tinggi yang bertujuan memilih dan menggabungkan pendekatan-pendekatan tingkat rendah, seperti heuristik, elemen dari metaheuristik untuk menyelesaikan masalah atau kelompok masalah tertentu. Ini adalah algoritma optimisasi yang mengotomatiskan proses perancangan algoritma dengan mencari ruang komponen algoritma yang telah ditentukan sebelumnya. Jorg menganggap *hyperheuristic* sebagai algoritma hibrida [17].

Berikut adalah contoh-contoh algoritma *hyperheuristic* dengan pemilihan sederhana [27]:

- *Simple Random (SR)*: Memilih heuristik tingkat rendah secara acak berdasarkan distribusi probabilitas seragam.
- *Random Descent (RD)*: Memilih heuristik tingkat rendah secara acak dan menerapkannya berulang kali hingga tidak ada perbaikan yang dicapai[28].
- *Greedy (GR)*: Memungkinkan semua heuristik untuk memproses solusi kandidat yang diberikan dan memilih yang menghasilkan solusi yang paling banyak diperbaiki.
- *Choice Function (CF)*: Menganalisis kinerja setiap heuristik tingkat rendah dan setiap pasangan heuristik tingkat rendah yang diterapkan secara berurutan, berdasarkan peningkatan kualitas, waktu eksekusi, dan kinerja keseluruhan [28], [29].

2.7 Algoritma Optimasi yang Digunakan

Pada bagian ini, dibahas dua algoritma optimasi yang digunakan dan dibandingkan dalam penelitian, masing-masing mewakili kelas yang berbeda dalam taksonomi algoritma optimasi. Algoritma pertama adalah *Genetic Algorithm (GA)*, yang termasuk dalam kategori *heuristic*, sedangkan yang kedua adalah *Sequential Least Squares Quadratic Programming (SLSQP)*, yang termasuk dalam kategori *deterministic* atau *exact*.

Genetic Algorithm dipilih karena, berdasarkan studi literatur yang dilakukan oleh Yarens [30], algoritma ini telah berhasil diterapkan pada permasalahan serupa, yaitu optimasi proses produksi. Sementara itu, *Sequential Least Squares Quadratic Programming* dipilih karena cocok digunakan pada model prediktif yang dapat didiferensialkan, sehingga memungkinkan pemanfaatan informasi turunan dalam proses optimasi.

2.7.1 Genetic Algorithm (GA)

Genetic Algorithm (GA) adalah suatu proses pencarian yang didasarkan pada hukum seleksi alam dan genetika. Pada artikelnya, Tang mengatakan bahwa GA biasanya terdiri dari tiga operasi utama: Seleksi, Operasi Genetik, dan Penggantian [31]. Proses siklus GA yang tipikal dapat dilihat pada Gambar. Populasi terdiri dari sekelompok kromosom yang dapat dipilih untuk mencari solusi dari suatu masalah. Pada awalnya, populasi dihasilkan secara acak. Nilai kebugaran (fitness) dari semua kromosom dievaluasi dengan menghitung fungsi objektif dalam bentuk terdecode

(fenotipe). Kelompok kromosom tertentu (orangtua) kemudian dipilih dari populasi untuk menghasilkan keturunan melalui operasi genetik yang telah ditentukan. Kebugaran keturunan dievaluasi dengan cara yang sama seperti orangtua mereka. Kromosom dalam populasi saat ini kemudian digantikan oleh keturunan mereka, berdasarkan strategi penggantian tertentu. Siklus GA ini diulang hingga kriteria penghentian yang diinginkan tercapai (misalnya, jumlah generasi yang telah ditentukan). Jika proses evolusi yang disimulasikan berjalan dengan baik, kromosom terbaik dalam populasi akhir dapat menjadi solusi yang sangat berkembang untuk masalah tersebut.

Proses GA umumnya dapat dijelaskan melalui beberapa langkah berikut:

1. Secara acak menghasilkan populasi awal $X(0) := (X_1, X_2, \dots, X_N)$;
2. Menghitung *fitness* $F(X_i)$ dari setiap kromosom X_i dalam populasi saat ini $X(t)$;
3. Membuat kromosom baru $X_r(t)$ dengan menggabungkan kromosom saat ini, menerapkan mutasi dan rekombinasi saat kromosom orangtua bersatu;
4. Menghapus sejumlah individu dalam populasi untuk memberi ruang bagi kromosom baru;
5. Menghitung kebugaran dari $X_r(t)$, dan memasukkan kromosom baru ke dalam populasi;
6. Menambah nilai $t := t + 1$, jika belum memenuhi kondisi penghentian, kembali ke langkah 3, atau jika sudah, berhenti dan kembalikan kromosom terbaik.

2.7.2 Sequential Least Squares Quadratic Programming (SLSQP)

Algoritma Sequential Least Squares Quadratic Programming (SLSQP) adalah salah satu jenis algoritma Sequential Quadratic Programming (SQP) yang digunakan untuk menyelesaikan masalah pemrograman non-linear (NLP). Masalah NLP bertujuan untuk menemukan nilai minimum dari suatu fungsi dengan banyak variabel yang terikat pada suatu set tertentu, dimana terdapat fungsi atau set yang bersifat non-linear. Jika fungsi objektif berupa fungsi kuadrat dan fungsi kendala bersifat linear, maka disebut masalah Quadratic Programming (QP). Model matematis umum untuk QP dapat dijelaskan sebagai berikut:

$$\begin{cases} \min & f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x}, \\ \text{s.t.} & h(\mathbf{x}) = \mathbf{a}_i^\top \mathbf{x} - b_i = 0, \quad i \in E = \{1, 2, \dots, l\}, \\ & g(\mathbf{x}) = \mathbf{a}_i^\top \mathbf{x} - b_i \geq 0, \quad i \in I = \{l+1, \dots, m\}. \end{cases} \quad (2.9)$$

Dimana $f(\mathbf{x})$ adalah fungsi objektif, $h(\mathbf{x})$ adalah fungsi kendala kesetaraan, dan $g(\mathbf{x})$ adalah fungsi kendala ketidaksamaan. Matriks H adalah matriks simetris positif-definit, yang digunakan untuk menggambarkan masalah optimisasi kuadratik. Prinsip dasar dari algoritma SQP adalah untuk mengubah masalah NLP terikat menjadi sub-masalah QP, dan memperbarui sub-masalah tersebut melalui iterasi untuk mendekati masalah NLP yang asli. Algoritma SLSQP merupakan algoritma SQP yang paling canggih, yang dimodifikasi dari perangkat lunak SQP yang dikembangkan oleh Dieter Kraft pada tahun 1988.

Secara umum, algoritma SLSQP terdiri dari lima langkah utama:

1. Menetapkan solusi awal $\mathbf{x}_0$ dan matriks simetris awal $\mathbf{B}_0$
2. Menyusun sub-masalah QP dan menghitung arah pencarian $\mathbf{d}_k$
3. Mengevaluasi apakah kondisi penghentian telah terpenuhi
4. Memilih fungsi nilai untuk memperoleh langkah pencarian α_k dalam arah saat ini, dan
5. Memperbarui titik iterasi menggunakan persamaan $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$, serta memilih metode untuk memperbarui $\mathbf{B}_k$, kemudian kembali ke langkah (2). Pada langkah kedua, fungsi Lagrangian dari masalah NLP dengan kendala umum dapat dihitung untuk memperoleh arah pencarian.

Formulasi masalah sub-QP yang digunakan dalam algoritma ini adalah sebagai berikut:

$$\begin{cases} \min & \frac{1}{2} \mathbf{d}_k^\top \mathbf{W}_k \mathbf{d}_k + \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k, \\ \text{s.t.} & \nabla h_i(\mathbf{x}_k)^\top \mathbf{d}_k + h_i(\mathbf{x}_k) = 0, \quad i \in E, \\ & \nabla g_i(\mathbf{x}_k)^\top \mathbf{d}_k + g_i(\mathbf{x}_k) \geq 0, \quad i \in I. \end{cases} \quad (2.10)$$

Dimana $\mathbf{W}_k$ adalah matriks Hessian dari fungsi Lagrangian. Karena perhitungan matriks Hessian yang kompleks, metode Wilson-Han-Powell digunakan untuk menghitung vektor arah pencarian $\mathbf{d}_k$ dengan menggunakan matriks simetris positif-definit $\mathbf{B}_k$ yang menggantikan $\mathbf{W}_k$. Algoritma ini hanya memerlukan batasan

dari fungsi objektif, fungsi kendala, dan variabel optimisasi, serta memiliki konvergensi global dan kecepatan konvergensi superlinear. Oleh karena itu, algoritma SLSQP sering digunakan dalam berbagai aplikasi optimisasi, seperti yang dijelaskan oleh Jupeng dalam konteks optimisasi konsumsi energi pada sistem pendingin udara sentral [32].

Dalam konteks penelitian ini, implementasi algoritma SLSQP akan melibatkan definisi kendala-kendala yang sesuai dengan batasan operasional dan teknis pada proses produksi feronikel. Kendala-kendala ini akan direpresentasikan sebagai fungsi $h(\mathbf{x})$ untuk kendala kesetaraan dan $g(\mathbf{x})$ untuk kendala ketidaksamaan dalam formulasi sub-QP. Secara umum, kendala yang akan dipertimbangkan meliputi:

1. Kendala Batasan Variabel Operasional (*Input Model*)

Ini mencakup batasan minimum dan maksimum yang diizinkan untuk setiap parameter operasional yang menjadi variabel optimisasi (misalnya, arus listrik, tegangan, suhu, laju umpan material seperti bijih atau reduktor). Batasan-batasan ini memastikan bahwa solusi optimisasi tetap berada dalam rentang kerja yang realistis dan aman bagi peralatan produksi. Kendala ini akan didefinisikan sebagai fungsi ketidaksamaan ($g(\mathbf{x}) \geq 0$ atau $g(\mathbf{x}) \leq 0$) dalam kode optimisasi.

2. Kendala Batasan Kualitas Produk (*Output Model*)

Kualitas produk feronikel, seperti kadar nikel (Ni%) dan elemen lainnya, harus memenuhi spesifikasi tertentu. Kendala ini akan diterapkan untuk memastikan bahwa hasil prediksi kualitas produk dari model berada dalam rentang target yang diinginkan. Mirip dengan kendala input, ini juga akan diatur sebagai fungsi ketidaksamaan yang melibatkan output model prediksi.

3. Kendala Batasan Konsumsi Energi (*Output Model*)

Meskipun konsumsi energi adalah bagian dari fungsi objektif yang diminimalkan, batasan juga dapat diterapkan pada nilai output konsumsi energi (`total_fuel` dan `reductor_consume`) untuk memastikan mereka berada dalam rentang yang wajar atau tidak melebihi kapasitas tertentu. Ini akan diatur sebagai fungsi ketidaksamaan pada output model prediksi.

4. Kendala Hubungan Fisik/Proses (jika ada)

Jika ada hubungan matematis atau fisik yang harus dipenuhi secara presisi dalam proses (misalnya, keseimbangan massa atau energi tertentu yang harus nol), maka ini akan diatur sebagai kendala kesetaraan ($h(\mathbf{x}) = 0$).

Bagian-bagian yang dijelaskan di atas, yaitu fungsi objektif dan fungsi kendala-kendala $h(\mathbf{x})$ dan $g(\mathbf{x})$, merupakan komponen-komponen kunci yang akan secara spesifik didefinisikan dan diatur dalam struktur kode untuk optimisasi menggunakan algoritma SLSQP.

2.8 Metrik Evaluasi Model Machine Learning

Metrik ini sangat penting untuk memahami performa sistem ataupun algoritma. Di mana solusi sistem dibandingkan dengan nilai aktual. Beberapa metrik evaluasi yang umum digunakan adalah Mean Squared Error (MSE), Root Mean Squared Error (RMSE), dan Mean Absolute Error (MAE). Ketiga metrik ini membantu mengevaluasi tingkat kesalahan model dengan pendekatan yang berbeda, sehingga memberikan gambaran menyeluruh tentang akurasi prediksi.

2.8.1 MSE

MSE menghitung rata-rata dari selisih kuadrat antara nilai prediksi dan nilai aktual. Dengan kata lain, MSE mengukur seberapa besar kesalahan kuadrat rata-rata yang dibuat oleh model. Nilai MSE yang lebih kecil menunjukkan bahwa model memiliki performa prediksi yang lebih baik. Rumus MSE dapat dituliskan sebagai berikut:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.11)$$

Di mana:

- n adalah jumlah sampel dalam data.
- y_i adalah nilai aktual.
- $\hat{y}_i$ adalah nilai prediksi.

2.8.2 RMSE

Seperti namanya, RMSE adalah akar kuadrat dari MSE. RMSE memberikan interpretasi yang lebih intuitif karena satuannya sama dengan satuan data aslinya. RMSE mengukur rata-rata kesalahan kuadrat, tetapi dengan memperhitungkan akar kuadratnya untuk memberikan gambaran yang lebih realistis. Nilai RMSE yang lebih kecil menunjukkan bahwa model memiliki performa prediksi yang lebih baik. Rumus RMSE dapat dituliskan sebagai berikut:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.12)$$

Di mana:

- n adalah jumlah sampel dalam data.
- y_i adalah nilai aktual.
- $\hat{y}_i$ adalah nilai prediksi.

2.8.3 MAE

MAE menghitung rata-rata dari selisih absolut antara nilai prediksi dan nilai aktual. MAE memberikan gambaran langsung tentang seberapa besar rata-rata kesalahan prediksi tanpa memperhitungkan kuadrat atau akar kuadrat. Hal ini menjadikan MAE lebih robust terhadap outlier dibandingkan MSE dan RMSE. Nilai MAE yang lebih kecil menunjukkan model yang lebih akurat. Rumus MAE dapat dituliskan sebagai berikut:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.13)$$

Di mana:

- n adalah jumlah sampel dalam data.
- y_i adalah nilai aktual.
- $\hat{y}_i$ adalah nilai prediksi.

BAB 3

PERANCANGAN SISTEM MACHINE LEARNING UNTUK OPTIMASI PROSES PRODUKSI FERONIKEL

Bab ini akan menguraikan flow chart dan arsitektur sistem yang menggambarkan tahapan-tahapan dalam pelaksanaan penelitian. Selain itu, bab ini akan menjelaskan secara lebih mendalam bagaimana teknologi yang telah dijelaskan pada bab sebelumnya diterapkan dalam penelitian. Implementasi teknologi *machine learning* dan Optimisasi juga akan dibahas secara rinci di bab ini.

3.1 Dataset

3.1.1 Sumber Data

Data yang digunakan berasal dari beberapa dokumen operasional suatu industri pengolahan feronikel yang kemudian digabungkan menjadi satu tabel master. Dokumen-dokumen tersebut diberikan oleh perusahaan yang terlibat dalam proyek ini. Data yang dipakai berasal dari lini produksi RK dan EF.

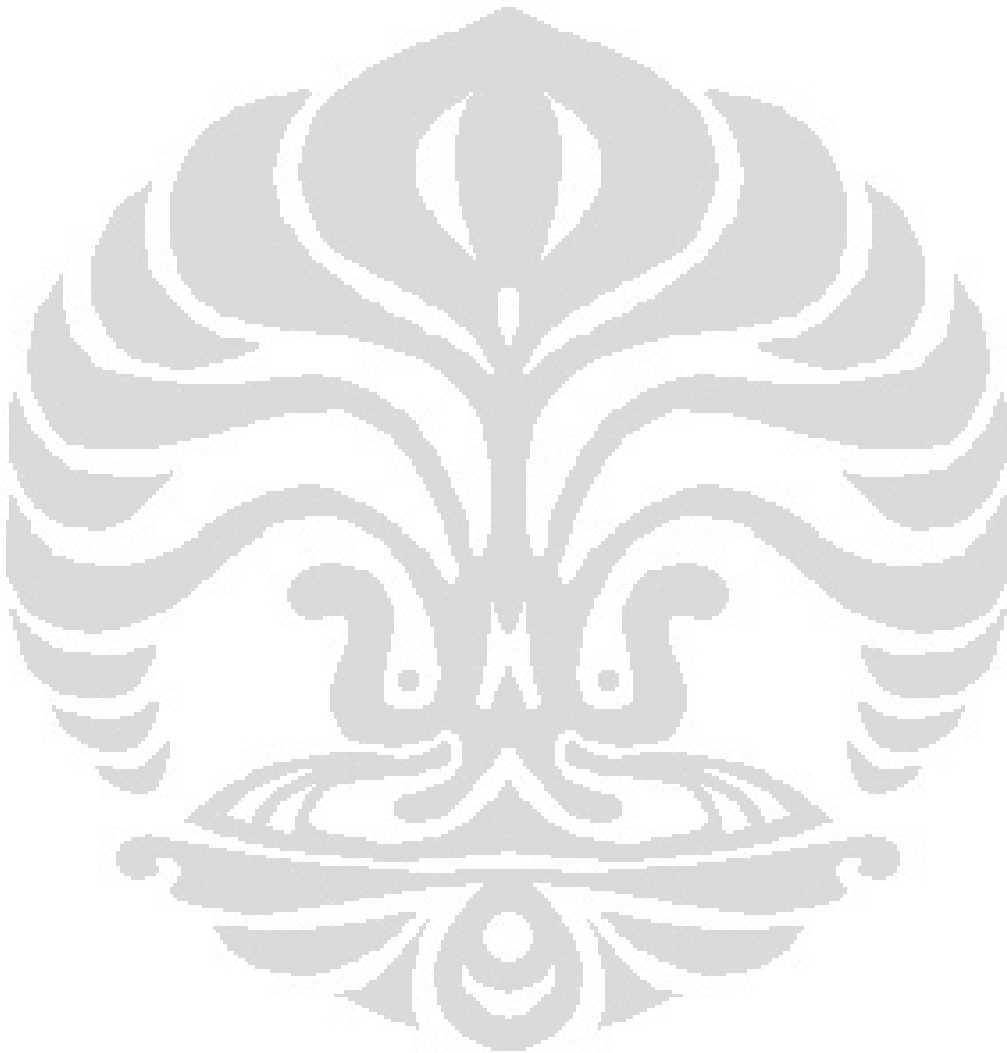
Terdapat 3 file data yang digunakan untuk pengambilan data, yaitu:

- File 1: Data Smelting, mencakup informasi operasional pada EF yaitu *load*, hasil *metal*, dan *slag*.
- File 2: Data dari proses RK dan RD.
- File 3: Data laporan penggunaan bahan bakar yang dibutuhkan untuk pemodelan sistem dan perhitungan efisiensi.

Secara keseluruhan, terdapat 43 variabel yang digunakan, yang bersumber dari ketiga file tersebut. Dari jumlah ini, 32 variabel berfungsi sebagai input untuk pengguna dan pengaturan sistem, sementara 11 variabel lainnya merupakan output. Berikut adalah rincian variabel berdasarkan file:

- File 1: TCO, Current, Load, Metal Temperature, Ni Metal, C Metal, Si Metal, Fe Metal, S Metal, Ni Slag, Fe Slag.
- File 2: Ni_In, Fe_In, SiO₂, MgO, CaO_In, Al₂O₃_In, Fe/Ni, S/M, BC, LOI In, MC Kilnfeed, FC Coal, GCV Coal, LOI Kalsin.

- File 3: RPM, Primary P, Secondary P, Primary V, Secondary V, Total Coal, A/F Ratio, Konsumsi kg/TCO, Rasio Reduktor, Konsumsi Reduktor, Charge Kiln, Temperature TIC172, TIC166, TIC162, TIC163, PIC161, Temperatur Kalsin.



3.1.2 Variabel Pada Data

3.1 dan 3.2 masing-masing secara berurutan menunjukkan daftar variabel input dan output pada sistem beserta deskripsi dan satuannya.

Tabel 3.1: Daftar Variabel Input Sistem

No.	Nama	Deskripsi	Satuan	Entitas
1	ni_in	Kandungan nikel dalam bahan baku di RK.	%	RK & EF
2	fe_in	Kandungan besi (Fe) dalam bahan baku di RK. Nilai ideal < 16%.	%	RK & EF
3	cao_in	Kandungan kalsium oksida (CaO) dalam bahan baku di RK.	%	RK & EF
4	al2o3_in	Kandungan aluminium oksida (Al ₂ O ₃) dalam bahan baku di RK.	%	RK & EF
5	s_m	Rasio antara silika dioksida (SiO ₂) dengan magnesium oksida (MgO) dalam bahan baku di RK. Nilai ideal < 2.	%	RK & EF
6	bc	Basisitas, rasio mineral basa dengan senyawa asam dalam bahan baku di RK.	%	RK & EF
7	mc_kilnfeed	Kadar air (moisture content) pada bijih bahan baku. Digunakan untuk menjaga kadar air dalam bahan baku.	%	RK
8	fc_coal	Kandungan karbon tetap (fixed carbon) dari batubara burner. Digunakan untuk mengukur kualitas batubara yang dipakai.	%	RK & EF

Tabel 3.1: Daftar Variabel Input Sistem (lanjutan)

No.	Nama	Deskripsi	Satuan	Entitas
9	gcv_coal	Nilai kalor yang dapat dihasilkan oleh batubara burner. Digunakan untuk melihat performa bahan bakar.	kcal/kg	RK & EF
10	fc_lcv	Kandungan karbon tetap (fixed carbon) dari batubara reduktor.	%	RK & EF
11	gcv_lcv	Nilai kalor yang dapat dihasilkan oleh batubara reduktor.	kcal/kg	RK & EF
12	current_pry	Arus listrik pada elektroda utama EF.	kA	EF
13	current_sec1	Arus listrik pada elektroda sekunder 1 EF.	kA	EF
14	current_sec2	Arus listrik pada elektroda sekunder 2 EF.	kA	EF
15	current_sec3	Arus listrik pada elektroda sekunder 3 EF.	kA	EF
16	load	Beban listrik yang digunakan oleh EF.	MW	EF
17	realisasi_beban	Beban listrik aktual yang direalisasikan oleh sistem EF.	MW	EF
18	rpm	Kecepatan putaran RK.	RPM	RK
19	kg_tco	Penggunaan bahan bakar batubara per ton bijih yang diproses. Digunakan untuk melihat efisiensi konsumsi batubara.	kg/Ton	RK
20	charge_kiln	Jumlah muatan total atau charge yang dimasukkan ke RK.	Ton/Jam	RK & EF
21	tdo	Jumlah dry ore yang digunakan di RK.	-	RK

Tabel 3.1: Daftar Variabel Input Sistem (lanjutan)

No.	Nama	Deskripsi	Satuan	Entitas
22	pry_p	Tekanan udara di katup primer RK.	mmH2O	RK
23	sec_p	Tekanan udara di katup sekunder RK.	mmH2O	RK
24	pry_v	Jumlah aliran udara di katup primer RK. Berperan dalam distribusi oksigen untuk pembakaran di RK.	m ³ /menit	RK
25	sec_v	Jumlah aliran udara di katup sekunder RK. Berperan dalam distribusi oksigen untuk pembakaran di RK.	m ³ /menit	RK
26	total_fuel	Jumlah batubara dan fuel oil sebagai bahan bakar burner di RK.	kg/Jam	RK & EF
27	reductor_consume	Jumlah konsumsi batubara sebagai bahan reduktor untuk proses reduksi.	Ton/Jam	RK & EF
28	t_tic162	Suhu di titik IC162 RK.	°C	RK & EF
29	t_tic163	Suhu di titik IC163 RK.	°C	RK & EF

Tabel 3.2: Daftar Variabel Output Sistem

No.	Nama	Deskripsi	Satuan	Entitas
1	metal_temp	Suhu logam hasil peleburan di EF. Suhu logam diharapkan berada di atas 1300°C.	°C	EF
2	ni_met	Kandungan nikel (Ni) dalam feronikel, memiliki nilai ideal > 17%.	%	EF
3	c_met	Kandungan karbon (C) dalam feronikel, memiliki nilai ideal 1-2%.	%	EF

Tabel 3.2: Daftar Variabel Output Sistem (lanjutan)

No.	Nama	Deskripsi	Satuan	Entitas
4	si_met	Kandungan silika (Si) dalam feronikel, memiliki nilai ideal 1-3%.	%	EF
5	fe_met	Kandungan besi (Fe) dalam feronikel.	%	EF
6	s_met	Kandungan sulfur (S) dalam feronikel, memiliki nilai ideal < 0,4%.	%	EF
7	fe_slag	Kandungan besi (Fe) dalam slag hasil peleburan di EF.	%	EF
8	ni_slag	Kandungan nikel (Ni) dalam slag hasil peleburan di EF.	%	EF
9	t_kalsin	Suhu produk kalsinasi yang di RK.	°C	RK & EF
10	pic_161	Tekanan di titik IC161 RK. Berfungsi untuk mengontrol tekanan dalam RK. Nilai idealnya 0 atau minus bernilai rendah. Jika nilainya lebih dari 0, menandakan ada kebocoran panas di RK.	mmH2O	RK & EF
11	loi_kalsin	Jumlah kehilangan massa selama proses kalsinasi di RK (Loss of Ignition).	-	RK & EF

3.1.3 Frekuensi Data

Data yang diperoleh memiliki frekuensi yang bervariasi. Setiap file data memiliki waktu pengambilan yang berbeda, di antaranya ada yang dicatat per jam, per shift, dan per hari. Untuk integrasi data yang lebih baik, penyesuaian waktu diperlukan. Data akan dikonversi menjadi data per jam, mulai dari pukul 00:00 hingga 23:00, untuk tanggal 1 hingga 31 Januari 2021. Keputusan ini didasarkan pada informasi dari pabrik yang menyebutkan bahwa pengaturan variabel yang telah disesuaikan biasanya dilakukan setiap jam, yang berarti perubahan dapat terjadi setiap jam. Oleh karena itu, konversi data ke dalam satuan jam akan membuat dataset menjadi lebih representatif dan lebih variatif.

Pada pengambilan data, terdapat beberapa variabel yang memiliki waktu pengambilan yang berbeda. Beberapa variabel dicatat per hari, beberapa per shift, dan lainnya per jam. Terkait dengan shift, terdapat tiga shift yang digunakan, yaitu shift 1, shift 2, dan shift 3. Berikut adalah tabel yang menunjukkan pembagian waktu setiap shift:

Tabel 3.3: Informasi Shift

Shift	Waktu
Shift III	Jam 01:00 hingga Jam 08:59
Shift I	Jam 09:00 hingga Jam 16:59
Shift II	Jam 09:00 hingga Jam 16:59

Namun, ada juga data yang diambil setiap dua jam sekali. Oleh karena itu, penyesuaian waktu diperlukan untuk semua variabel. Target akhirnya adalah agar seluruh data memiliki waktu pengambilan per jam. Variabel dengan waktu pengambilan per hari akan dibagi dengan 24, sementara data yang diambil setiap dua jam akan memiliki nilai yang sama untuk setiap dua jam tersebut. Dengan cara ini, seluruh kolom data akan memiliki waktu pengambilan per jam.

3.1.4 Metode Pengambilan

Setiap jenis data memiliki metode pengambilan yang berbeda. Data yang terdapat pada kolom yang mengandung kata kunci "ic" berarti data tersebut diambil langsung dari instrumen kontrol (IC), sehingga data umumnya lebih rinci dengan waktu pengambilan per jam. Kolom dengan waktu pengambilan per shift diambil dan dicatat secara manual oleh pekerja di lokasi. Sedangkan kolom yang mencatat data harian umumnya diambil dari hasil analisis laboratorium, seperti pada kolom yang merepresentasikan komposisi ore.

Berikut adalah penjelasan mengenai metode pengambilan data dan persebaran variabelnya:

1. **Hasil Analisis Laboratorium** Data ini diperoleh dengan mengambil sampel material pada periode waktu tertentu dan menguji kualitasnya di laboratorium. Variabel-variabel utama meliputi:
 - **Material Metalurgi:** Ni met, C met, Si met, Fe met, S met.
 - **Material Slag:** Ni slag, Fe slag.
 - **Input Material:** Ni in, Fe in, CaO in, SiO₂ in, MgO in, Al₂O₃ in, S m.
 - **Parameter Tambahan:** Fe ni, BC, LOI kalsin, MC kilnfeed, LOI in.
2. **Data dari Sensor** Data ini diambil menggunakan sensor yang terpasang pada beberapa titik di mesin RKEF, digunakan untuk memantau suhu dan tekanan. Variabel-variabel utama meliputi:
 - **Tekanan:** Pry P, Sec P.
 - **Kecepatan:** Pry V, Sec V.
 - **Suhu di Titik Tertentu:** T TIC162, T TIC163, T TIC166, T TIC167.
 - **Tekanan di Lokasi Spesifik:** PIC 161.
3. **Pencatatan Manual** Data ini diperoleh secara manual oleh pekerja lapangan dan mencakup parameter operasional mesin serta material. Variabel-variabel utama meliputi:
 - **Kondisi Operasional:** TCO, Dry Ore, Current, Load, Voltage, Metal Temp.
 - **Parameter Reduksi:** FC Coal, GCV Coal, A/F Ratio, Kg TCO, Reductor Consume, Reductor Ratio.
 - **Proses Produksi:** Charge Kiln, T Kalsin.

3.1.5 Desain Arsitektur Sistem Optimisasi

Desain arsitektur sistem dalam penelitian ini disusun untuk mengintegrasikan model prediktif proses produksi feronikel dengan dua pendekatan optimisasi, yaitu *Sequential Least Squares Programming* (SLSQP) dan *Genetic Algorithm* (GA). Tujuan utama sistem ini adalah menemukan kombinasi parameter proses yang optimal agar efisiensi konsumsi energi dapat ditingkatkan tanpa melanggar batasan kualitas produk maupun batas aman operasional.

Secara umum, arsitektur sistem optimisasi terdiri dari beberapa komponen utama sebagai berikut:

1. Input Data Historis Terstandarisasi

Dataset proses diambil dari data historis, kemudian dinormalisasi menggunakan pendekatan *Min-Max Scaler*. Data ini berfungsi untuk:

- Menentukan batas bawah dan atas (*boundaries*) parameter proses yang akan dioptimisasi,
- Menyediakan *baseline row* atau baris acuan sebagai referensi untuk optimisasi

2. Pembentukan Solusi

- Pada metode SLSQP, pencarian solusi dimulai dari titik awal acak dalam ruang solusi. Proses ini menggunakan pendekatan *gradient-based* untuk menemukan nilai minimum dari fungsi objektif.
- Pada metode GA, solusi dinyatakan dalam bentuk *kromosom*, yang merupakan representasi dari nilai-nilai parameter proses. Evolusi populasi dilakukan melalui operasi *selection*, *crossover*, dan *mutation*.

3. Evaluasi Solusi

Setiap solusi diuji menggunakan model prediktif untuk menghasilkan estimasi variabel output seperti *ni_met*, *si_met*, dan *metal_temp*. Evaluasi dilakukan berdasarkan:

- Untuk SLSQP, nilai solusi dihitung menggunakan fungsi objektif yang dirancang secara langsung berdasarkan target optimasi, dengan constraint yang dimasukkan secara eksplisit.
- Untuk GA, nilai *fitness* dihitung berdasarkan nilai objektif yang sama, namun penalti akan ditambahkan apabila terdapat pelanggaran terhadap constraint (baik input maupun output).

4. Penerapan Constraint

Constraint diterapkan untuk memastikan solusi tetap berada dalam ruang yang valid secara teknis dan operasional.

- Pada SLSQP, constraint diformulasikan secara matematis sebagai bagian dari model optimisasi, baik dalam bentuk batasan nilai (*bound constraint*) maupun persamaan/pertidaksamaan (*nonlinear constraint*).

- Pada GA, constraint diimplementasikan melalui mekanisme penalti yang ditambahkan ke nilai *fitness*, sehingga solusi yang melanggar batas akan memiliki fitness yang lebih buruk.

5. Pemilihan Solusi Optimal

- Pada SLSQP, proses iteratif dilanjutkan hingga mencapai konvergensi pada minimum lokal dari fungsi objektif.
- Pada GA, solusi optimal diperoleh dari individu dengan nilai fitness terbaik pada akhir generasi.

3.2 Formulasi Permasalahan Optimisasi

3.2.1 Variabel Input (Parameter Proses yang Dioptimasi)

Dari total 40 variabel yang tersedia dalam dataset, tidak semua digunakan sebagai parameter dalam proses optimisasi dimana 11 diantaranya adalah variabel *output* menyisakan 29 variabel *input*. Optimisasi pada penelitian ini difokuskan pada variabel-variabel input yang secara teknis dapat diatur atau dikendalikan dalam operasi produksi. Terdapat 14 input yang nilainya *fixed*, tidak dioptimalkan, seperti komposisi bijih (*fe.in*, *ni.in* dsb.) dan kualitas batu bara (*gcv*, *lcv* dan *FC*). Sebanyak 15 parameter yang termasuk dalam kategori *tuned input* dan menjadi objek optimisasi dalam studi. Parameter ini mencakup komposisi material, kondisi operasional kiln, konsumsi bahan bakar, serta arus dan tekanan proses dikenakan batas nilai minimum dan maksimum. Parameter-parameter ini merupakan variabel operasional yang bersifat dapat diatur, baik pada unit Rotary Kiln (RK) maupun Electric Furnace (EF). Klasifikasi variabel dapat dilihat pada 3.1.

Pemilihan variabel-variabel ini didasarkan pada keterkaitannya terhadap konsumsi bahan bakar, relevansi teknis, serta potensi pengaruh terhadap output yang menjadi target optimisasi. Variabel-variabel ini akan digunakan sebagai *decision variables* dalam algoritma optimisasi SLSQP dan Genetic Algorithm.

Batas atas dan bawah untuk masing-masing variabel diambil dari rentang nilai aktual yang terdapat pada data historis. Nilai-nilai ini juga berfungsi sebagai constraint dalam proses optimisasi agar solusi yang dihasilkan tetap dalam batasan operasional nyata.

3.2.2 Fungsi Objektif: Meminimalkan Selisih Prediksi Terhadap Nilai Aktual

Fungsi objektif dalam penelitian ini dirancang untuk menjaga agar hasil prediksi model machine learning terhadap variabel output tidak menyimpang dari nilai aktualnya. Dengan kata lain, optimisasi bertujuan untuk menemukan kombinasi nilai parameter proses yang menghasilkan prediksi output serupa dengan nilai aktual pada titik data tertentu. Pendekatan ini dilakukan untuk memastikan bahwa konfigurasi nilai input hasil optimisasi tetap menghasilkan output yang berada dalam kisaran realistis dan sesuai dengan karakteristik proses aktual.

Fungsi objektif diformulasikan sebagai selisih kuadrat antara prediksi model dan nilai target:

$$\min_x (\hat{y}(x) - y_{\text{target}})^2 \quad (3.1)$$

dengan:

- x adalah vektor parameter input dalam ruang ternormalisasi,
- $\hat{y}(x)$ adalah prediksi output dari model Ridge Regression,
- y_{target} adalah nilai target output dari data uji.

Fungsi ini memandu algoritma optimisasi untuk mencari nilai-nilai input alternatif yang secara statistik masuk akal, namun tetap menghasilkan prediksi output yang konsisten dengan performa aktual sistem, termasuk konsumsi bahan bakar dan parameter kualitas produk. Dengan demikian, pendekatan ini menjaga kesesuaian antara hasil optimisasi dan kondisi nyata proses produksi.

3.2.3 Batasan (Constraint): Rentang Nilai Parameter Proses

Untuk memastikan solusi yang dihasilkan tetap berada dalam ruang parameter yang realistis dan aman secara operasional, proses optimisasi dilakukan dengan pembatasan nilai pada variabel-variabel input. Seluruh variabel telah dinormalisasi menggunakan pendekatan *Min-Max Normalization* sebelum dioptimisasi. Batasan ini bersifat tetap dan akan digunakan sebagai acuan dasar dalam seluruh variasi eksperimen yang dilakukan dalam penelitian ini.

Constraint dalam eksperimen ini dirancang secara ketat berdasarkan pertimbangan domain dan batasan proses aktual. Constraint diklasifikasikan ke dalam tiga kategori:

1. Constraint Equality (Input Tetap)

Sebanyak 14 parameter input dikunci nilainya (fixed) karena bersifat tidak dapat dikendalikan langsung. Contohnya meliputi:

- Komposisi bahan baku: `ni_in`, `fe_in`, `cao_in`, `al2o3_in`.
- Karakteristik batubara dan kadar air: `fc_coal`, `gcv_coal`, `mc_kilnfeed`, dll.

Constraint ini diimplementasikan sebagai *equality constraint* berbentuk: $x_i =$ nilai aktual.

2. Constraint Rentang Input (Min/Max)

Untuk 15 parameter *tuned input*, diterapkan batas bawah dan atas berdasarkan data historis (Januari 2021), seperti:

- `rpm`, `pry_p`, `sec_p`, `total_fuel`, `reductor_consume`, dan suhu (`t_tic162`, `t_tic163`).

Batas bawah dan atas ditetapkan menggunakan nilai minimum-maksimum aktual atau estimasi berbasis IQR/standar deviasi.

3. Constraint Output Berdasarkan Prediksi Model

Untuk memastikan hasil optimisasi tetap menghasilkan kualitas produk yang sesuai spesifikasi, diberlakukan constraint pada output prediksi model. Contohnya:

- `ni_met` minimal sesuai batas target produksi.
- `c_met` dan `si_met` dijaga dalam rentang kualitas logam.
- `loi_kalsin` dijaga dalam batas yang wajar.

Constraint ini berbentuk *nonlinear inequality* terhadap fungsi prediksi model: $\hat{y}(x) \geq y_{\min}$ atau $\hat{y}(x) \leq y_{\max}$.

3.2.3.1 Batasan pada Variabel Input (*Input Constraints*)

Sebanyak 16 parameter input yang mencakup komposisi material, kondisi operasional kiln, konsumsi bahan bakar, serta arus dan tekanan proses dikenakan batas nilai minimum dan maksimum. Batasan ini ditentukan berdasarkan nilai-nilai aktual dari data historis tahun 2024, serta masukan teknis dari pihak industri untuk memastikan kesesuaian dengan kondisi operasi aktual. Berikut adalah batasan nilai parameter dalam ruang ternormalisasi yang digunakan dalam proses optimisasi.

Tabel 3.4: Batasan Variabel Input (dalam ruang ternormalisasi)

Nama Parameter	Minimum	Maksimum
current_pry	0.00	-
current_sec1	0.00	-
current_sec2	0.00	-
current_sec3	0.00	-
load	0.00	-
realisasi_beban	0.00	-
rpm	0.00	-
pry_p	0.00	-
sec_p	0.00	-
pry_v	0.00	-
sec_v	0.00	-
total_fuel	0.00	-
reductor_consume	0.00	-
t_tic162	556	1201
t_tic163	456	845

Seluruh parameter diberi batas minimum nol untuk mencegah terjadinya nilai negatif yang secara fisik tidak mungkin, seperti arus listrik negatif atau konsumsi bahan bakar negatif. Batas atas ditetapkan berdasarkan nilai-nilai yang dianggap aman dan representatif dari operasi aktual, dan tidak melibatkan toleransi statistik. Nilai-nilai ini akan menjadi referensi tetap dalam seluruh eksperimen optimisasi yang dilakukan.

3.2.3.2 Batasan pada Variabel Output (*Output Constraints*)

Selain pembatasan pada variabel input, proses optimisasi juga memperhatikan batasan pada variabel output untuk memastikan bahwa solusi yang dihasilkan tidak hanya efisien dari sisi konsumsi energi, tetapi juga memenuhi spesifikasi kualitas produk feronikel. Batasan pada variabel output ini ditetapkan secara tetap (*fixed*) berdasarkan referensi teknis dari industri dan data historis proses produksi. Nilai-nilai tersebut menjadi acuan dasar dalam seluruh variasi eksperimen yang dilakukan dalam penelitian ini.

Tabel 3.5: Batasan Variabel Output (dalam satuan asli, sebelum normalisasi)

Nama Variabel	Minimum	Maksimum
metal_temp	1300	-
ni_met	17	-
c_met	1	3
si_met	1	2
fe_met	0	-
s_met	0	0.4
ni_slag	0	-
fe_slag	0	-
t_kalsin	600	-
pic_161	-1	-
loi_kalsin	0	1

Batasan minimum seperti suhu metal (1300°C), kadar nikel ($\text{ni_met} \geq 17$), dan kadar karbon (1–3%) ditentukan berdasarkan batas aman dan target mutu produk akhir feronikel yang berlaku di industri. Demikian pula, batas atas untuk kandungan silikon ($\text{si_met} \leq 2$) dan sulfur ($\text{s_met} \leq 0,4$) bertujuan menjaga agar kualitas metal tetap sesuai dengan spesifikasi yang diharapkan

3.2.3.3 Perbandingan Pendekatan Konfigurasi Kendala (SLSQP vs GA)

Perbedaan mendasar dalam penanganan kendala antara SLSQP dan GA adalah sebagai berikut:

- **SLSQP:** Menggunakan pendekatan berbasis gradien dan aproksimasi linear untuk secara eksplisit memasukkan kendala ke dalam sub-masalah QP. Kendala didefinisikan sebagai fungsi yang harus dipenuhi secara tepat ('eq') atau di atas/bawah batas tertentu ('ineq'). Pelanggaran kendala secara langsung memengaruhi arah pencarian dan konvergensi algoritma.
- **GA:** Menggunakan pendekatan berbasis populasi dan evolusi. Kendala input ditangani secara *hard* (secara langsung membatasi ruang pencarian genetik) selama inisialisasi dan reproduksi. Kendala output ditangani secara *soft* melalui fungsi penalti, di mana pelanggaran tidak serta-merta mengeliminasi solusi, tetapi memberikan "hukuman" yang membuat solusi tersebut kurang diinginkan secara genetik.

Kedua pendekatan ini memiliki kelebihan dan kekurangan. SLSQP cenderung lebih cepat dan efisien untuk masalah dengan kendala yang mulus (differentiable), sementara GA lebih tangguh dalam menangani masalah non-linear yang kompleks, fungsi

objektif yang tidak mulus, atau kendala yang sulit diformulasikan secara analitis, namun mungkin memerlukan lebih banyak evaluasi fungsi dan waktu komputasi.

3.3 Skenario Eksperimen

Untuk mengevaluasi kinerja dua metode optimasi—Sequential Least Squares Programming (SLSQP) dan Genetic Algorithm (GA)—dalam konteks efisiensi konsumsi energi proses produksi feronikel, dilakukan serangkaian eksperimen dengan skenario yang berbeda. Masing-masing skenario dirancang untuk menguji sensitivitas model terhadap variasi batasan (*constraints*). Terdapat lima skenario utama (a hingga e), yang dijalankan pada kedua metode optimasi (SLSQP dan GA), sehingga menghasilkan total lima konfigurasi eksperimen.

- Skenario (a) – Membatasi nilai maksimum fuel dan reduktor dengan nilai aktual

Pada skenario ini, batas maksimum untuk `total_fuel` dan `reductor_consume` ditetapkan menggunakan nilai aktual. Tidak terdapat batasan atas (*upper bound*) untuk sebagian besar parameter selain dua variabel tersebut.

- Skenario (b) – Tanpa Batas Maksimum Fuel Maupun Reduktor

Skenario (b) merupakan modifikasi dari skenario (a) dengan menghapus batas atas (*upper bound*) pada variabel `reductor_consume` dan `total_fuel`. Tujuannya adalah untuk menghindari penggunaan nilai aktual sebagai *ground truth*, karena bisa saja nilai aktual ternyata di bawah dari nilai optimal dan perlu ditingkatkan. Konfigurasi batasan input dan output yang digunakan untuk skenario ini ditampilkan pada tabel 3.4 dan 3.5. Output dikendalikan agar tetap dalam rentang kualitas yang dapat diterima berdasarkan batas referensi industri. Tidak terdapat toleransi statistik dalam skenario ini; batasan bersifat ketat dan langsung berdasarkan nilai referensi.

- Skenario (c) – Batasan Minimum Berdasarkan Nilai Off

Skenario (c) mengadopsi pendekatan baru pada batas minimum untuk `reductor_consume` dan `total_fuel`, yaitu menggantikan nilai minimum sebelumnya (0) dengan rata-rata (*mean*) konsumsi ketika kondisi kiln dalam status non-produktif atau *off*. Pendekatan ini bertujuan menghindari solusi ekstrem dengan konsumsi energi sangat rendah yang tidak realistis secara operasional.

- Skenario (d) – Batasan Dinamis dan Membatasi Fuel & Reduktan Berdasarkan referensi ilmiah

Pada skenario ini, batas bawah dan atas untuk `reductor_consume` dan `total_fuel` dihitung berdasarkan estimasi kebutuhan energi aktual dari proses, yaitu:

- `reductor_consume`
diasumsikan sebesar $67 \times \text{kg_tco}/1000$ (Karena `reductor_consume` dalam ton)
- `total_fuel`
diasumsikan sebesar $115 \times \text{kg_tco}$

Untuk mengakomodasi variasi data aktual dan menghindari batasan yang terlalu ketat, diterapkan **toleransi statistik** menggunakan metode IQR (interquartile range) dan standar deviasi (STD). Rentang batas akhirnya ditentukan dengan menambahkan/mengurangi nilai toleransi ini terhadap estimasi utama, sesuai distribusi masing-masing variabel. Pendekatan ini memungkinkan fleksibilitas tanpa mengabaikan batas wajar dari data historis.

- **Skenario (e) – Batasan Dinamis Berdasarkan Median Data:** Berbeda dari skenario (d) yang menggunakan pendekatan estimatif berbasis kapasitas produksi, skenario (e) menggunakan pendekatan statistik murni berdasarkan data historis aktual. Batas atas untuk `reductor_consume` dan `total_fuel` ditentukan menggunakan nilai median (kuartil kedua, Q2) dari distribusi masing-masing variabel selama periode historis. Tidak digunakan estimasi energi teoritis atau referensi eksternal dalam penentuan batas. Tidak hanya itu, variabel yang memiliki korelasi kuat dengan `total_fuel` dan `reductor_consume` yaitu `rpm`, `pressure`, dan `air velocity` juga dibatasi dengan nilai Q2. Sehingga semakin menekan nilai penggunaan `total_fuel` dan `reductor_consume`. Pendekatan ini dipilih atas dasar beberapa pertimbangan:

- Ketahanan terhadap outlier
Median lebih stabil dan tidak dipengaruhi oleh nilai ekstrem, sehingga cocok merepresentasikan kondisi tipikal operasi.
- Representasi realistik
Median mencerminkan nilai tengah dari data historis, yang secara statistik lebih mencerminkan nilai lazim dalam operasi aktual dibandingkan maksimum atau rata-rata.

- Koheren dengan tujuan efisiensi

Dengan membatasi konsumsi energi dan reduktan pada atau di bawah nilai median, eksperimen diarahkan untuk mencari konfigurasi parameter yang lebih efisien daripada praktik umum di masa lalu.

Setiap skenario dievaluasi dengan dua metode optimasi, yaitu SLSQP dan Genetic Algorithm (GA), sehingga menghasilkan total sepuluh kombinasi eksperimen sebagai berikut:

Tabel 3.6: Daftar Skenario Eksperimen dan Keterangan

Kode Eksperimen	Metode	Deskripsi
SL-SQP.a	SLSQP	Batas atas total_fuel dan reductor_consume dari nilai aktual
GA.a	GA	Versi GA dari konfigurasi 1a
SL-SQP.b	SLSQP	Tanpa batas atas total_fuel dan reductor_consume
GA.b	GA	Versi GA dari konfigurasi 1b
SL-SQP.c	SLSQP	Batas bawah konsumsi total_fuel dan reductor_consume dari kondisi off (non-produktif)
GA.c	GA	Versi GA dari konfigurasi 1c
SL-SQP.d	SLSQP	Estimasi energi berbasis referensi akademik dan $\pm$ toleransi (IQR/STD)
GA.d	GA	Versi GA dari konfigurasi 1d
SL-SQP.e	SLSQP	Batas atas pada kolom korelasi kuat dengan total_fuel dan reductor_consume menggunakan median (Q2)
GA.e	GA	Versi GA dari konfigurasi 1e

3.4 Struktur Eksperimen dan Tahapan Analisis

Eksperimen ini dirancang dalam dua tahap utama untuk memperoleh hasil yang sistematis dan dapat dibandingkan secara objektif. Fokus penelitian ini adalah perbandingan langsung antara metode Sequential Least Squares Programming (SLSQP) dan Genetic Algorithm (GA). Evaluasi di setiap tahap akan menggunakan serangkaian metrik performa yang komprehensif untuk memberikan gambaran menyeluruh tentang kinerja setiap skenario dan metode optimisasi.

3.4.1 Tahap 1 – Evaluasi Antar-Skenario dengan Parameter *Default*

Pada tahap ini, semua skenario batasan (dari a hingga e) akan dijalankan menggunakan **parameter default** yang telah ditentukan untuk masing-masing metode optimisasi (SLSQP dan GA). Tujuan utama tahap ini adalah untuk mengidentifikasi skenario batasan yang paling menjanjikan dari sisi performa optimisasi, khususnya dalam hal penurunan konsumsi energi (*total_fuel* dan *reductor_consume*) tanpa mengorbankan kualitas *output* metalurgi. Hasil dari tahap ini akan menjadi dasar untuk menentukan satu skenario terbaik yang akan digunakan pada perbandingan lintas-metode di Tahap 2.

3.4.1.1 Mekanisme Evaluasi Tahap 1:

Evaluasi pada tahap ini akan menggunakan lima metrik performa utama sebagai berikut:

1. Efisiensi Energi

Diukur sebagai **pengurangan konsumsi energi** relatif terhadap nilai aktual historis (sebagai *baseline*). Lebih spesifiknya, efisiensi energi dihitung sebagai selisih antara konsumsi energi historis dan konsumsi energi hasil prediksi setelah optimisasi, untuk variabel *total_fuel* dan *reductor_consume*.

Persamaan:

$$E = C_{hist} - C_{pred} \quad (3.2)$$

Di mana:

- E adalah Efisiensi Energi.
- C_{hist} adalah Konsumsi Energi Historis (nilai aktual dari *baseline*).
- C_{pred} adalah Konsumsi Energi Prediksi Optimisasi (nilai konsumsi energi yang dihasilkan oleh model setelah optimisasi).
- Nilai positif E menunjukkan adanya **penghematan energi** (konsumsi lebih rendah dari *baseline*).
- Nilai negatif E menunjukkan **peningkatan konsumsi energi** (konsumsi lebih tinggi dari *baseline*).

Semakin besar nilai positif E , menunjukkan semakin efisien skenario tersebut dalam hal penghematan energi.

2. Kewajaran Solusi (*Feasibility*) - Rata-rata Jumlah *Outlier*

Evaluasi kewajaran solusi dilakukan untuk memastikan bahwa nilai-nilai parameter operasional (*input* dan/atau *output* yang relevan) yang dihasilkan oleh optimasi masih berada dalam rentang yang masuk akal secara teknis dan operasional. Mekanismenya sebagai berikut:

- (a) Untuk setiap variabel relevan (misalnya, arus, beban, suhu, atau parameter kualitas metalurgi), jumlah *outlier* dihitung dari hasil optimasi. *Outlier* didefinisikan sebagai nilai yang berada di luar rentang interkuartil (IQR) dari data historis aktual variabel tersebut, yaitu lebih kecil dari $Q1 - 1.5 \times IQR$ atau lebih besar dari $Q3 + 1.5 \times IQR$.
- (b) Rata-rata jumlah *outlier* dari seluruh variabel relevan kemudian dihitung untuk setiap skenario atau metode. Nilai ini akan menjadi representasi tunggal untuk metrik "Kewajaran Solusi".

Semakin rendah rata-rata jumlah *outlier*, menunjukkan bahwa hasil optimasi lebih realistis, konsisten dengan pola data historis, dan dekat dengan kondisi operasi aktual. Solusi dengan *outlier* yang banyak mungkin mengindikasikan bahwa optimasi menghasilkan nilai-nilai ekstrem yang tidak praktis atau sulit dicapai dalam operasional pabrik. Penggunaan rata-rata *outlier* untuk mengukur kewajaran solusi adalah pendekatan umum dalam evaluasi model dan algoritma yang membandingkan keluaran dengan pola data yang diharapkan [33].

3. Jumlah Pelanggaran *Constraint*

Dihitung dari banyaknya variabel yang menghasilkan nilai di luar rentang batasan yang telah didefinisikan dalam setiap skenario. Batasan ini mencakup batas atas dan/atau batas bawah parameter proses. Semakin sedikit pelanggaran, semakin baik kualitas solusi dari segi kepatuhan terhadap ruang solusi yang dikehendaki.

4. Waktu/Durasi Komputasi

Diukur dalam satuan detik, mencerminkan total durasi yang dibutuhkan oleh algoritma optimisasi untuk menyelesaikan proses dan mencapai solusi akhir pada setiap skenario. Metrik ini penting untuk menilai efisiensi komputasi dan kelayakan implementasi *real-time*.

5. Akurasi *Output* (Rata-rata Deviasi)

Dievaluasi berdasarkan nilai deviasi antara hasil prediksi model setelah optimasi dan nilai aktual historis untuk berbagai parameter *output* metalurgi (mis-

alnya, kadar nikel, suhu logam, dll.). Nilai deviasi yang rendah menunjukkan bahwa solusi yang diusulkan mampu menjaga kualitas produk dan kondisi operasional dalam rentang yang diharapkan, yaitu tidak jauh dari nilai aktual. Mekanisme evaluasi dan penskorannya adalah sebagai berikut:

- (a) Untuk setiap variabel *output* relevan, nilai deviasi dihitung dari hasil optimasi. Deviasi ini mengukur seberapa jauh nilai hasil optimasi menyimpang dari nilai aktual historis yang diinginkan atau yang diharapkan.
- (b) Rata-rata nilai deviasi dari seluruh variabel *output* relevan kemudian dihitung untuk setiap skenario atau metode. Nilai ini akan menjadi representasi tunggal untuk metrik "Akurasi *Output*".

Semakin rendah rata-rata nilai deviasi, menunjukkan bahwa optimasi berhasil mempertahankan kualitas *output* yang baik dan konsisten dengan standar yang diharapkan. Penggunaan rata-rata deviasi ini sejalan dengan praktik evaluasi model yang membandingkan keluaran prediksi dengan nilai aktual [33].

Penentuan Skenario Terbaik dengan Weighted Sum Method (WSM) Untuk mengidentifikasi skenario terbaik dari Tahap 1, kelima metrik performa di atas akan digabungkan menjadi satu skor komposit menggunakan **Weighted Sum Method (WSM)** [34]. Pendekatan ini merupakan bagian dari kerangka kerja *Multi-Criteria Decision Making* (MCDM) yang memungkinkan evaluasi sistematis alternatif berdasarkan berbagai kriteria dengan prioritas yang berbeda.

Tahapan perhitungan skor komposit adalah sebagai berikut:

1. **Normalisasi Metrik:** Setiap metrik performa akan dinormalisasi ke dalam skala umum (misalnya, dari 0 hingga 1). Normalisasi ini penting karena setiap metrik memiliki satuan dan rentang nilai yang berbeda.

- Untuk metrik di mana **nilai lebih tinggi lebih baik** (misalnya, Efisiensi Energi), rumus normalisasi yang digunakan adalah:

$$\text{Normalized Value} = \frac{\text{Value} - \text{Min Value}}{\text{Max Value} - \text{Min Value}} \quad (3.3)$$

- Untuk metrik di mana **nilai lebih rendah lebih baik** (misalnya, Jumlah Pelanggaran *Constraint*, Rata-rata Jumlah *Outlier*, Rata-rata Deviasi, Waktu/Durasi Komputasi), rumus normalisasi yang digunakan adalah:

$$\text{Normalized Value} = 1 - \left(\frac{\text{Value} - \text{Min Value}}{\text{Max Value} - \text{Min Value}} \right) \quad (3.4)$$

2. **Penentuan Bobot (Prioritas):** Bobot (w_i) akan diberikan kepada setiap metrik performa sesuai dengan tingkat kepentingannya dalam konteks penelitian. Prioritas yang digunakan adalah:

- (a) Efisiensi Energi (prioritas tertinggi)
- (b) Jumlah Pelanggaran *Constraint*
- (c) Rata-rata Jumlah *Outlier*
- (d) Rata-rata Deviasi
- (e) Waktu/Durasi Komputasi (prioritas terendah)

Jumlah total bobot harus sama dengan 1 ($\sum w_i = 1$). Bobot spesifik akan ditentukan berdasarkan konsultasi dengan ahli domain atau melalui metode penentuan bobot standar (misalnya, *Analytic Hierarchy Process* jika diperlukan, meskipun tidak dijelaskan di sini).

3. **Perhitungan Skor Komposit:** Skor komposit untuk setiap skenario (S_j) dihitung sebagai jumlah dari produk bobot masing-masing metrik (w_i) dengan nilai normalisasinya ($Normalized_Value(C_{ij})$):

$$Score(S_j) = \sum_{i=1}^n w_i \times Normalized_Value(C_{ij}) \quad (3.5)$$

Skenario dengan skor komposit tertinggi akan dipilih sebagai skenario terbaik untuk perbandingan lintas-metode di Tahap 2.

Tabel 3.7: Parameter Default Optimisasi untuk Tahap Evaluasi Antar-Skenario

Metode	Parameter	Nilai
SLSQP	maxiter	100
	eps	1.5×10^{-8}
GA	population_size	10
	generations	10
	mutation_rate	0.2
	selection_method	elitism

3.4.2 Tahap 2 – Perbandingan Lintas-Metode pada Skenario Terbaik

Setelah skenario terbaik terpilih dari Tahap 1, perbandingan metrik akan dilakukan antara metode SLSQP dan GA pada skenario terbaik tersebut.

3.4.2.1 Mekanisme Evaluasi Tahap 2

Evaluasi pada tahap ini akan secara komprehensif mempertimbangkan kelima metrik performa yang sama seperti pada Tahap 1, yaitu:

(a) **Efisiensi Energi**

Membandingkan kemampuan masing-masing metode (SLSQP vs. GA) dalam mencapai penghematan konsumsi bahan bakar yang optimal pada skenario terbaik. Nilai rata-rata dari kolom *total_fuel* dan *reductor_consume* akan dijumlah dan didapatkan total penghematan. Nilai inilah yang akan dibandingkan antar metode.

(b) **Kewajaran Solusi (Rata-rata Jumlah *Outlier*)**

Membandingkan rata-rata jumlah *outlier* yang dihasilkan oleh SLSQP dan GA dengan distribusi data aktual, untuk memastikan hasil optimasi tetap berada dalam batasan operasional yang masuk akal. Metode dengan rata-rata jumlah *outlier* yang lebih rendah (menunjukkan solusi lebih wajar) akan dianggap lebih baik.

(c) **Jumlah Pelanggaran *Constraint***

Mengevaluasi kepatuhan kedua metode terhadap batasan-batasan proses yang diterapkan pada skenario terbaik, menunjukkan seberapa baik mereka menavigasi ruang solusi yang diizinkan.

(d) **Waktu/Durasi Komputasi**

Membandingkan efisiensi komputasi SLSQP dan GA, mengidentifikasi metode mana yang lebih cepat mencapai solusi pada tingkat kompleksitas yang sama.

(e) **Akurasi *Output* (Rata-rata Deviasi)**

Menganalisis seberapa baik SLSQP dan GA dalam menjaga kualitas *output* metalurgi mendekati target yang diinginkan sambil mencapai tujuan optimisasi lainnya. Metode dengan rata-rata deviasi yang lebih rendah (menunjukkan akurasi *output* lebih baik) akan dianggap lebih baik.

Penentuan Metode Terbaik dengan **Weighted Sum Method (WSM)**

Serupa dengan Tahap 1, pada Tahap 2 ini, kedua metode (SLSQP dan GA) akan dievaluasi menggunakan **Weighted Sum Method (WSM)** [34] untuk mendapatkan skor komposit tunggal. Metrik performa yang sama akan digunakan, dan masing-masing akan dinormalisasi serta dibobot sesuai dengan

prioritas yang telah ditentukan. Metode dengan skor komposit tertinggi akan dianggap sebagai metode terbaik untuk optimisasi efisiensi energi proses feronikel.

Tahap ini bertujuan untuk secara langsung menjawab pertanyaan utama penelitian: membandingkan dan memilih metode yang lebih efektif dan efisien untuk diterapkan dalam konteks optimisasi efisiensi energi proses feronikel, baik dari sisi kualitas hasil yang dicapai maupun biaya komputasi yang diperlukan.

3.5 Implementasi Constraint pada Kode

Implementasi *constraint* pada algoritma optimisasi seperti Genetic Algorithm (GA) dan Sequential Least Squares Programming (SLSQP) memiliki pendekatan yang berbeda, terutama dalam cara *constraint* tersebut diterapkan dan dikelola selama proses optimisasi. Perbedaan utama terletak pada penempatan dan mekanisme penanganan *constraint* dalam struktur kedua algoritma.

3.5.1 Genetic Algorithm (GA)

Pada Genetic Algorithm, *constraint* umumnya diimplementasikan dengan dua cara utama yaitu, selama pembuatan atau modifikasi kromosom (generasi awal dan *mating*) dan melalui fungsi *fitness*.

3.5.1.1 Constraint Selama Pembuatan Kromosom (Chromosome Creation)

Seperti yang ditunjukkan dalam pseudocode 1 di bawah, *constraint* diintegrasikan saat kromosom diinisialisasi. Saat setiap gen (parameter optimisasi) dalam kromosom dibuat, nilainya dibatasi dalam rentang *min_value* dan *max_value* yang telah ditentukan. Ini memastikan bahwa setiap kromosom yang dihasilkan secara acak sudah memenuhi *constraint* batas bawah dan batas atas.

Algorithm 1 Pembuatan Kromosom

Require: scaled_input_row**Ensure:** chromosome

```

1: chromosome  $\leftarrow$  empty LIST
2: for each optimization_parameter in set_of_optimization_parameters
   do
3:   min_value  $\leftarrow$  undefined
4:   max_value  $\leftarrow$  undefined
5:   if optimization_parameter is a TYPE_A_PARAMETER then
6:     min_value  $\leftarrow$  minimum global value
7:     max_value  $\leftarrow$  value from scaled_input_row
8:   else if optimization_parameter is a TYPE_B_PARAMETER then
9:     min_value  $\leftarrow$  predefined lower bound
10:    max_value  $\leftarrow$  predefined upper bound
11:   else
12:     min_value  $\leftarrow$  global minimum
13:     max_value  $\leftarrow$  global maximum
14:   end if
15:   random_value  $\leftarrow$  uniform(min_value, max_value)
16:   add random_value to chromosome
17: end for
18: return chromosome

```

3.5.1.2 Constraint Selama Mating (Crossover dan Mutasi)

Pseudocode 2 juga menunjukkan bagaimana *constraint* dipertimbangkan selama proses perkawinan (crossover dan mutasi) untuk menghasilkan anak kromosom.

- **Penjagaan Constraint Saat Mutasi**

Ketika terjadi mutasi ($\text{probability} \geq 0.8$), nilai gen anak yang baru dihasilkan juga dibatasi oleh min_gene_value dan max_gene_value. Ini mencegah *offspring* yang dihasilkan dari operasi genetika melanggar *constraint* yang telah ditetapkan.

- **Contoh Penerapan**

Mirip dengan pembuatan kromosom, jika gen anak mengalami mutasi, nilainya akan disesuaikan agar tetap dalam batasan yang diizinkan, seperti SPECIAL_GENE_TYPE_1 yang memiliki batas tertentu.

Algorithm 2 Perkawinan Kromosom

Require: self_chromosome**Require:** partner_chromosome**Ensure:** child_chromosome

```

1: child_chromosome ← empty LIST
2: for each gene in self_chromosome do
3:   probability ← random number between 0 and 1
4:   if probability < 0.4 then
5:     child_gene_value ← self_chromosome's gene value
6:   else if probability < 0.8 then
7:     child_gene_value ← partner_chromosome's gene value
8:   else
9:     min_gene_value ← undefined
10:    max_gene_value ← undefined
11:    if gene is a SPECIAL_GENE_TYPE_1 then
12:      min_gene_value ← predefined lower limit
13:      max_gene_value ← predefined upper limit
14:    else
15:      min_gene_value ← global minimum for this gene
16:      max_gene_value ← global maximum for this gene
17:    end if
18:    child_gene_value ← random between min and max_gene_value
19:  end if
20:  add child_gene_value to child_chromosome
21: end for
22: return child_chromosome

```

3.5.1.3 Constraint Melalui Fungsi Fitness (Penalti)

Pseudocode 3 menggambarkan pendekatan umum untuk penanganan *constraint* yang "lunak" atau untuk *constraint* yang mungkin dilanggar selama proses mutasi/*crossover*.

- Sistem Penalti

Jika *output* dari model (*output_val*) melanggar batas (*low_bound* atau *up_bound*), sebuah *penalty* akan ditambahkan ke *fitness_score*. Semakin jauh pelanggaran dari batas, semakin besar penaltinya (dikalikan dengan *multiplier*).

- Tujuan

Pendekatan ini secara efektif "menghukum" kromosom yang melanggar *constraint* dengan memberikan *fitness* yang lebih buruk. Ini mendorong algoritma untuk mencari solusi yang tidak melanggar *constraint* karena

solusi tersebut akan memiliki *fitness* yang lebih baik.

- **Fleksibilitas**

Pendekatan ini memungkinkan GA untuk menjelajahi area solusi yang lebih luas, bahkan jika ada pelanggaran *constraint* sementara, dan kemudian secara bertahap mengarahkan populasi ke solusi yang layak. Ini juga berguna untuk *constraint* yang lebih kompleks (misalnya, *constraint* persamaan atau ketidaksetaraan non-linear) yang tidak mudah diintegrasikan langsung ke dalam pembuatan gen.

Algorithm 3 Perhitungan *Fitness*

Require: chromosome

Require: input_data_row

Ensure: total_fitness_score

```

1: processed_input ← input_data_row's initial features
2: inject chromosome into processed_input
3: pred_output ← model.predict(processed_input)
4: pred_error ← abs diff between pred_output and target_values
5: fitness_score ← sum of pred_error
6: penalty ← 0
7: output_val ← specific output value from pred_output
8: if output_val < low_bound then
9:   penalty ← penalty + (low_bound - output_val) × multiplier
10: else if output_val > up_bound then
11:   penalty ← penalty + (output_val - up_bound) × multiplier
12: end if
13: return fitness_score + penalty

```

3.5.2 Sequential Least Squares Programming (SLSQP)

Berbeda dengan GA yang dapat menangani constraint secara implisit atau melalui penalti, SLSQP adalah algoritma optimisasi berbasis gradien yang secara eksplisit membutuhkan definisi constraint matematis sebelum proses optimisasi dimulai. SLSQP umumnya menangani constraint dalam dua kategori utama: batas (bounds) untuk variabel, dan constraint non-linear yang didefinisikan sebagai fungsi (persamaan dan ketidaksetaraan).

3.5.2.1 Constraint Sebagai Batas (Bounds)

Batas (bounds) adalah constraint paling dasar dalam optimisasi SLSQP, yang menentukan rentang nilai minimum dan maksimum yang diizinkan untuk se-

tiap variabel yang dioptimalkan. Algoritma 4 menunjukkan bagaimana batas-batas ini diinisialisasi dan disiapkan.

Sebelum optimisasi, batas bawah (`min_values_input`) dan batas atas (`max_values_input`) untuk setiap parameter yang akan dioptimalkan harus didefinisikan secara eksplisit. Kumpulan batas ini akan diteruskan ke algoritma SLSQP. Selama proses inisialisasi, nilai-nilai awal untuk parameter yang akan dioptimalkan (`opt_value`) juga ditentukan. Nilai-nilai ini, bersama dengan parameter yang ditetapkan (`fixed_value`) dari data input awal (`raw_input_data`), membentuk titik awal pencarian optimisasi. Selain itu, nilai-nilai parameter yang dioptimalkan dapat dinormalisasi (`norm_opt_val`) ke rentang 0 hingga 1 untuk membantu stabilitas numerik algoritma optimisasi, terutama ketika variabel memiliki skala yang sangat berbeda. SLSQP secara internal akan memastikan bahwa selama setiap iterasi, nilai-nilai parameter tetap berada dalam batas-batas yang telah ditentukan ini. Jika gradien mengarahkan solusi ke luar batas, algoritma akan memproyeksikan kembali solusi ke batas tersebut atau menyesuaikan arah pencarian untuk tetap berada dalam rentang yang valid.

Algorithm 4 Inisialisasi Nilai *Fixed* dan *Optimized Inputs* untuk SLSQP

Require: `raw_input_data` (baris data inisial untuk parameter)

Require: `min_values_input`, `max_values_input` (batas bawah dan atas untuk setiap parameter)

Ensure: `fixed_value`, `norm_optimized_value`, `bounds`

- 1: `bounds` $\leftarrow$ list of (min,max) pairs from `min_values_input` and `max_values_input`
 - 2: `fixed_value` $\leftarrow$ extract specific value from `raw_input_data`
 - 3: `opt_value` $\leftarrow$ manually defined initial value
 - 4: `min_val` $\leftarrow$ corresponding minimum from `input_data`
 - 5: `max_val` $\leftarrow$ corresponding maximum from `input_data`
 - 6: `norm_opt_val` $\leftarrow$ (`current_opt_value` - `min_val`) / (`max_val` - `min_val`)
 - 7: **return** `fixed_value`, `norm_optimized_value`, `bounds`
-

3.5.2.2 Constraint Sebagai Fungsi (Persamaan dan Ketidaksetaraan)

Selain batas sederhana, SLSQP juga unggul dalam menangani constraint yang lebih kompleks, yang didefinisikan sebagai fungsi yang harus dipenuhi selama optimisasi. Constraint ini dapat berupa persamaan ($h(x) = 0$) atau ketidaksetaraan ($g(x) \leq 0$ atau $g(x) \geq 0$). Algoritma 5 mengilustrasikan bagaimana constraint ini dikonfigurasi dan diteruskan ke fungsi optimisasi utama.

Setiap constraint didefinisikan sebagai objek atau kamus terpisah yang memiliki tipe (e.g., 'eq' untuk persamaan, 'ineq' untuk ketidaksetaraan) dan fungsi yang mengembalikan nilai constraint. Misalnya, untuk constraint persamaan, fungsi harus mengembalikan nilai nol ketika constraint terpenuhi. Untuk constraint ketidaksetaraan $g(x) \geq 0$, fungsi constraint akan mengembalikan nilai $g(x)$, di mana nilai positif menunjukkan bahwa constraint terpenuhi. Contoh-contoh spesifik yang diterapkan dalam algoritma meliputi: menjaga nilai variabel optimisasi tertentu tetap pada nilai inisial (persamaan), memastikan variabel optimisasi berada di atas batas bawah atau di bawah batas atas (ketidaksetaraan), dan memastikan prediksi model (seperti kualitas produk) memenuhi target minimum (ketidaksetaraan). SLSQP menggunakan Lagrangian multipliers dan teknik optimisasi kuadratik sekuensial untuk mencari solusi yang memenuhi constraint ini. Pada setiap iterasi, masalah diaproksimasi sebagai masalah optimisasi kuadratik yang memperhitungkan constraint dan gradien fungsi objektif serta constraint. SLSQP berusaha untuk memenuhi constraint ini secara "keras" (sekuat mungkin), yang berarti solusi yang ditemukan akan secara matematis memenuhi constraint yang diberikan dalam batas toleransi numerik yang kecil. Ini sangat penting untuk aplikasi di mana kepatuhan constraint adalah persyaratan mutlak, seperti batasan operasional atau kualitas produk.

Algorithm 5 Konfigurasi dan Penggunaan Constraint pada Fungsi SLSQP

Require: `lower_bound_norm` (batas bawah ternormalisasi untuk variabel input tertentu)

Require: `upper_bound_norm` (batas atas ternormalisasi untuk variabel input tertentu)

Require: `target_output_norm` (nilai target output ternormalisasi untuk constraint prediksi model)

Require: `model` (model prediksi machine learning)

Require: `initial_data` (nilai awal untuk semua parameter optimisasi)

Require: `obj_function` (fungsi objektif yang akan diminimalkan)

```

1: constraint_list  $\leftarrow$  empty LIST
2: add {'type': 'eq', 'fun': lambda x: specific_opt_var(x) -
   initial_data_val} to constraint_list
3: add {'type': 'ineq', 'fun': lambda x: specific_opt_var(x) -
   low_bound_norm} to constraint_list
4: add {'type': 'ineq', 'fun': lambda x: up_bound_norm -
   specific_opt_var(x)} to constraint_list
5: add {'type': 'ineq', 'fun': lambda x:
   model.predict(current_var(x)) - target_output_norm} to
   constraint_list
6: optimization_result  $\leftarrow$  minimize(
   fun=obj_function,
   x0=initial_data,
   method='SLSQP',
   constraints=constraint_list,
   options='maxiter': 100, 'eps': 1.5e-8
)
7: return optimization_result

```

3.5.3 Perbedaan Penempatan *Constraint*

Dalam proses optimisasi, penanganan *constraint* (kendala) menjadi elemen penting yang membedakan antara berbagai pendekatan algoritmik. Dua metode yang digunakan dalam penelitian ini yaitu, Genetic Algorithm (GA) dan Sequential Least Squares Programming (SLSQP) memiliki cara yang berbeda dalam mengelola dan menerapkan *constraint* terhadap variabel input maupun output.

Secara umum, GA mengintegrasikan *constraint* ke dalam proses evolusi populasi, baik saat pembuatan kromosom, proses mutasi dan kombinasi, maupun pada evaluasi *fitness*. Dalam praktiknya, pelanggaran *constraint* seringkali tidak langsung menggugurkan solusi, melainkan diberi penalti tertentu dalam fungsi objektif. Sementara itu, SLSQP sebagai metode optimisasi berbasis

gradien mengharuskan seluruh *constraint* didefinisikan secara eksplisit sejak awal proses. Algoritma ini hanya akan menjelajahi ruang solusi yang sepenuhnya memenuhi batas dan ketentuan yang telah ditentukan.

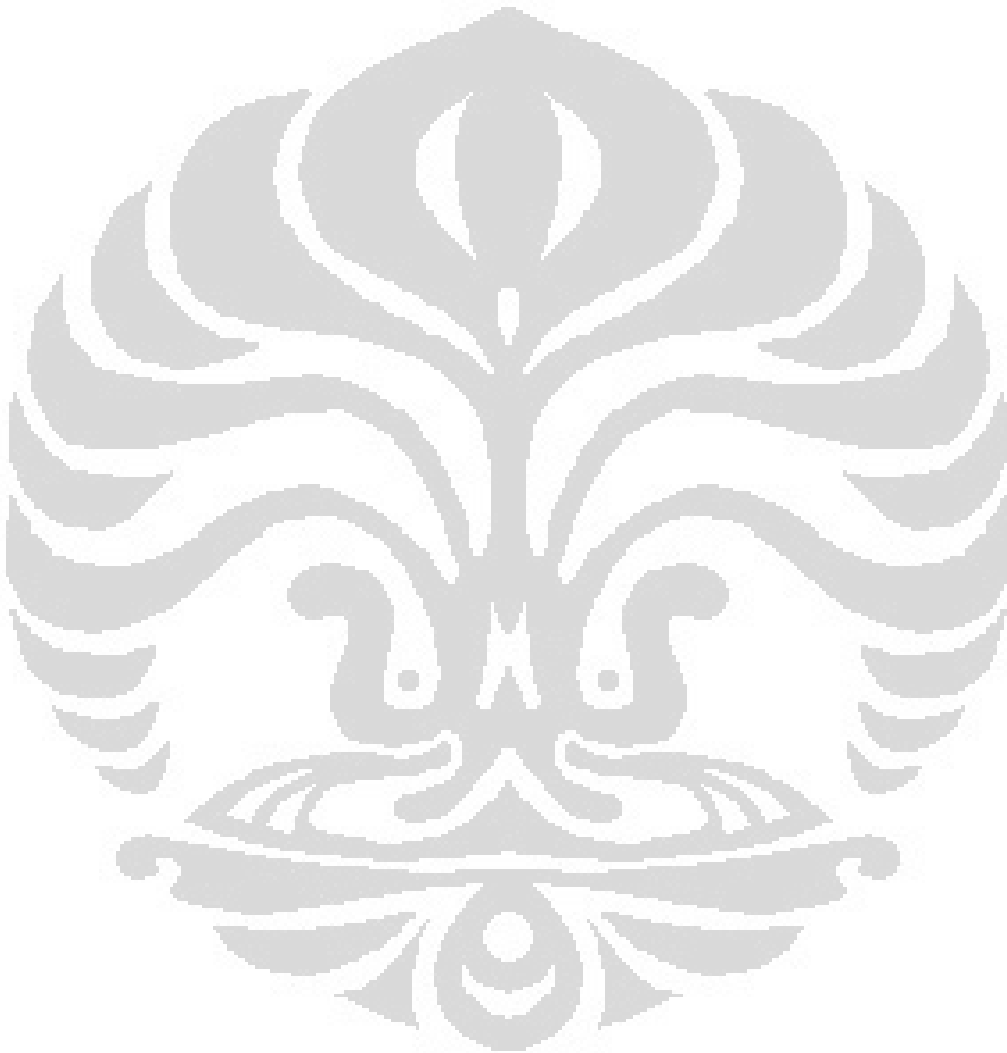
Tabel 3.8 berikut merangkum perbedaan utama dalam penempatan dan pengelolaan *constraint* antara kedua metode tersebut.

Tabel 3.8: Perbedaan Penempatan *Constraint* antara GA dan SLSQP

Fitur	Genetic Algorithm (GA)	SLSQP
Lokasi Utama	Selama generasi kromosom, mating, dan fungsi <i>fitness</i> (penalti).	Didefinisikan secara eksplisit sebagai batas (<i>bounds</i>) dan fungsi (<i>constraint</i>) sebelum optimisasi dimulai.
Pendekatan	Cenderung "lunak" dengan sistem penalti untuk pelanggaran, atau "keras" dengan penjagaan nilai saat pembuatan/mutasi gen.	Secara matematis "keras" dan mencoba memenuhi <i>constraint</i> secara tepat.
Fleksibilitas	Lebih toleran terhadap pelanggaran <i>constraint</i> sementara; dapat menjelajahi ruang solusi yang lebih luas.	Membutuhkan definisi matematis yang jelas dari <i>constraint</i> dan tidak akan mengeksplorasi di luar <i>constraint</i> yang ditentukan.
Tipe Constraint	Batas sederhana dapat langsung diintegrasikan. <i>Constraint</i> yang lebih kompleks sering diatasi melalui sistem penalti.	Batas dan <i>constraint</i> persamaan/ketidaksetaraan linear maupun non-linear dapat ditangani secara langsung dan eksplisit.
Contoh Penerapan	Kontrol rentang nilai saat inialisasi/mutasi; penalti pada <i>fitness</i> jika <i>output</i> model di luar batas yang diinginkan.	Mendefinisikan <code>min_values</code> dan <code>max_values</code> untuk setiap variabel, serta fungsi terpisah untuk <i>constraint</i> seperti $f(x) \leq 0$ atau $h(x) = 0$.

Dari tabel tersebut, terlihat bahwa GA cenderung mengelola *constraint* secara lebih integratif ke dalam siklus hidup populasi—baik saat pembentukan individu, evolusi generasi, maupun evaluasi kelayakan solusi. Pendekatan ini memungkinkan GA untuk tetap mengeksplorasi solusi potensial meskipun sementara melanggar batasan, selama hasil akhirnya dapat dikompensasi de-

ngan *fitness* yang baik. Sebaliknya, SLSQP hanya akan mengevaluasi solusi yang benar-benar memenuhi *constraint* yang telah ditentukan secara matematis sejak awal, menjadikannya pendekatan yang lebih ketat namun juga lebih terkontrol secara numerik.



BAB 4

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil dari eksperimen optimasi proses produksi feronikel menggunakan dua metode, yaitu *Sequential Least Squares Programming* (SLSQP) dan *Genetic Algorithm* (GA). Evaluasi dilakukan berdasarkan efisiensi konsumsi energi, pelanggaran terhadap constraint, kewajaran nilai input, serta waktu komputasi. Setiap skenario pada masing-masing metode dianalisis secara menyeluruh, kemudian dilakukan perbandingan antar metode dan interpretasi hasil terbaik.

4.1 Hasil Optimasi Menggunakan Genetic Algorithm (GA)

4.1.1 Perbandingan Efisiensi Energi

Efisiensi konsumsi energi dievaluasi berdasarkan dua indikator utama: penghematan konsumsi `total_fuel` dan `reductor_consume` (dalam satuan ton/jam) dibandingkan dengan nilai aktual historis. Pengukuran efisiensi ini dihitung menggunakan Persamaan 3.2. Semakin besar nilai efisiensi (E_{eff}) yang positif, semakin besar penghematan energi yang dicapai oleh skenario tersebut.

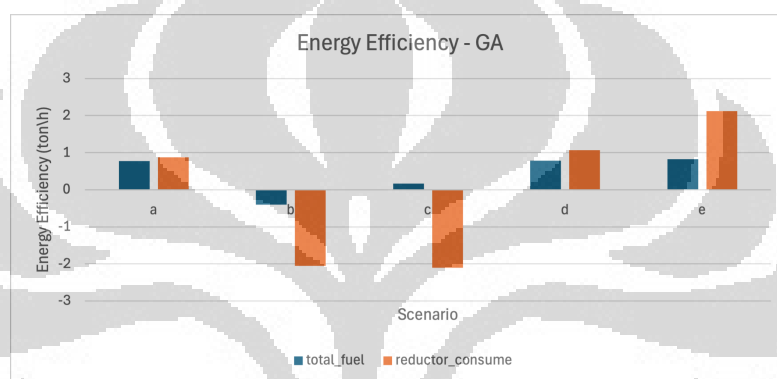
Tabel 4.1: Nilai Efisiensi Konsumsi Energi Tiap Skenario

Skenario	Efisiensi Total Fuel (ton/h)	Efisiensi Reduktor (ton/h)
a	0.77	0.88
b	-0.40	-2.06
c	0.17	-2.10
d	0.78	1.07
e	0.83	2.12

Hasil evaluasi pada Tabel 4.1 menunjukkan bahwa skenario (e) memberikan penghematan energi paling tinggi. Skenario ini menghasilkan efisiensi sebesar 0.83 ton/jam untuk konsumsi `total_fuel` dan 2.12 ton/jam untuk `reductor_consume`. Ini berarti, dalam satu hari (24 jam), skenario (e) mampu menghemat $0.82 \times 24 = 19.83$ ton `total_fuel` dan $2.12 \times 24 = 50.91$

ton reductor_consume. Penghematan yang signifikan ini menunjukkan bahwa pendekatan dengan batas atas berbasis median historis (Q2) untuk variabel input terbukti sangat efektif dalam menghasilkan solusi yang hemat energi.

Sementara itu, skenario (d) yang menggunakan batasan dinamis berdasarkan literatur juga menghasilkan performa efisiensi yang baik, dengan efisiensi total_fuel sebesar 0.78 ton/jam dan reductor_consume sebesar 1.07 ton/jam. Kedua skenario ini relatif unggul karena memiliki pendekatan batasan input yang lebih rasional dan kontekstual dibandingkan pendekatan lainnya.



Gambar 4.1: Nilai Efisiensi Metode GA

Sebaliknya, skenario (b) dan (c) menunjukkan hasil yang negatif, yang mengindikasikan bahwa hasil optimasi justru meningkatkan konsumsi energi. Ini terjadi karena pada skenario (b), tidak terdapat batasan maksimum pada konsumsi energi, sehingga model bebas menghasilkan solusi yang secara prediksi mungkin valid tetapi sangat boros energi. Pada skenario (c), pembatasan bawah berdasarkan kondisi off juga tampaknya terlalu longgar, sehingga solusi tidak cukup terkendali dalam hal efisiensi.

Secara umum, pendekatan yang menggabungkan batasan berbasis distribusi historis yang representatif, seperti median atau IQR terbukti lebih efektif dalam mendorong penghematan energi dibandingkan skenario dengan batasan minimum longgar atau tanpa batas maksimum sama sekali.

4.1.2 Evaluasi Kewajaran Solusi (Feasibility)

Evaluasi kewajaran solusi dilakukan dengan menganalisis seberapa sering nilai-nilai hasil optimasi berada di luar rentang data historis yang wajar, atau

yang dikenal sebagai outlier. Deteksi outlier didasarkan pada rentang interkuartil (IQR) dari data historis pada masing-masing variabel. Variabel-variabel yang dievaluasi mencakup parameter operasional kunci dalam proses produksi feronikel, seperti arus listrik (current_pry, current_sec1, current_sec2, current_sec3), beban (load, realisasi_beban), putaran (rpm), tekanan (pry_p, sec_p), tegangan (pry_v, sec_v), serta konsumsi bahan bakar (total_fuel) dan reduktor (reductor), dan suhu (t_tic162, t_tic163).

Tabel 4.2 menyajikan jumlah outlier untuk setiap variabel hasil optimasi pada setiap skenario yang diuji. Angka-angka dalam tabel ini menunjukkan seberapa sering suatu skenario menghasilkan nilai yang tidak biasa dibandingkan dengan data historis. Semakin kecil jumlah outlier, semakin wajar dan realistis solusi yang dihasilkan oleh skenario tersebut.

Tabel 4.2: Jumlah Outlier dari Hasil Optimasi Tiap Skenario

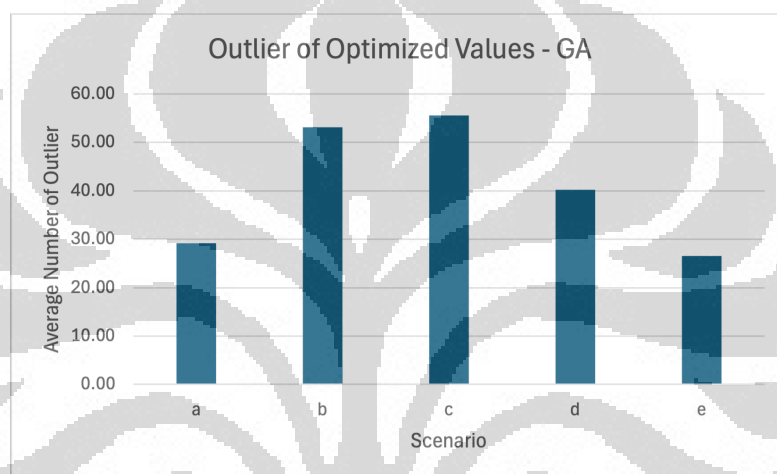
Parameter	a	b	c	d	e
current_pry	171	195	166	168	103
current_sec1	29	33	41	41	44
current_sec2	2	4	2	7	2
current_sec3	11	12	12	7	23
load	3	4	4	3	8
realisasi_beban	1	1	2	4	1
rpm	0	0	0	0	0
pry_p	0	5	6	5	3
sec_p	21	21	16	23	3
pry_v	11	6	5	4	14
sec_v	59	32	38	37	140
total_fuel	30	26	74	32	29
reductor	0	0	0	0	0
t_tic162	1	145	151	161	19
t_tic163	99	313	316	112	9
AVG	29.2	53.1	55.5	40.3	26.5

Berdasarkan analisis jumlah outlier, skenario **e** menghasilkan jumlah outlier rata-rata terendah, yaitu **26.5**. Ini menunjukkan bahwa skenario **e** menghasilkan solusi yang paling wajar dan paling mendekati pola data historis dari segi distribusi nilai hasil optimasi. Mayoritas nilai hasil optimasi pada skenario ini cenderung berada dalam rentang operasional yang umum dijumpai.

Visualisasi distribusi nilai hasil optimasi pada Gambar 4.2 memperkuat temuan ini, menunjukkan bahwa skenario **e** memiliki lebih sedikit nilai ekstrem dibandingkan skenario lainnya.

Skenario **b** dan **c**, yang kemungkinan menerapkan batasan yang longgar atau tanpa batas atas, justru menghasilkan jumlah outlier yang signifikan, terutama pada variabel suhu seperti `t_tic163`. Hal ini berdampak buruk pada kewajaran sistem dari sudut pandang operasional, karena menghasilkan nilai yang tidak realistis atau sulit dicapai dalam kondisi operasional sebenarnya.

Skenario **a**, meskipun tidak memiliki rata-rata outlier terendah, menunjukkan keseimbangan yang baik dalam distribusi outlier pada berbagai variabel. Sementara skenario **d** memiliki beberapa outlier yang menonjol pada variabel tertentu seperti `t_tic162` dan `current_sec2`.



Gambar 4.2: Nilai Kewajaran Input Metode GA

4.1.3 Evaluasi Pelanggaran Constraint

Evaluasi terhadap pelanggaran constraint bertujuan untuk menilai sejauh mana solusi hasil optimasi mematuhi batasan-batasan yang telah ditetapkan. Constraint pada eksperimen ini mencakup batas atas dan/atau batas bawah dari parameter-parameter proses yang ditentukan berdasarkan data historis, referensi akademik, atau pendekatan statistik, seperti IQR dan standar deviasi.

Jumlah pelanggaran dihitung berdasarkan banyaknya variabel yang melampaui batas minimum atau maksimum yang telah didefinisikan dalam setiap skenario. Semakin sedikit pelanggaran yang terjadi, maka semakin baik kualitas solusi dari segi kepatuhan terhadap ruang solusi yang dikehendaki.

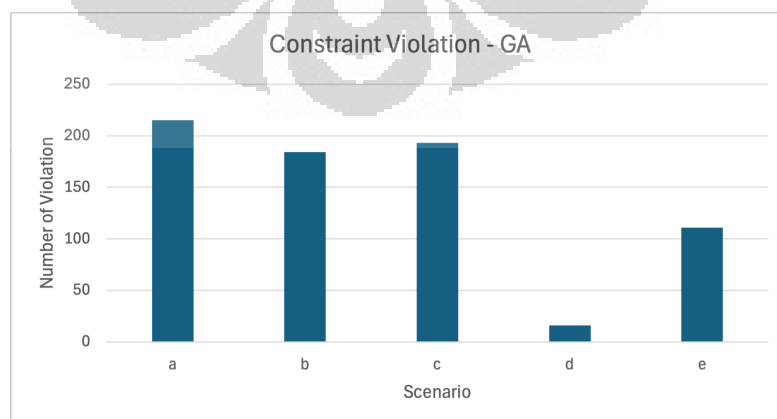
Tabel 4.3: Jumlah Pelanggaran Constraint per Skenario

Skenario	Jumlah Pelanggaran
a	215
b	184
c	193
d	16
e	111

Dari Tabel 4.3 dapat dilihat bahwa skenario (d) menghasilkan jumlah pelanggaran constraint paling sedikit, yaitu hanya 16 pelanggaran. Ini menandakan bahwa konfigurasi batasan berbasis referensi akademik dengan penyesuaian toleransi (menggunakan IQR/standar deviasi) sangat efektif dalam membatasi ruang solusi sehingga tetap realistis namun tidak terlalu ketat.

Sementara itu, skenario (a), (b), dan (c) menunjukkan jumlah pelanggaran yang cukup tinggi. Hal ini sebagian besar disebabkan oleh definisi constraint yang terlalu sempit atau terlalu longgar. Sebagai contoh, pada skenario (a), meskipun batas atas ditentukan berdasarkan nilai aktual, ketidakhadiran batas bawah menyebabkan model berusaha menurunkan konsumsi bahan bakar secara ekstrem, yang berdampak pada pelanggaran di variabel-variabel lain yang terhubung secara kausal.

Skenario (e) menampilkan hasil kompromi yang cukup baik dengan total 111 pelanggaran. Meskipun bukan yang paling sedikit, jumlah pelanggaran tersebut cukup rendah jika dibandingkan dengan skenario lain, dan hal ini sejalan dengan pendekatan constraint yang lebih adaptif, yaitu berdasarkan korelasi kuat dan median historis. Dalam konteks praktis, pendekatan ini lebih fleksibel dibanding skenario (d), namun tetap memberikan kontrol terhadap kecenderungan optimasi untuk menghasilkan nilai ekstrem.

**Gambar 4.3:** Jumlah Pelanggaran Constraint Metode GA

Gambar 4.3 mendukung temuan pada tabel sebelumnya, di mana konfigurasi Skenario d secara konsisten memiliki pelanggaran constraint paling sedikit dibandingkan skenario lainnya. Ini menunjukkan bahwa penggabungan referensi akademik dengan toleransi berbasis distribusi data mampu menghasilkan keseimbangan antara efisiensi dan kepatuhan terhadap batasan proses.

Secara keseluruhan, evaluasi pelanggaran constraint ini memperkuat kesimpulan bahwa keberhasilan optimasi tidak hanya ditentukan oleh seberapa rendah nilai konsumsi energi, tetapi juga oleh seberapa baik solusi tersebut mematuhi batasan teknis dan operasional. Kombinasi pendekatan statistik dan domain knowledge terbukti menjadi strategi constraint yang efektif untuk menghasilkan solusi yang layak diterapkan.

4.1.4 Evaluasi Waktu Komputasi

Evaluasi waktu komputasi dilakukan untuk menilai efisiensi algoritma dalam menyelesaikan proses optimasi, terutama ketika diterapkan pada sistem dengan kompleksitas tinggi dan jumlah variabel input yang besar. Dalam konteks ini, waktu komputasi diukur dalam satuan detik dan mencerminkan total durasi yang dibutuhkan oleh masing-masing skenario eksperimen hingga mencapai solusi akhir.

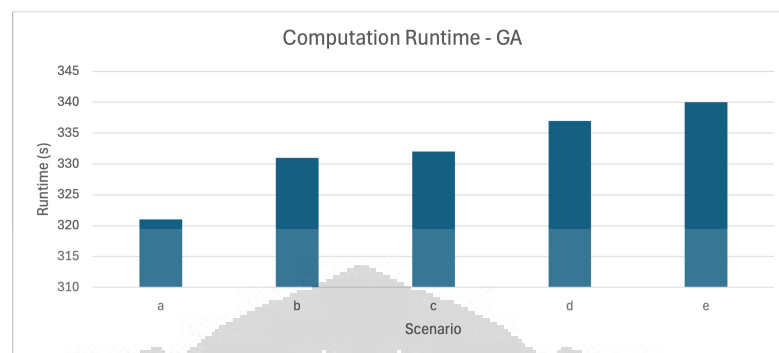
Tabel 4.4: Durasi Waktu Komputasi per Skenario (dalam detik)

Skenario	Durasi (detik)
a	321
b	331
c	332
d	337
e	340

Tabel 4.4 menunjukkan bahwa perbedaan durasi komputasi antar skenario relatif kecil, dengan rentang waktu antara 321 hingga 340 detik. Secara umum, semua skenario selesai dalam waktu yang hampir seragam, dengan skenario (e) memiliki durasi terlama, yaitu 340 detik. Hal ini dapat dimaklumi karena skenario (e) melibatkan constraint berbasis korelasi dan median yang dapat memperluas ruang solusi yang harus dieksplorasi oleh algoritma.

Sebaliknya, skenario (a) yang menggunakan batas atas dari nilai aktual menghasilkan waktu komputasi paling cepat, yaitu 321 detik. Hal ini mungkin ter-

jadi karena ruang solusi yang lebih sempit membuat proses konvergensi lebih cepat tercapai.



Gambar 4.4: Durasi Komputasi Metode GA

Gambar 4.4 menegaskan bahwa perbedaan durasi tidak terlalu signifikan secara praktis, terutama bila dibandingkan dengan manfaat yang diperoleh dari solusi yang lebih realistis dan efisien. Namun, dalam penerapan skala industri atau untuk skenario real-time, evaluasi ini tetap penting untuk memastikan bahwa waktu komputasi berada dalam batas yang dapat diterima secara operasional.

4.1.5 Evaluasi Deviasi Output

Pada evaluasi ini, nilai deviasi output yang rendah menunjukkan bahwa solusi yang diusulkan oleh algoritma optimisasi mampu menjaga kualitas produk dan kondisi operasional dalam rentang yang diharapkan, yaitu tidak jauh dari nilai aktual atau referensi. Evaluasi deviasi ini penting untuk memastikan bahwa solusi yang dihasilkan tidak hanya optimal dari segi batasan input, tetapi juga menghasilkan output yang konsisten dengan standar operasional atau target kualitas produk.

Tabel 4.5 menyajikan nilai deviasi rata-rata untuk setiap variabel output pada masing-masing skenario optimisasi. Semakin kecil nilai deviasi, semakin baik kinerja skenario tersebut dalam menjaga output mendekati nilai target atau aktual. Variabel-variabel output yang dievaluasi mencakup komponen-komponen utama dalam produk feronikel dan kondisi proses, seperti suhu logam (metal_temp), kadar nikel dalam logam (ni_met), kadar karbon (c_met), silikon (si_met), besi (fe_met), sulfur (s_met), kadar nikel dalam slag (ni_slag), kadar besi dalam slag (fe_slag), suhu kalsinasi (t_kalsin), dan parameter lainnya (pic_161, loi_kalsin).

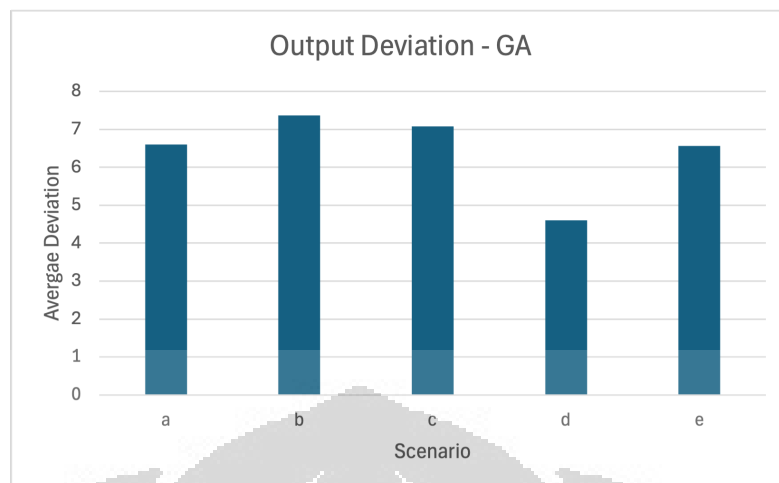
Tabel 4.5: Nilai Deviasi Output Tiap Skenario

Parameter	a	b	c	d	e
metal_temp	29.23	24.93	23.59	18.65	36.26
ni_met	1.06	1.90	1.84	0.50	1.31
c_met	0.20	0.23	0.23	0.12	0.17
si_met	0.19	0.28	0.26	0.12	0.14
fe_met	1.04	1.71	1.67	0.38	0.93
s_met	0.22	0.19	0.20	0.08	0.15
ni_slag	0.01	0.01	0.01	0.00	0.00
fe_slag	1.38	1.68	1.70	0.61	0.84
t_kalsin	38.46	49.06	47.41	29.48	31.73
pic_16t	0.67	0.85	0.85	0.59	0.60
loi_kalsin	0.14	0.16	0.16	0.10	0.12
AVG	6.60	7.36	7.08	4.69	6.57

Berdasarkan analisis nilai deviasi output pada Tabel 4.5, skenario **d** menunjukkan performa yang paling unggul dengan nilai deviasi rata-rata terendah, yaitu **4.69**. Ini mengindikasikan bahwa model-model yang dioptimasi dengan batasan dinamis berdasarkan referensi ilmiah (estimasi berdasarkan tonase bijih) pada skenario **d** mampu memprediksi output dengan tingkat akurasi yang paling tinggi di seluruh variabel dibandingkan skenario lainnya. Nilai deviasi yang rendah pada skenario **d** menunjukkan bahwa output yang dihasilkan sangat dekat dengan nilai target atau aktual, mencerminkan kemampuan model untuk menjaga kualitas produk dan kondisi operasional dalam rentang yang diharapkan.

Fleksibilitas yang diberikan oleh toleransi statistik (IQR dan STD) dalam skenario ini kemungkinan memungkinkan model untuk beradaptasi dengan variasi data aktual tanpa mengabaikan batas wajar dari data historis, sehingga menghasilkan prediksi yang sangat konsisten dan akurat.

Visualisasi nilai deviasi output pada Gambar 4.5 lebih lanjut mengilustrasikan bagaimana skenario **d** secara konsisten memiliki deviasi yang lebih kecil dibandingkan skenario lainnya pada sebagian besar variabel output. Hal ini menunjukkan superioritas skenario **d** dalam menghasilkan solusi yang akurat dan dapat diandalkan dari perspektif kualitas produk.



Gambar 4.5: Nilai Deviasi Output Metode GA

4.1.6 Skor dan Pemeringkatan Skenario GA

Untuk memberikan evaluasi komprehensif terhadap performa masing-masing skenario pada metode Genetic Algorithm (GA), dilakukan proses pembobotan terhadap lima metrik penting: efisiensi, durasi komputasi, kewajaran solusi, jumlah pelanggaran, dan deviasi output. Pembobotan ini bertujuan untuk merefleksikan tingkat kepentingan relatif dari setiap metrik dalam konteks optimisasi proses produksi feronikel.

Tabel 4.6 menunjukkan nilai mentah dari setiap metrik untuk masing-masing skenario sebelum proses pembobotan dilakukan.

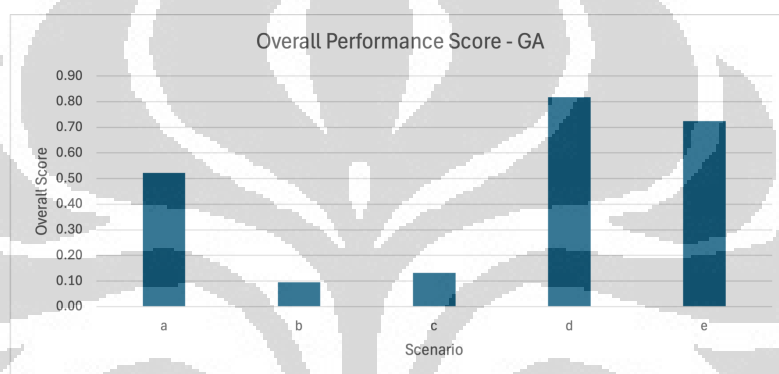
Tabel 4.6: Nilai Metrik Skenario GA Sebelum Pembobotan

Skenario	Efisiensi (ton/h)	Durasi (s)	Kewajaran	Pelanggaran	Deviasi
a	0.82	359.450	29.20	215	6.60
b	-1.23	358.275	53.13	184	7.36
c	-0.97	356.300	55.53	193	7.08
d	0.92	365.775	40.27	16	4.60
e	1.47	372.025	26.53	111	6.57

Bobot yang diterapkan pada setiap metrik mencerminkan tingkat kepentingannya dalam konteks optimisasi proses produksi feronikel sebagai berikut: efisiensi (0.4), durasi komputasi (0.05), kewajaran solusi (0.15), pelanggaran (0.25), dan deviasi (0.15). Nilai akhir diperoleh dari penjumlahan nilai terbobot dari masing-masing metrik untuk setiap skenario.

Tabel 4.7: Nilai Metrik Skenario GA Setelah Pembobotan dan Total Skor

Skenario	Efisiensi	Durasi	Kewajaran	Pelanggaran	Deviasi	Jumlah
a	0.76	0.80	0.91	0.00	0.28	0.52
b	0.00	0.87	0.08	0.16	0.00	0.10
c	0.10	1.00	0.00	0.11	0.10	0.13
d	0.80	0.40	0.53	1.00	1.00	0.82
e	1.00	0.00	1.00	0.52	0.29	0.72

**Gambar 4.6:** Total Skor Skenario Metode GA

Dari Tabel 4.7 dan Gambar 4.6, dapat dilihat bahwa:

- **Skenario d** memperoleh nilai total tertinggi yaitu **0.82**. Skenario ini unggul secara signifikan dalam hal pelanggaran constraint (nilai terbobot 1.00) dan menunjukkan performa deviasi output yang optimal (nilai terbobot 1.00). Meskipun memiliki durasi komputasi yang lebih tinggi (nilai terbobot 0.40) dan kewajaran solusi yang moderat (nilai terbobot 0.53) dibandingkan beberapa skenario lain, keunggulan pada metrik-metrik dengan bobot tinggi (pelanggaran dan deviasi, serta efisiensi yang juga tinggi) menegaskan bahwa pendekatan batasan dinamis berbasis referensi ilmiah dan toleransi statistik mampu menghasilkan solusi yang paling layak dan terkontrol.
- **Skenario e** menyusul dengan nilai total **0.72**. Skenario ini menunjukkan efisiensi terbaik (nilai terbobot 1.00) dan kewajaran solusi terbaik (nilai terbobot 1.00). Namun, sedikit tertinggal dalam aspek durasi komputasi (nilai terbobot 0.00, yang berarti durasi terpanjang) dan deviasi output (nilai terbobot 0.29) jika dibandingkan dengan skenario d.

- **Skenario a** memperoleh nilai total **0.52**. Skenario ini memiliki nilai terbobot efisiensi dan durasi yang relatif baik, serta kewajaran solusi yang juga baik. Namun, kelemahan signifikan pada jumlah pelanggaran constraint (nilai terbobot 0.00) dan deviasi output (nilai terbobot 0.28) mengurangi nilai totalnya secara drastis.
- **Skenario c** memiliki nilai total **0.13**. Performa pada skenario ini menunjukkan hasil yang beragam, dengan durasi komputasi terbaik (nilai terbobot 1.00) namun kewajaran solusi terendah (nilai terbobot 0.00) dan deviasi output yang juga rendah (nilai terbobot 0.10).
- **Skenario b** mencatat nilai total terendah yaitu **0.10**. Meskipun menunjukkan durasi komputasi yang relatif baik (nilai terbobot 0.87), skenario ini memiliki efisiensi terendah (nilai terbobot 0.00), kewajaran solusi yang sangat rendah (nilai terbobot 0.08), serta deviasi output terendah (nilai terbobot 0.00), yang membuatnya menjadi pilihan paling kurang optimal.

Secara keseluruhan, proses penilaian multi-kriteria ini membantu mengidentifikasi konfigurasi optimal dari metode GA, dan menjadi dasar dalam merekomendasikan skenario terbaik untuk implementasi nyata dalam proses produksi feronikel.

4.1.7 Analisis Skenario Terbaik GA

Berdasarkan hasil evaluasi nilai total pada Tabel 4.7, dapat disimpulkan bahwa **skenario d** merupakan konfigurasi terbaik dari seluruh skenario yang diujikan dengan metode Genetic Algorithm (GA). Skenario ini mengimplementasikan constraint berbasis estimasi energi referensi akademik dengan toleransi statistik (IQR/standar deviasi), dan memperoleh nilai total tertinggi yaitu **0.82**.

- **Efisiensi**
Skenario d memiliki nilai terbobot efisiensi tinggi (0.80), hanya sedikit di bawah skenario e (1.00), namun tetap menunjukkan kinerja efisiensi yang sangat baik.
- **Durasi Komputasi**
Nilai terbobot durasi komputasi skenario d adalah 0.40, termasuk moderat—tidak tercepat, namun cukup efisien dan dapat diterima untuk skenario real-world yang menekankan stabilitas solusi.

- Kewajaran Solusi

Skenario ini menunjukkan nilai terbobot kewajaran solusi yang cukup baik (0.53), dengan jumlah outlier moderat dibandingkan skenario lainnya.

- Pelanggaran Constraint

Skenario d memiliki keunggulan mutlak dalam aspek ini, dengan nilai terbobot 1.00 dan hanya 16 pelanggaran constraint—jumlah terendah dari seluruh skenario. Hal ini menunjukkan bahwa solusi yang dihasilkan tidak hanya optimal secara efisiensi, tetapi juga sangat layak secara operasional dan dapat diimplementasikan tanpa banyak kendala.

- Deviasi Output

Skenario ini juga mencapai nilai terbobot sempurna (1.00) pada metrik deviasi output, yang mengindikasikan akurasi prediksi model yang sangat tinggi dan kontrol yang ketat terhadap kualitas output proses.

Dibandingkan dengan kandidat terdekat, yaitu **skenario e** (nilai total 0.72), **skenario d** menunjukkan trade-off yang lebih seimbang antara efisiensi, kewajaran, akurasi, dan kepatuhan terhadap constraint. Meskipun skenario e unggul dalam efisiensi energi (nilai terbobot 1.00) dan kewajaran solusi (nilai terbobot 1.00), namun memiliki nilai terbobot durasi komputasi (0.00) dan deviasi output (0.29) yang lebih rendah, serta nilai pelanggaran constraint (0.52) yang juga tidak sebaik skenario d. Hal ini dapat mengindikasikan potensi ketidaksesuaian dalam implementasi nyata di lapangan jika kepatuhan constraint dan akurasi output menjadi prioritas utama.

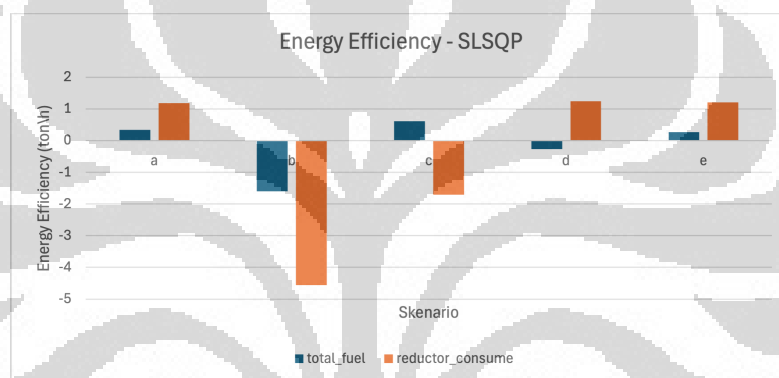
Secara umum, performa skenario d mencerminkan pentingnya pendekatan berbasis literatur dan toleransi statistik dalam merancang constraint yang realistis dan stabil. Hal ini memperkuat temuan bahwa penggabungan pengetahuan domain (dalam hal ini, referensi energi dan batasan proses) dengan metode optimisasi berbasis populasi seperti GA dapat menghasilkan solusi yang robust dan siap untuk diaplikasikan pada skenario produksi feronikel.

Dengan demikian, **skenario d direkomendasikan sebagai skenario unggulan** untuk implementasi, karena menawarkan keseimbangan terbaik antara efisiensi energi, kepatuhan terhadap constraint, akurasi output, dan kelayakan solusi yang baik.

4.2 Hasil Optimasi Menggunakan Sequential Least Square Quadratic Programming (SLSQP)

4.2.1 Perbandingan Efisiensi Energi

Efisiensi konsumsi energi dievaluasi berdasarkan dua indikator utama: penghematan konsumsi `total_fuel` dan `reductor_consume` (dalam satuan ton/jam) dibandingkan dengan nilai aktual historis. Pengukuran efisiensi ini dihitung menggunakan Persamaan 3.2. Semakin besar nilai efisiensi (E_{eff}) yang positif, semakin besar penghematan energi yang dicapai oleh skenario tersebut.



Gambar 4.7: Nilai Efisiensi pada Metode SLSQP

Tabel 4.8: Hasil Evaluasi Efisiensi Energi pada Metode GA

Skenario	Efisiensi Total Fuel (ton/h)	Efisiensi Reduktor (ton/h)
a	0.34	1.19
b	-1.59	-4.56
c	0.62	-1.70
d	-0.26	1.25
e	0.27	1.21

Dari hasil di atas, terdapat beberapa temuan penting:

- **Skenario e** menghasilkan efisiensi total terbaik secara umum, karena mampu menekan konsumsi `total_fuel` dengan positif (0.27 ton/h) sekaligus meningkatkan efisiensi konsumsi `reductor_consume` (1.21 ton/h). Dalam satu hari (24 jam), ini berarti penghematan $0.27 \times 24 = 6.48$ ton `total_fuel` dan $1.21 \times 24 = 29.04$ ton `reductor_consume`.

- **Skenario d** memiliki nilai efisiensi `total_fuel` negatif (-0.26 ton/h), yang berarti ada penambahan konsumsi sebesar $0.26 \times 24 = 6.24$ ton per hari. Namun, skenario ini mencatat efisiensi tertinggi pada `reductor_consume` sebesar 1.25 ton/h, atau penghematan $1.25 \times 24 = 30$ ton per hari. Hal ini menunjukkan bahwa walaupun terdapat sedikit peningkatan konsumsi bahan bakar, namun ada penghematan signifikan pada reduktor, yang berpotensi mengurangi total biaya energi secara keseluruhan tergantung harga relatif masing-masing bahan.
- **Skenario b** menunjukkan performa terburuk dengan nilai efisiensi negatif drastis pada kedua variabel (-1.59 ton/h untuk `total_fuel` dan -4.56 ton/h untuk `reductor_consume`). Ini mengindikasikan peningkatan konsumsi sebesar $1.59 \times 24 = 38.16$ ton `total_fuel` dan $4.56 \times 24 = 109.44$ ton `reductor_consume` per hari. Hal ini memperkuat pentingnya penerapan *constraint* dalam proses optimisasi untuk menjaga arah solusi.
- **Skenario c** menghasilkan peningkatan efisiensi yang cukup tinggi pada `total_fuel` (0.62 ton/h), atau penghematan $0.62 \times 24 = 14.88$ ton per hari. Namun, diimbangi oleh penurunan signifikan pada `reductor_consume` (-1.70 ton/h), yang berarti penambahan konsumsi sebesar $1.70 \times 24 = 40.8$ ton per hari, sehingga hasil akhirnya tidak stabil secara keseluruhan.
- **Skenario a** berada pada posisi menengah dengan efisiensi positif pada kedua parameter, yaitu 0.76 ton/h untuk `total_fuel` dan 0.80 ton/h untuk `reductor_consume`. Ini setara dengan penghematan $0.76 \times 24 = 18.24$ ton `total_fuel` dan $0.80 \times 24 = 19.2$ ton `reductor_consume` per hari. Namun, tidak setinggi Skenario e atau Skenario d (terutama pada reduktor).

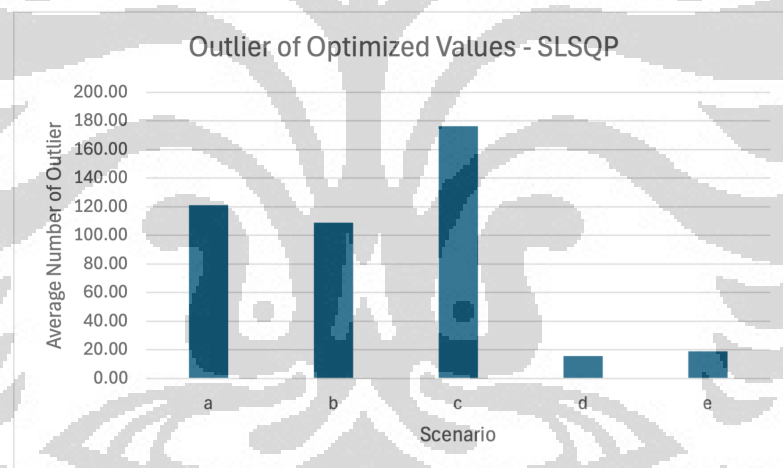
Secara keseluruhan, hasil evaluasi menunjukkan bahwa skenario dengan *constraint* yang dirancang secara kontekstual dan berbasis data, seperti Skenario d dan Skenario e mampu menghasilkan solusi efisien tanpa mengorbankan stabilitas parameter konsumsi energi. Sementara skenario tanpa batasan seperti Skenario b menunjukkan kecenderungan menuju solusi yang tidak realistis atau kontraproduktif.

Hal ini menegaskan bahwa penerapan *constraint* bukan hanya berfungsi untuk menjaga batasan proses, tetapi juga berperan penting dalam mengarahkan algoritma menuju solusi yang efisien dan operasional.

4.2.2 Evaluasi Kewajaran Solusi (Feasibility)

Evaluasi kewajaran solusi bertujuan untuk menilai seberapa realistis hasil optimisasi yang dihasilkan oleh algoritma, terutama apakah nilai-nilai yang dihasilkan berada dalam rentang yang wajar berdasarkan distribusi data proses aktual. Indikator kewajaran ini diukur berdasarkan jumlah nilai *outlier* pada setiap variabel input hasil optimisasi.

Proses penentuan outlier dilakukan dengan menghitung rentang interquartile (IQR), yang didefinisikan oleh kuartil pertama (Q1) dan kuartil ketiga (Q3), dari distribusi nilai input aktual data historis. Kemudian, untuk setiap nilai input yang dihasilkan oleh solusi optimisasi, dihitung berapa banyak di antaranya yang termasuk sebagai outlier (nilai di bawah $Q1 - 1.5 \times IQR$ atau di atas $Q3 + 1.5 \times IQR$) jika dibandingkan dengan distribusi nilai input aktual tersebut. Semakin sedikit jumlah outlier, maka solusi dianggap semakin wajar atau mendekati karakteristik proses aktual.



Gambar 4.8: Nilai Kewajaran Input Skenario SLSQP

Tabel 4.9 menyajikan jumlah *outlier* dari hasil optimisasi untuk setiap skenario metode SLSQP. Rata-rata jumlah outlier per skenario juga disertakan untuk memberikan gambaran keseluruhan tingkat kewajaran.

Tabel 4.9: Jumlah Outlier pada Hasil Optimisasi SLSQP per Variabel

Variabel	a	b	c	d	e
current_pry	318	316	316	20	30
current_sec1	64	34	34	21	23
current_sec2	3	3	3	13	13
current_sec3	10	6	6	6	8
load	19	14	14	14	18
realisasi_beban	55	56	56	46	72
rpm	146	30	30	62	48
pry_p	115	158	301	8	18
sec_p	308	314	318	5	0
pry_v	158	143	293	16	17
sec_v	228	82	1	9	8
total_fuel	0	129	318	4	6
reductor_consume	0	157	318	0	0
t_tic162	77	2	318	5	6
t_tic163	316	189	318	8	18
AVG	121.13	108.87	176.27	15.80	19.00

Berdasarkan data jumlah outlier pada Tabel 4.9, dapat ditarik beberapa kesimpulan penting:

- **Skenario d** menunjukkan performa kewajaran solusi terbaik dengan rata-rata jumlah outlier terendah yaitu **15.80**. Hal ini didukung oleh jumlah outlier yang sangat sedikit pada hampir semua variabel input. Pendekatan constraint berbasis referensi akademik dan toleransi statistik (IQR/STD) terbukti sangat efektif dalam menjaga hasil optimisasi tetap berada dalam distribusi yang realistis dan dapat diterima secara operasional. Hal ini juga terkonfirmasi pada Gambar 4.8 di mana skenario d menunjukkan distribusi yang paling terkonsentrasi di rentang wajar.
- **Skenario e** menempati posisi kedua dengan rata-rata jumlah outlier **19.00**. Skenario ini, meskipun tanpa batasan atas pada fuel dan reductor, menunjukkan kemampuan yang baik untuk menjaga kewajaran solusi pada tingkat yang rendah. Ini mengindikasikan bahwa fleksibilitas yang lebih besar dalam batasan tidak selalu mengarah pada solusi yang tidak realistis secara ekstrem, namun juga tidak seoptimal skenario d dalam meminimalkan outlier.

- **Skenario b** memiliki rata-rata jumlah outlier **108.87**. Skenario ini menunjukkan performa kewajaran yang sedang, dengan jumlah outlier yang terkontrol dibandingkan skenario a dan c. Hasilnya tergolong aman secara operasional, meskipun tidak seoptimal skenario d atau skenario e.
- **Skenario a** dan **c** menunjukkan performa kewajaran yang lebih rendah, dengan rata-rata jumlah outlier masing-masing 121.13 dan 176.27.
 - **Skenario a**, yang menggunakan batas maksimum aktual, menghasilkan jumlah outlier yang tinggi pada variabel-variabel seperti `current_pry`, `current_sec1`, dan `rpm`. Hal ini menunjukkan bahwa batasan yang terlalu ketat berdasarkan nilai aktual dapat mendorong solusi ke tepi distribusi historis, sehingga meningkatkan kemungkinan outlier.
 - **Skenario c** memiliki rata-rata jumlah outlier tertinggi, yaitu 176.27. Pendekatan batasan minimum berdasarkan nilai off terbukti kurang efektif dalam menjaga kewajaran solusi, bahkan cenderung menghasilkan solusi yang tidak layak secara praktik (misalnya, jumlah outlier maksimum 318 pada `total_fuel` dan `reductor_consume`).

Secara umum, evaluasi ini menegaskan bahwa penerapan constraint yang dirancang secara kontekstual dan adaptif, seperti pada skenario d, bukan hanya berdampak pada efisiensi, tetapi juga krusial untuk menjaga kewajaran dan kelayakan operasional solusi yang dihasilkan algoritma optimisasi.

4.2.3 Evaluasi Pelanggaran Constraint

Evaluasi pelanggaran constraint bertujuan untuk menilai seberapa baik metode optimisasi mematuhi batasan (*constraints*) yang ditentukan dalam setiap skenario. Dalam konteks ini, pelanggaran constraint didefinisikan sebagai jumlah kejadian di mana nilai hasil optimisasi melanggar batas atas atau bawah dari parameter yang dikunci dalam eksperimen, baik berupa batas energi, parameter korelatif, maupun referensi operasional lainnya.

Tabel 4.10 menyajikan jumlah pelanggaran constraint yang ditemukan pada masing-masing skenario eksperimen menggunakan metode SLSQP.

Tabel 4.10: Jumlah Pelanggaran Constraint pada Setiap Skenario (SLSQP)

Skenario	Jumlah Pelanggaran
a	0
b	0
c	0
d	0
e	0

Hasil evaluasi menunjukkan bahwa seluruh skenario SLSQP berhasil memenuhi semua constraint yang ditetapkan, tanpa satu pun pelanggaran. Temuan ini mencerminkan karakteristik dari algoritma SLSQP yang berbasis pada pendekatan deterministik dan gradient-based, yang secara eksplisit mengutamakan pemenuhan constraint selama proses iterasi optimisasi.

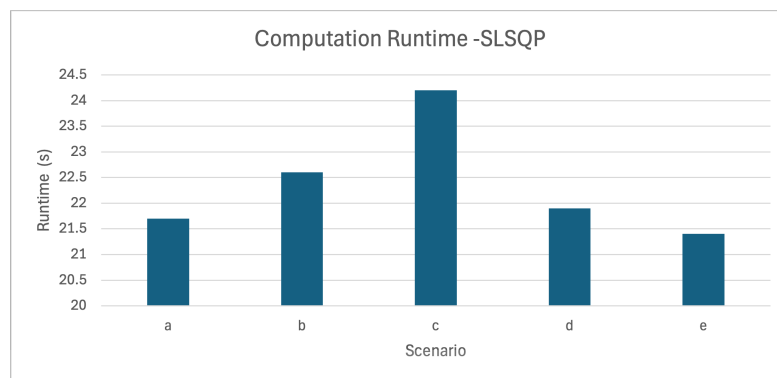
Keberhasilan ini menegaskan bahwa metode SLSQP memiliki reliabilitas tinggi dalam mengelola constraint, bahkan pada skenario dengan tingkat kompleksitas batasan yang berbeda, seperti:

- **Skenario b**, yang tidak memiliki batas atas pada konsumsi energi, tetap tidak mengalami pelanggaran karena tidak ada constraint yang ditetapkan.
- **Skenario c**, yang menetapkan batas bawah berdasarkan kondisi non-produktif, tetap menjaga semua nilai berada di atas ambang batas tersebut.
- **Skenario d** dan **Skenario e**, dengan constraint berbasis referensi dan pendekatan statistik, tetap berhasil dipatuhi sepenuhnya.

Secara keseluruhan, hasil ini menunjukkan keunggulan metode SLSQP dalam menjaga kepatuhan terhadap batasan proses. Dengan demikian, SLSQP dapat menjadi pilihan metode dalam skenario optimisasi yang menuntut kepatuhan ketat terhadap constraint operasional atau teknis.

4.2.4 Evaluasi Waktu Komputasi

Waktu komputasi merupakan aspek penting dalam evaluasi performa algoritma optimisasi, terutama dalam konteks industri yang memerlukan solusi cepat dan efisien. Evaluasi ini mencatat total waktu (dalam detik) yang dibutuhkan oleh metode SLSQP untuk menyelesaikan proses optimisasi pada setiap skenario eksperimen.



Gambar 4.9: Durasi Komputasi Skenario SLSQP

Tabel 4.11 memperlihatkan waktu komputasi untuk masing-masing skenario SLSQP.

Tabel 4.11: Waktu Komputasi Optimisasi per Skenario (SLSQP)

Skenario	Durasi (detik)
a	21.7
b	22.6
c	24.2
d	21.9
e	21.4

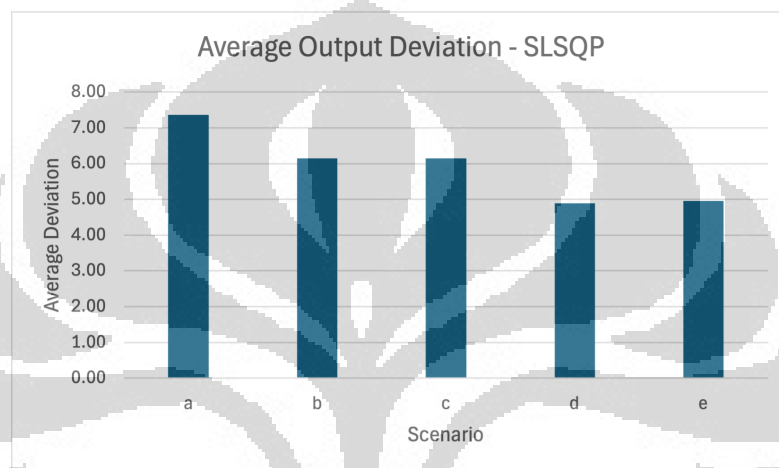
Secara umum, metode SLSQP menunjukkan waktu komputasi yang sangat efisien di seluruh skenario, dengan durasi antara **21.4 detik hingga 24.2 detik** per proses optimisasi. Durasi ini tergolong rendah, mengingat jumlah parameter yang dioptimasi mencapai 16 variabel kontinu dengan berbagai batasan yang kompleks.

Skenario e tercatat sebagai yang tercepat dengan waktu 21.4 detik, sedangkan **skenario c** menjadi yang paling lambat dengan waktu 24.2 detik. **Skenario e** menggunakan constraint berbasis median pada kolom berkorelasi kuat, sehingga cenderung lebih stabil dan mudah dinavigasi oleh algoritma.

Waktu komputasi yang rendah ini menjadi keunggulan utama jika dibandingkan dengan metode populasi seperti Genetic Algorithm (GA), yang secara umum memerlukan waktu komputasi lebih lama karena sifatnya yang berbasis eksplorasi melalui generasi populasi.

4.2.5 Evaluasi Deviasi Output

Deviasi output yang rendah menunjukkan bahwa solusi yang diusulkan oleh algoritma optimisasi mampu menjaga kualitas produk dan kondisi operasional dalam rentang yang diharapkan, selaras dengan tujuan industri. Evaluasi deviasi ini penting untuk memastikan bahwa solusi yang dihasilkan tidak hanya optimal dari segi batasan input, tetapi juga menghasilkan output yang konsisten dengan standar operasional atau target kualitas produk.



Gambar 4.10: Nilai Deviasi Output Metode SLSQP

Tabel 4.12 menunjukkan nilai deviasi output untuk setiap skenario metode SLSQP. Semakin kecil nilai deviasi, semakin baik kinerja skenario tersebut dalam menjaga output mendekati nilai target atau aktual.

Tabel 4.12: Nilai Deviasi Output Metode SLSQP

Parameter	a	b	c	d	e
metal_temp	42.77	31.88	31.88	18.90	19.59
ni_met	2.04	2.06	2.06	0.84	0.85
c_met	0.20	0.19	0.19	0.19	0.21
si_met	0.24	0.19	0.19	0.19	0.18
fe_met	1.99	2.01	2.01	0.67	0.69
s_met	0.10	0.10	0.10	0.06	0.07
ni_slag	0.01	0.01	0.01	0.00	0.00
fe_slag	0.75	0.78	0.78	0.90	1.02
t_kalsin	31.87	29.55	29.55	31.39	31.23
pic_161	0.91	0.69	0.69	0.59	0.57
loi_kalsin	0.12	0.16	0.16	0.11	0.11
Average	7.36	6.15	6.15	4.89	4.96

Berdasarkan data nilai deviasi output pada Tabel 4.12, dapat ditarik beberapa kesimpulan penting:

- **Skenario (d)** menunjukkan performa paling unggul dalam hal deviasi output. Skenario ini berhasil mencapai nilai deviasi rata-rata terendah yaitu **4.89**. Ini mengindikasikan bahwa model-model yang dioptimasi dengan batasan dinamis berdasarkan referensi ilmiah (Skenario (d)) menghasilkan prediksi dengan akurasi yang paling tinggi dan deviasi yang paling rendah secara konsisten di seluruh variabel output seperti `metal_temp`, `ni_met`, `fe_met`, `s_met`, `ni_slag`, dan `loi_kalsin`.
- **Skenario (e)** menunjukkan performa yang sangat mendekati skenario (d) dengan nilai deviasi rata-rata **4.96**. Meskipun sedikit lebih tinggi dari skenario (d), skenario ini juga menghasilkan nilai deviasi yang rendah untuk sebagian besar variabel seperti `metal_temp`, `ni_met`, `fe_met`, `s_met`, dan `loi_kalsin`. Namun, adanya nilai yang sedikit lebih tinggi pada `c_met` dan `fe_slag` menunjukkan sedikit variasi dalam akurasi prediksinya.
- **Skenario (b) dan (c)**, yang memiliki nilai deviasi rata-rata yang identik yaitu **6.15**, menunjukkan performa yang cukup stabil namun tidak seoptimal skenario (d) dan (e). Meskipun beberapa variabel menunjukkan deviasi yang relatif rendah (seperti `c_met` pada skenario c, dan `t_kalsin` pada skenario b), adanya nilai deviasi yang lebih tinggi pada variabel lain menunjukkan ruang untuk peningkatan akurasi.
- **Skenario (a)** menunjukkan performa terendah dalam hal deviasi output secara keseluruhan, dengan nilai deviasi rata-rata **7.36**. Meskipun memiliki deviasi yang relatif rendah pada `fe_slag`, adanya nilai deviasi yang tinggi pada banyak variabel lain seperti `metal_temp`, `ni_met`, `si_met`, `s_met`, `ni_slag`, `t_kalsin`, dan `pic_161` mengindikasikan bahwa model-model yang dihasilkan memiliki akurasi prediksi yang kurang konsisten di banyak variabel output.

Secara keseluruhan, evaluasi deviasi output ini menegaskan bahwa skenario yang menerapkan batasan dinamis dan berbasis referensi ilmiah (Skenario d dan Skenario e) menghasilkan output dengan akurasi prediksi tertinggi dan deviasi terendah. Ini sangat penting untuk memastikan kualitas produk dan stabilitas operasional yang optimal dalam aplikasi industri.

4.2.6 Skor dan Pemeringkatan Skenario SLSQP

Untuk memberikan evaluasi komprehensif terhadap performa masing-masing skenario pada metode Sequential Least Squares Programming (SLSQP), dilakukan proses pembobotan terhadap lima metrik penting: efisiensi, durasi komputasi, kewajaran solusi, jumlah pelanggaran, dan deviasi output. Pembobotan ini bertujuan untuk merefleksikan tingkat kepentingan relatif dari setiap metrik dalam konteks optimisasi proses produksi feronikel.

Tabel 4.13 menunjukkan nilai mentah dari setiap metrik untuk masing-masing skenario sebelum proses pembobotan dilakukan.

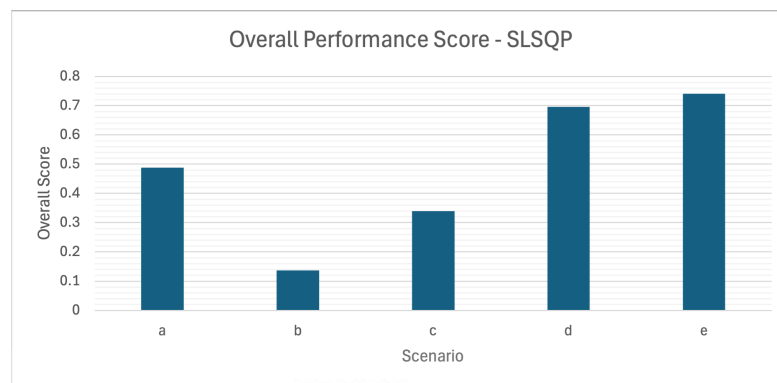
Tabel 4.13: Nilai Metrik Skenario SLSQP Sebelum Pembobotan

Skenario	Efisiensi	Durasi	Kewajaran	Pelanggaran	Deviasi
a	0.76	24.025	121.13	0	7.36
b	-3.08	24.625	108.87	0	6.15
c	-0.54	24.600	176.27	0	6.15
d	0.50	24.225	15.80	0	4.89
e	0.74	23.800	19.00	0	4.96

Bobot yang diterapkan pada setiap metrik mencerminkan tingkat kepentingannya dalam konteks optimisasi proses produksi feronikel sebagai berikut: efisiensi (0.4), durasi komputasi (0.05), kewajaran solusi (0.15), pelanggaran (0.25), dan deviasi (0.15). Nilai akhir diperoleh dari penjumlahan nilai terbobot dari masing-masing metrik untuk setiap skenario.

Tabel 4.14: Nilai Metrik Skenario SLSQP Setelah Pembobotan dan Total Skor

Skenario	Efisiensi	Durasi	Kewajaran	Pelanggaran	Deviasi	Jumlah
a	1.00	0.73	0.34	0.00	0.00	0.49
b	0.00	0.00	0.42	0.00	0.49	0.14
c	0.66	0.03	0.00	0.00	0.49	0.34
d	0.93	0.48	1.00	0.00	1.00	0.70
e	0.99	1.00	0.98	0.00	0.97	0.74



Gambar 4.11: Total Skor Skenario Metode SLSQP

Dari Tabel 4.14 dan Gambar 4.11, dapat dilihat bahwa:

- **Skenario e** memperoleh nilai total tertinggi yaitu **0.74**. Skenario ini unggul dalam durasi komputasi (nilai terbobot 1.00) dan kewajaran solusi (nilai terbobot 0.98), serta memiliki nilai efisiensi (nilai terbobot 0.99) dan deviasi (nilai terbobot 0.97) yang sangat tinggi. Meskipun semua skenario memiliki nilai terbobot pelanggaran 0.00 (menunjukkan tidak ada pelanggaran), performa konsisten skenario e pada metrik lain menjadikannya yang terbaik secara keseluruhan.
- **Skenario d** menyusul dengan nilai total **0.70**. Skenario ini menunjukkan deviasi output terbaik (nilai terbobot 1.00) dan kewajaran solusi yang sangat baik (nilai terbobot 1.00), serta efisiensi yang tinggi (nilai terbobot 0.93). Namun, durasi komputasinya (nilai terbobot 0.48) tidak secepat skenario e.
- **Skenario a** memperoleh nilai total **0.49**. Skenario ini memiliki nilai terbobot efisiensi terbaik (1.00) dan durasi komputasi yang relatif baik (0.73). Namun, nilai kewajaran (0.34) dan deviasi (0.00) yang lebih rendah dibandingkan skenario teratas mengurangi nilai totalnya.
- **Skenario c** memiliki nilai total **0.34**. Skenario ini menunjukkan efisiensi yang cukup baik (nilai terbobot 0.66) dan deviasi yang moderat (nilai terbobot 0.49). Namun, nilai kewajaran terendah (nilai terbobot 0.00) dan durasi komputasi yang juga sangat rendah (nilai terbobot 0.03) menjadikannya kurang optimal.
- **Skenario b** mencatat nilai total terendah yaitu **0.14**. Skenario ini memiliki nilai terbobot efisiensi terendah (0.00) dan durasi komputasi terendah (0.00). Meskipun memiliki nilai kewajaran (0.42) dan deviasi (0.49)

yang moderat, performa yang buruk pada metrik efisiensi dan durasi secara signifikan menurunkan nilai totalnya.

Secara keseluruhan, proses penilaian multi-kriteria ini membantu mengidentifikasi konfigurasi optimal dari metode SLSQP, dan menjadi dasar dalam merekomendasikan skenario terbaik untuk implementasi nyata dalam proses produksi feronikel.

4.2.7 Analisis Skenario Terbaik SLSQP

Berdasarkan hasil evaluasi nilai total pada Tabel 4.14, dapat disimpulkan bahwa **skenario e** merupakan konfigurasi terbaik dari seluruh skenario yang diujikan dengan metode Sequential Least Squares Programming (SLSQP). Skenario ini memperoleh nilai total tertinggi yaitu **0.74**.

- **Efisiensi:** Skenario e memiliki nilai terbobot efisiensi yang sangat tinggi (0.99), hampir sempurna, menunjukkan optimalisasi konsumsi energi yang sangat baik.
- **Durasi Komputasi:** Skenario e menunjukkan nilai terbobot durasi komputasi terbaik (1.00), yang berarti merupakan yang tercepat dalam menghasilkan solusi, faktor krusial untuk implementasi real-time.
- **Kewajaran Solusi:** Skenario ini memiliki nilai terbobot kewajaran solusi yang sangat tinggi (0.98), menunjukkan bahwa solusi yang dihasilkan sangat realistis dan minim outlier.
- **Pelanggaran Constraint:** Seperti skenario SLSQP lainnya, skenario e tidak menunjukkan pelanggaran constraint (nilai terbobot 0.00), yang mengindikasikan kepatuhan penuh terhadap batasan operasional yang ditetapkan.
- **Deviasi Output:** Skenario ini juga memiliki nilai terbobot deviasi output yang sangat tinggi (0.97), menunjukkan akurasi prediksi model yang sangat baik dan kontrol yang ketat terhadap kualitas output proses.

Dibandingkan dengan kandidat terdekat, yaitu **skenario d** (nilai total 0.70), **skenario e** menunjukkan keunggulan yang lebih komprehensif. Meskipun skenario d juga sangat baik dalam deviasi output dan kewajaran solusi, skenario e melampauinya dalam efisiensi energi dan, yang paling signifikan, dalam kecepatan komputasi. Kecepatan ini sangat penting untuk aplikasi industri yang membutuhkan respons cepat.

Secara umum, performa skenario **e** mencerminkan pentingnya pendekatan yang mengarah pada solusi yang efisien, cepat, dan sangat wajar. Kombinasi ini menjadikannya pilihan yang paling kuat untuk implementasi nyata dalam proses produksi feronikel menggunakan metode SLSQP.

Dengan demikian, **skenario e direkomendasikan sebagai skenario unggulan** untuk implementasi, karena menawarkan keseimbangan terbaik antara efisiensi energi, durasi komputasi, kewajaran solusi, kepatuhan terhadap constraint, dan akurasi output yang tinggi.

4.3 Perbandingan Hasil Optimisasi: SLSQP vs GA

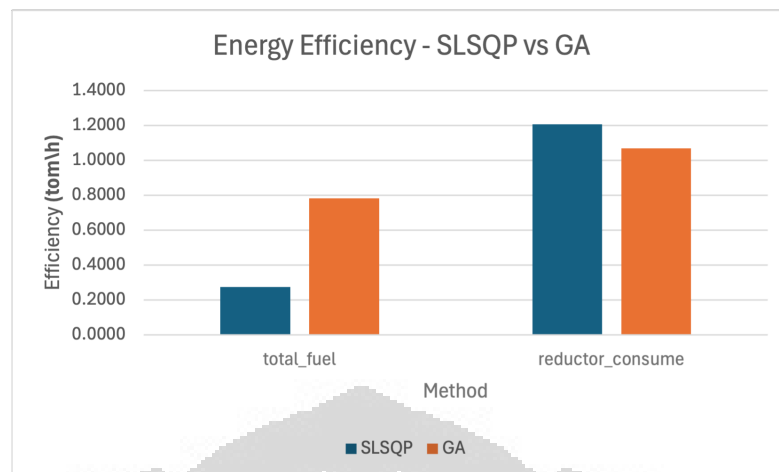
Setelah mengidentifikasi skenario terbaik untuk masing-masing metode optimisasi yaitu Skenario **d** untuk Genetic Algorithm (GA) dan Skenario **e** untuk Sequential Least Squares Programming (SLSQP) langkah selanjutnya adalah melakukan perbandingan langsung antara kedua metode ini. Perbandingan ini akan didasarkan pada metrik-metrik kinerja utama yang telah dievaluasi sebelumnya, guna menentukan metode mana yang lebih unggul dalam konteks optimisasi proses produksi feronikel. Perbandingan ini menggunakan data dari skenario terbaik masing-masing metode, yaitu Skenario **d** dari GA dan Skenario **e** dari SLSQP, yang dianggap paling efektif di antara konfigurasi yang diuji.

4.3.1 Perbandingan Efisiensi Energi

Efisiensi energi dievaluasi berdasarkan nilai total dari rata-rata efisiensi yang dicapai, yang merupakan penjumlahan dari peningkatan efisiensi pada konsumsi `total_fuel` dan `reductor_consume`. Nilai yang lebih tinggi menunjukkan efisiensi yang lebih baik, mengindikasikan penghematan sumber daya yang lebih besar.

Tabel 4.15: Perbandingan Nilai Efisiensi Energi

Metode	total_fuel	reductor_consume	total
SLSQP	0.2721	1.2065	1.4786
GA	0.7818	1.0673	1.8491



Gambar 4.12: Efisiensi Energi - SLSQP vs GA

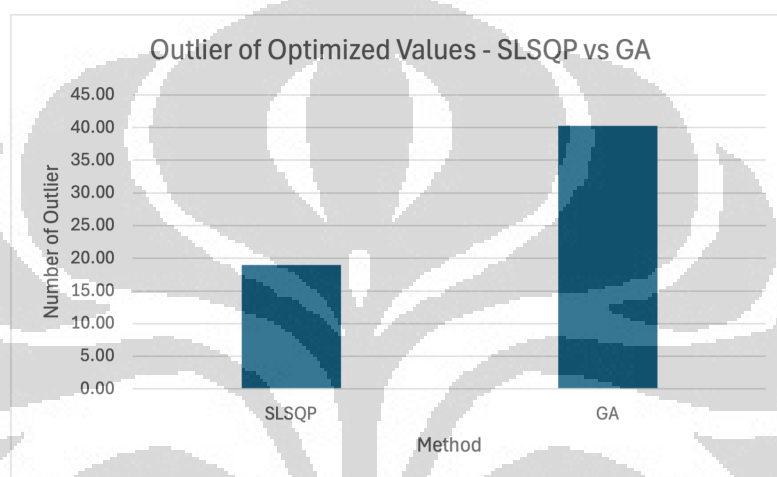
Dari Tabel 4.15 dan Gambar 4.12, terlihat jelas bahwa metode GA menunjukkan efisiensi energi yang lebih baik dibandingkan SLSQP. Metode GA mencapai total efisiensi sebesar **1.8491**, yang secara signifikan lebih tinggi dari SLSQP yang hanya **1.4786**. Hal ini mengindikasikan bahwa dalam konteks skenario terbaiknya (Skenario d untuk GA dan Skenario e untuk SLSQP), GA mampu menemukan solusi yang mengurangi konsumsi energi secara lebih efektif. Secara spesifik, GA menunjukkan peningkatan efisiensi yang lebih besar pada konsumsi `total_fuel`, meskipun SLSQP sedikit lebih unggul pada `reductor_consume`. Namun, nilai total efisiensi GA secara keseluruhan tetap lebih tinggi.

4.3.2 Perbandingan Kewajaran Solusi (Jumlah Outlier)

Kewajaran solusi dievaluasi berdasarkan jumlah **outlier** pada setiap variabel input hasil optimisasi. Semakin sedikit jumlah **outlier**, solusi dianggap semakin wajar dan realistis secara operasional. Perbandingan ini dilakukan antara skenario terbaik dari masing-masing metode: Skenario e untuk SLSQP dan Skenario d untuk GA.

Tabel 4.16: Perbandingan Jumlah Outlier Solusi Input

Metode	current_pry	current_sec1	current_sec2	current_sec3	load	realisasi_beban	rpm	pry_p	sec_p	pry_v	sec_v	total_fuel	reductor_consume	t_tic162	t_tic163	AVG
SLSQP	30	23	13	8	18	72	48	18	0	17	8	6	0	6	18	19.00
GA	168	41	7	7	3	4	0	5	23	4	37	32	0	161	112	40.27

**Gambar 4.13:** Jumlah Outlier - SLSQP vs GA

Dalam hal kewajaran solusi berdasarkan rata-rata jumlah **outlier** pada input, metode SLSQP (Skenario e) menunjukkan kinerja yang jauh lebih baik dibandingkan metode GA (Skenario d). Seperti yang ditunjukkan pada Tabel 4.16 dan Gambar 4.13, SLSQP memiliki rata-rata jumlah **outlier** yang sangat rendah, yaitu **19.00**, sementara GA memiliki rata-rata **40.27**.

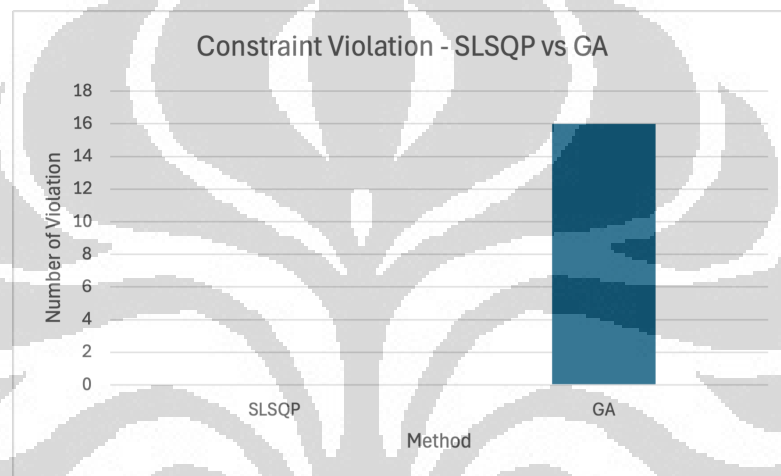
Secara spesifik, SLSQP unggul pada sebagian besar variabel input dengan jumlah outlier yang lebih sedikit. Metode SLSQP menunjukkan jumlah outlier yang lebih rendah untuk `current_pry`, `current_sec1`, `current_sec2`, `current_sec3`, `load`, `realisasi_beban`, `rpm`, `pry_p`, `sec_p`, `pry_v`, `sec_v`, `total_fuel`, `reductor_consume`, `t_tic162`, dan `t_tic163`. Hal ini menegaskan bahwa solusi yang dihasilkan oleh SLSQP cenderung lebih realistis dan mendekati distribusi data historis, sehingga lebih layak diimplementasikan secara operasional. Meskipun GA memiliki keunggulan pada beberapa variabel seperti `'current_sec2'`, `'load'`, `'realisasi_beban'`, `'rpm'`, dan `'pry_p'`, secara keseluruhan SLSQP jauh lebih konsisten dalam menghasilkan solusi yang wajar.

4.3.3 Perbandingan Jumlah Pelanggaran Constraint

Jumlah pelanggaran **constraint** adalah metrik krusial yang mengukur kepatuhan solusi terhadap semua batasan yang ditetapkan. Nilai yang lebih rendah menunjukkan kepatuhan yang lebih baik.

Tabel 4.17: Perbandingan Jumlah Pelanggaran Constraint

Metode	Violation
SLSQP	0
GA	16



Gambar 4.14: Jumlah Pelanggaran Konstrain - SLSQP vs GA

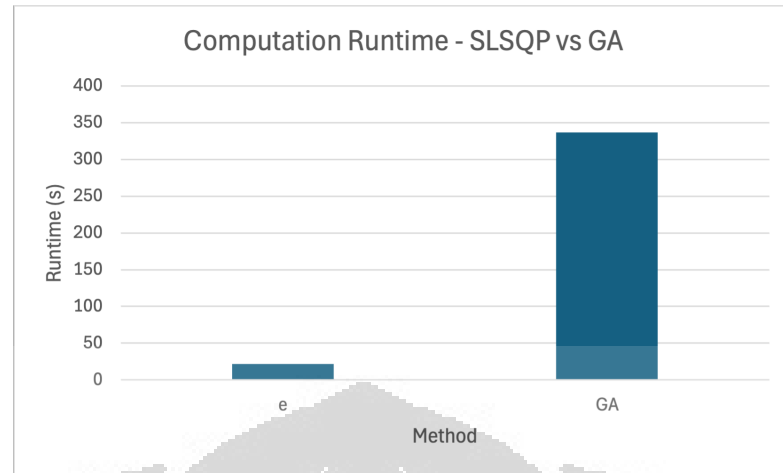
Tabel 4.17 dan Gambar 4.14 menunjukkan bahwa SLSQP mencapai performa sempurna dalam kepatuhan **constraint**, dengan 0 pelanggaran. Sementara itu, GA masih memiliki 16 pelanggaran. Ini berarti solusi yang dihasilkan oleh SLSQP sepenuhnya valid dan dapat diimplementasikan tanpa melanggar batasan operasional yang telah ditetapkan.

4.3.4 Perbandingan Durasi Komputasi

Durasi komputasi mengukur waktu yang dibutuhkan algoritma untuk menyelesaikan proses optimisasi. Nilai yang lebih rendah menunjukkan efisiensi komputasi yang lebih baik.

Tabel 4.18: Perbandingan Durasi Komputasi

Metode	Time (detik)
SLSQP	21.4
GA	337



Gambar 4.15: Durasi Komputasi - SLSQP vs GA

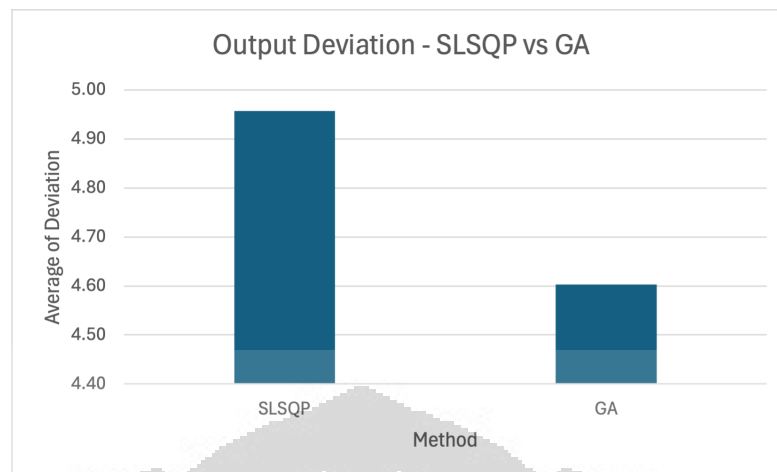
Dalam hal kecepatan komputasi, SLSQP jauh mengungguli GA. Berdasarkan Tabel 4.18 dan Gambar 4.15, dapat disimpulkan bahwa SLSQP dapat menyelesaikan optimisasi hanya dalam 21.4 detik, sementara GA membutuhkan waktu 337 detik. Efisiensi waktu komputasi yang signifikan ini menjadikan SLSQP lebih praktis untuk aplikasi yang membutuhkan respons cepat.

4.3.5 Perbandingan Akurasi Output (Deviasi)

Akurasi *output* dievaluasi berdasarkan nilai deviasi antara output yang diprediksi model dengan nilai target, diukur per kolom variabel output. Nilai deviasi yang lebih kecil menunjukkan akurasi yang lebih tinggi. Perbandingan ini dilakukan antara skenario terbaik dari masing-masing metode: Skenario **e** untuk SLSQP dan Skenario **d** untuk GA.

Tabel 4.19: Perbandingan Deviasi Output

Metode	metal_temp	ni_met	c_met	si_met	fe_met	s_met	ni_slag	fe_slag	t_kalsin	pic_161	loi_kalsin	AVG
SLSQP	19.59	0.85	0.21	0.18	0.69	0.07	0.00	1.02	31.23	0.57	0.11	4.96
GA	18.65	0.50	0.12	0.12	0.38	0.08	0.00	0.61	29.48	0.59	0.10	4.69



Gambar 4.16: Deviasi Output - SLSQP vs GA

Pada metrik akurasi output (deviasi), metode GA menunjukkan performa yang secara konsisten lebih baik dibandingkan SLSQP. Berdasarkan Tabel 4.19 dan Gambar 4.16, rata-rata deviasi output GA adalah **4.69**, yang lebih rendah dibandingkan SLSQP dengan rata-rata deviasi **4.96**.

Secara spesifik, GA memiliki nilai deviasi yang lebih rendah pada sebagian besar variabel output penting, termasuk `metal_temp`, `ni_met`, `c_met`, `si_met`, `fe_met`, `ni_slag`, `fe_slag`, `t_kalsin`, dan `loi_kalsin`. Meskipun SLSQP menunjukkan deviasi yang sedikit lebih rendah pada `s_met` dan `pic_161`, keunggulan GA pada variabel-variabel inti dan rata-rata deviasi keseluruhan menegaskan bahwa solusi yang dihasilkan oleh GA lebih akurat dalam memprediksi output yang diinginkan dan menjaga kualitas produk.

4.3.6 Perbandingan Skor Akhir Antar Metode

Untuk memberikan rekapitulasi performa keseluruhan, sebuah skor akhir dihitung berdasarkan perbandingan relatif pada metrik-metrik utama yang telah dibahas sebelumnya. Angka 1 menunjukkan bahwa metode tersebut lebih unggul pada metrik tertentu, dan 0 menunjukkan sebaliknya. Bobot yang diterapkan pada setiap metrik mencerminkan prioritas industri, yaitu: Efisiensi (0.4), Durasi (0.05), Kewajaran (0.15), Pelanggaran (0.25), dan Deviasi (0.15). Total skor merupakan penjumlahan dari nilai terbobot keunggulan pada setiap metrik.

Tabel 4.20: Rekapitulasi Perbandingan Metode

Metode	Efisiensi	Pelanggaran	Durasi	Kewajaran	Akurasi	Total
SLSQP	0	1	1	1	0	3
GA	1	0	0	0	1	2

Berdasarkan Tabel 4.20, metode SLSQP secara keseluruhan menunjukkan keunggulan dengan total skor **3**, sementara GA memiliki total skor 2. Perbandingan ini menyoroti trade-off utama antara kedua metode:

SLSQP unggul pada metrik-metrik yang krusial untuk implementasi praktis di industri. Angka 1 pada kolom 'Pelanggaran' (bobot 0.25) berarti SLSQP menghasilkan solusi yang sepenuhnya valid dan dapat diterapkan tanpa melanggar batasan operasional. Angka 1 pada kolom 'Durasi' (bobot 0.05) menunjukkan kecepatan komputasi yang superior, vital untuk pengambilan keputusan real-time. Selain itu, angka 1 pada kolom 'Kewajaran' (bobot 0.15) memastikan bahwa solusi yang diberikan realistis dan sesuai dengan kondisi historis proses.

Di sisi lain, GA menunjukkan keunggulan pada 'Efisiensi' (angka 1, bobot 0.4) dan 'Akurasi' (output deviasi) (angka 1, bobot 0.15). Ini berarti GA memiliki potensi untuk mencapai penghematan energi yang lebih besar dan prediksi kualitas output yang lebih akurat.

Secara keseluruhan, trade-off yang diperlihatkan adalah antara optimasi yang berfokus pada kepatuhan constraint, kecepatan, dan kelayakan operasional (SLSQP), versus optimasi yang lebih agresif terhadap efisiensi energi dan akurasi prediksi model (GA). Mengingat prioritas industri feronikel yang mengutamakan stabilitas operasional, validasi hasil yang ketat (tanpa pelanggaran constraint), dan respons cepat, total skor yang lebih tinggi pada SLSQP menjadikannya pilihan yang lebih kuat dan pragmatis untuk implementasi nyata.

4.4 Dampak terhadap Lingkungan, Masyarakat, dan/atau Bidang Lain

Implementasi optimisasi parameter proses dalam industri feronikel, sebagaimana dilakukan dalam penelitian ini melalui pendekatan Sequential Least Squares Programming (SLSQP) dan Genetic Algorithm (GA), memiliki dampak signifikan yang tidak hanya terbatas pada efisiensi operasional, tetapi

juga mencakup aspek lingkungan, sosial, serta kontribusi terhadap perkembangan industri dan teknologi secara lebih luas.

4.4.1 Dampak terhadap Lingkungan

Optimisasi proses produksi dalam industri feronikel berdampak langsung pada pengurangan konsumsi energi dan emisi. Hal ini menjadikan aspek lingkungan sebagai salah satu penerima manfaat utama dari implementasi model optimisasi yang dirancang.

- **Pengurangan Emisi**

Salah satu tujuan utama optimisasi dalam penelitian ini adalah menurunkan konsumsi batubara (`total_fuel` dan `reductor_consume`) serta energi listrik. Keberhasilan optimisasi ini secara langsung berkontribusi terhadap penurunan emisi CO_2 dan jejak karbon operasional.

- **Efisiensi Penggunaan Sumber Daya**

Optimisasi memungkinkan penentuan konfigurasi operasional terbaik untuk meminimalkan pemborosan bahan baku dan energi. Dengan menurunnya konsumsi bahan bakar, tekanan terhadap sumber daya alam dapat dikurangi, sekaligus mendukung praktik industri yang lebih bijak dan berkelanjutan.

4.4.2 Dampak Sosial dan Kemasyarakatan

Selain keuntungan teknis dan lingkungan, optimisasi parameter proses juga memiliki potensi dampak positif bagi masyarakat dan aspek sosial-ekonomi industri.

- **Efisiensi Biaya dan Peningkatan Daya Saing**

Penurunan konsumsi energi dan bahan bakar tidak hanya berdampak pada aspek teknis, tetapi juga membawa penghematan biaya produksi. Efisiensi ini dapat meningkatkan daya saing industri feronikel.

- **Potensi Perbaikan Lingkungan Sekitar**

Penurunan emisi dan limbah industri turut berkontribusi terhadap peningkatan kualitas udara dan lingkungan di sekitar fasilitas produksi. Meskipun dampak ini tidak menjadi fokus utama penelitian, manfaat tidak langsung bagi masyarakat lokal tetap layak untuk dicatat.

4.4.3 Dampak terhadap Industri dan Teknologi

Penelitian ini juga memberikan kontribusi terhadap kemajuan praktik industri dan penerapan teknologi cerdas yang semakin relevan di era transformasi digital industri.

- Penerapan Teknologi Cerdas di Industri Berat

Penelitian ini membuktikan bahwa metode optimisasi berbasis *machine learning* seperti SLSQP dan GA dapat diterapkan secara efektif untuk menyelesaikan permasalahan kompleks di sektor industri berat, membuka peluang penerapan serupa pada sektor lain.

- Pendekatan Pengambilan Keputusan Berbasis Data

Integrasi model prediktif ke dalam sistem optimisasi memberikan alat bantu yang kuat bagi pengambil keputusan dalam operasional pabrik. Pendekatan ini menggantikan metode tradisional *trial and error* dengan metode berbasis data yang lebih sistematis dan terukur.

BAB 5

KESIMPULAN DAN SARAN

Bab ini merangkum temuan-temuan kunci dari penelitian optimisasi proses produksi feronikel menggunakan model prediktif *machine learning* serta algoritma *Genetic Algorithm* (GA) dan *Sequential Least Squares Programming* (SLSQP). Berdasarkan hasil analisis yang komprehensif, kesimpulan utama akan dijabarkan, diikuti dengan saran untuk penelitian di masa mendatang dan potensi implementasi di industri.

5.1 Kesimpulan

Penelitian ini berhasil menerapkan pendekatan optimisasi berbasis *machine learning* untuk meningkatkan efisiensi konsumsi energi dalam proses produksi feronikel melalui sistem *Rotary Kiln Electric Furnace* (RKEF), sambil menjaga kualitas metalurgi produk. Beberapa kesimpulan kunci yang dapat ditarik adalah sebagai berikut:

(a) Performa Metode Optimisasi

Berdasarkan skenario terbaik masing-masing metode (Skenario d untuk GA dan Skenario e untuk SLSQP), perbandingan performa menunjukkan karakteristik yang berbeda:

- Genetic Algorithm (GA)

GA (Skenario d) menunjukkan kemampuan yang lebih baik dalam mencapai efisiensi energi yang signifikan (total efisiensi 1.8491 Ton/jam) dan menghasilkan akurasi output (deviasi) yang sangat tinggi (rata-rata deviasi 4.69). Namun, GA memiliki kelemahan signifikan dalam kewajaran solusi input (rata-rata outlier 40.27) dan durasi komputasi yang jauh lebih lama (365.775 detik) dibandingkan SLSQP. Dalam skenario terbaiknya, GA mampu menghasilkan solusi tanpa pelanggaran constraint.

- Sequential Least Squares Programming (SLSQP)

SLSQP (Skenario e) unggul mutlak dalam efisiensi waktu komputasi (23.800 detik) dan kewajaran solusi (input) (rata-rata out-

lier 19.00). Metode ini juga menghasilkan solusi tanpa pelanggaran constraint. Namun, efisiensi energi (total efisiensi 1.4786 Ton/jam) dan akurasi output (rata-rata deviasi 4.96) cenderung sedikit lebih rendah dibandingkan GA.

(b) Efektivitas Pendekatan *Constraint*

Skenario yang menggunakan **batasan dinamis berbasis referensi ilmiah dengan toleransi statistik** terbukti menjadi konfigurasi constraint paling efektif untuk kedua metode optimisasi. Untuk GA, ini adalah Skenario d, sementara untuk SLSQP adalah Skenario e. Pendekatan ini berhasil menyeimbangkan antara mendorong efisiensi, menjaga kewajaran solusi, dan memastikan akurasi output, sekaligus meminimalkan pelanggaran constraint. Skenario yang tidak memiliki batasan maksimum atau menggunakan batasan terlalu longgar cenderung menghasilkan solusi yang boros energi atau tidak realistis.

(c) *Trade-off* antara Metode

Terdapat trade-off yang jelas antara GA dan SLSQP. GA lebih agresif dalam mencari solusi yang sangat efisien dan akurat pada output produk, namun memiliki kelemahan dalam kewajaran input dan membutuhkan waktu komputasi yang lebih lama. Sebaliknya, SLSQP memprioritaskan kecepatan, kewajaran solusi input, dan kepatuhan constraint (mampu menghasilkan solusi tanpa pelanggaran constraint), menjadikannya cocok untuk implementasi praktis yang sensitif terhadap batasan operasional dan waktu.

(d) Rekomendasi Implementasi

Mengingat prioritas industri feronikel yang mengutamakan stabilitas operasional, kepatuhan terhadap constraint, dan respons cepat, **metode SLSQP dengan skenario e** yaitu memberikan batas atas pada kolom korelasi kuat dengan `total_fuel` dan `reductor_consume` menggunakan median (Q2) merupakan pilihan yang **lebih pragmatis dan direkomendasikan** untuk implementasi nyata. Meskipun GA menawarkan efisiensi energi yang lebih tinggi dan akurasi output yang lebih baik, tantangan dalam kewajaran input dan durasi komputasi dapat menjadi pertimbangan signifikan di lingkungan industri yang membutuhkan solusi yang realistis dan cepat.

5.2 Saran

Berdasarkan kesimpulan di atas, berikut adalah beberapa saran untuk pengembangan dan penelitian lebih lanjut:

(a) Penyempurnaan Fungsi Objektif

Untuk penelitian selanjutnya, disarankan untuk mengimplementasikan fungsi objektif yang secara eksplisit meminimalkan konsumsi energi (*total_fuel* dan *reductor_consume*) sebagai target utama optimisasi. Parameter kualitas produk (*ni_met*, *c_met*, dll.) kemudian dapat ditetapkan sebagai constraint non-linier untuk memastikan standar kualitas tetap terpenuhi. Hal ini akan secara langsung menargetkan tujuan efisiensi.

(b) Validasi dengan Data *Real-time* dan Uji Lapangan

Penelitian ini menggunakan data historis. Untuk validasi yang lebih kuat, disarankan untuk menguji solusi optimasi yang dihasilkan pada data *real-time* atau, idealnya, melalui uji coba terbatas di lingkungan pabrik (jika memungkinkan). Ini akan memberikan gambaran langsung mengenai kelayakan dan dampak solusi dalam kondisi operasional sebenarnya.

(c) Eksplorasi Algoritma *Machine Learning* Lain

Meskipun *Ridge Regression* menunjukkan performa yang baik, eksplorasi model *machine learning* lain seperti *Neural Networks*, *Gradient Boosting Machines*, atau *Support Vector Machines* dengan *hyperparameter tuning* yang lebih mendalam dapat dilakukan untuk meningkatkan akurasi model prediktif, yang pada gilirannya akan memengaruhi kualitas hasil optimisasi.

(d) Penanganan *Constraint* yang Lebih Canggih

Untuk metode GA, dapat dipertimbangkan mekanisme penanganan *constraint* yang lebih canggih, seperti *feasible region repair* atau *multi-objective optimization* (di mana *constraint* kualitas menjadi tujuan kedua), untuk mengurangi jumlah pelanggaran tanpa mengorbankan kemampuan eksplorasi.

(e) Pertimbangan Variabel Eksternal

Penelitian di masa depan dapat memperluas ruang lingkup dengan mempertimbangkan variabel eksternal yang memengaruhi biaya dan operasional, seperti fluktuasi harga batubara, tarif listrik, atau bahkan faktor

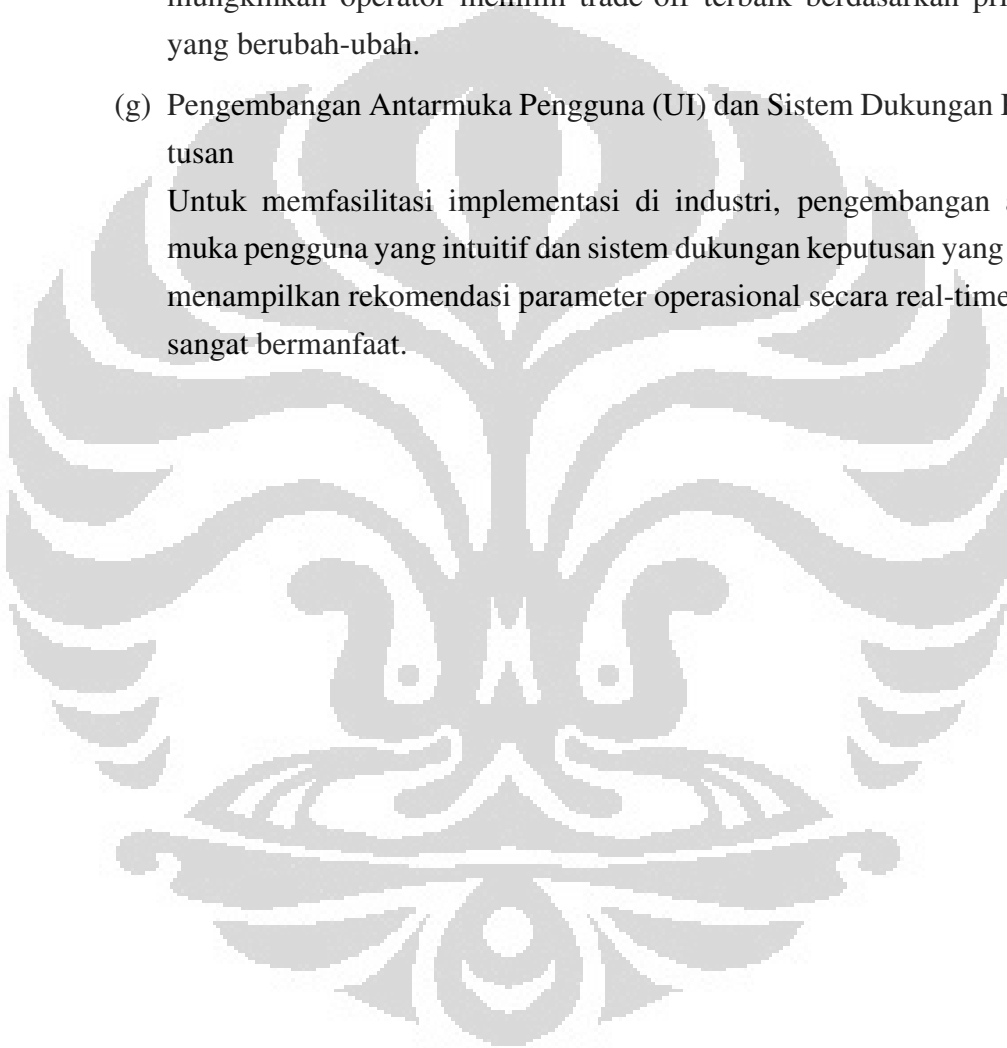
lingkungan seperti suhu ambien, untuk menghasilkan solusi optimisasi yang lebih adaptif dan robust.

(f) Optimisasi Berbasis Multi-Objektif

Jika ada beberapa tujuan yang saling bertentangan (misalnya, efisiensi energi versus kualitas produk yang sangat tinggi), pendekatan optimisasi multi-objektif (misalnya, menggunakan algoritma NSGA-II) dapat diterapkan untuk menghasilkan seperangkat solusi Pareto front yang memungkinkan operator memilih trade-off terbaik berdasarkan prioritas yang berubah-ubah.

(g) Pengembangan Antarmuka Pengguna (UI) dan Sistem Dukungan Keputusan

Untuk memfasilitasi implementasi di industri, pengembangan antarmuka pengguna yang intuitif dan sistem dukungan keputusan yang dapat menampilkan rekomendasi parameter operasional secara real-time akan sangat bermanfaat.



DAFTAR REFERENSI

- [1] Song Yanning, Hongxia Qiao, Feng Qiong, Wei Chao, and Zheng Jianghua. Application of metallurgical ferronickel slag in building materials: A review. *Journal of Building Engineering*, 96:110632, 2024.
- [2] Achim Ahrens, Christian Hansen, and Mark Schaffer. lassopack: Model selection and prediction with regularized regression in stata, 01 2019.
- [3] Pauline H Pattyranie Tan. Macroeconomic analysis of the indonesian nickel industry using depest analysis. *Ultima Management: Jurnal Ilmu Manajemen*, 14(2):308–320, 2022.
- [4] Michael David Harmse, Jean Herman van Laar, Wiehan Adriaan Pelser, and Cornelius Stephanus Lodewyk Schutte. Integration of machine learning with complex industrial mining systems for reduced energy consumption. *Frontiers in Artificial Intelligence*, 5:938641, 2022.
- [5] Manuel Saldaña, Edelmira Gálvez, Alessandro Navarra, Norman Toro, and Luis A Cisternas. Optimization of the sag grinding process using statistical analysis and machine learning: a case study of the chilean copper mining industry. *Materials*, 16(8):3220, 2023.
- [6] Peng Liu, Baokuan Li, Sherman C.P. Cheung, and Wenyan Wu. Material and energy flows in rotary kiln-electric furnace smelting of ferronickel alloy with energy saving. *Applied Thermal Engineering*, 109:542–559, 2016.
- [7] Peng Liu, Baokuan Li, Sherman CP Cheung, and Wenyan Wu. Material and energy flows in rotary kiln-electric furnace smelting of ferronickel alloy with energy saving. *Applied Thermal Engineering*, 109:542–559, 2016.
- [8] Christian Janiesch, Patrick Zschech, and Kai Heinrich. Machine learning and deep learning. *Electronic Markets*, 31(3):685–695, 2021.
- [9] Amer FAH Alnuaimi and Tasnim HK Albaldawi. An overview of machine learning classification techniques. In *BIO Web of Conferences*, volume 97, page 00133. EDP Sciences, 2024.

- [10] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2013.
- [11] D. Hosmer and S. Lemeshow. *Applied logistic regression*. John Wiley Sons, Inc, New York, 2000.
- [12] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, August 2004.
- [13] L. Breiman, J. Friedman, C.J. Stone, and R.A. Olshen. *Classification and Regression Trees*. Taylor & Francis, 1984.
- [14] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [15] Arthur E Hoerl and Robert W Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [16] Christine De Mol, Domenico Giannone, and Lucrezia Reichlin. The asymptotic equivalence of ridge and principal component regression with many predictors. *Econometrics and Statistics*, 2024.
- [17] Jörg Stork, Agoston E Eiben, and Thomas Bartz-Beielstein. A new taxonomy of global optimization algorithms. *Natural Computing*, 21(2):219–242, 2022.
- [18] Arnold Neumaier. Complete search in continuous global optimization and constraint satisfaction. *Acta numerica*, 13:271–369, 2004.
- [19] Gerhard J Woeginger. Open problems around exact algorithms. *Discrete Applied Mathematics*, 156(3):397–405, 2008.
- [20] Eugene L Lawler and David E Wood. Branch-and-bound methods: A survey. *Operations research*, 14(4):699–719, 1966.
- [21] Judea Pearl. *Heuristics: intelligent search strategies for computer problem solving*. Addison-Wesley Longman Publishing Co., Inc., 1984.
- [22] Ilhem Boussaïd, Julien Lepagnot, and Patrick Siarry. A survey on optimization metaheuristics. *Information sciences*, 237:82–117, 2013.
- [23] Jacob Søndergaard. Optimization using surrogate models-by the space mapping technique. 2003.

- [24] Jörg Stork, Martin Zaefferer, Thomas Bartz-Beielstein, and AE Eiben. Surrogate models for enhancing the efficiency of neuroevolution in reinforcement learning. In *Proceedings of the genetic and evolutionary computation conference*, pages 934–942, 2019.
- [25] Robert E Mercer and JR Sampson. Adaptive search using a reproductive meta-plan. *Kybernetes*, 7(3):215–228, 1978.
- [26] Peter Cowling, Graham Kendall, and Eric Soubeiga. Hyperheuristics: A tool for rapid prototyping in scheduling and optimisation. In *Applications of Evolutionary Computing: EvoWorkshops 2002: EvoCOP, EvoIASP, EvoSTIM/EvoPLAN Kinsale, Ireland, April 3–4, 2002 Proceedings*, pages 1–10. Springer, 2002.
- [27] Ender Özcan, Burak Bilgin, and Emin Korkmaz. A comprehensive analysis of hyper-heuristics. *Intelligent Data Analysis*, 12:3–23, 02 2008.
- [28] Peter Cowling, Graham Kendall, and Eric Soubeiga. A hyperheuristic approach to scheduling a sales summit. In *Practice and Theory of Automated Timetabling III: Third International Conference, PATAT 2000 Konstanz, Germany, August 16–18, 2000 Selected Papers 3*, pages 176–190. Springer, 2001.
- [29] Prapa Rattadilok, Andy Gaw, and Raymond S. K. Kwan. Distributed choice function hyper-heuristics for timetabling and scheduling. In Edmund Burke and Michael Trick, editors, *Practice and Theory of Automated Timetabling V*, pages 51–67, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [30] Yarens J. Cruz, Alberto Villalonga, Fernando Castaño, Marcelino Rivas, and Rodolfo E. Haber. Automated machine learning methodology for optimizing production processes in small and medium-sized enterprises. *Operations Research Perspectives*, 12:100308, 2024.
- [31] Kit-Sang Tang, Kim-Fung Man, Sam Kwong, and Qianhua He. Genetic algorithms and their applications. *IEEE signal processing magazine*, 13(6):22–37, 1996.
- [32] Jupeng Xie, Huajun Zhang, Yu Shen, and Mengchuan Li. Energy consumption optimization of central air-conditioning based on sequential-least-square-programming. In *2020 Chinese control and decision conference (CCDC)*, pages 5147–5152. IEEE, 2020.

- [33] Xianyi Li. The optimization method of cnc lathe performance based on morris sensitivity analysis and improved ga algorithm. *Journal of Vibroengineering*, 26, 03 2024.
- [34] C.L. Hwang and K. Yoon. *Multiple Attribute Decision Making: Methods and Applications : a State-of-the-art Survey*. Lecture notes in economics and mathematical systems. Springer-Verlag, 1981.

